Loom: A Distributed Expert-Cache Architecture for Frontier Mixture-of-Experts Models

Technical design and preliminary validation

Author: Parthasarathy Ramanujam **Draft v0.3, 2026-08-01** **Status:** design complete for v1; v0 single-node streaming done; v1 first cuts validated on a real three-machine swarm (sharded serving, network-id isolation, RAG gossip, churn repair); Metal and Vulkan GPU backends measured on real silicon; devnet (Qwen3-30B-A3B) live. Implementation progress is tracked in ROADMAP. The wire protocol is specified in SPEC. A dated design-decision log for contributors is in Appendix A.

Contents

Abstract

The strongest open-weight language models are now Mixture-of-Experts (MoE) architectures: Kimi K2 [3], DeepSeek V3 [2], and the GLM family [4], with hundreds of billions to trillions of parameters. At usable quantization their routed experts alone run to hundreds of gigabytes. They do not fit on any commodity machine, and running them today means renting datacenter GPUs that price out individuals and small teams. Multi-machine alternatives exist, but the prevailing approach splits the computation across nodes and ships activations between them every layer of every token. That places the network on the serial critical path, so a slow or lost peer stalls the whole pipeline. The problem Loom addresses is how to serve a frontier MoE model that no single affordable machine can hold, across a pool of ordinary machines on ordinary networks, without putting the network in the critical path of every token.

The opportunity is structural. An MoE model activates only a few percent of its weights per token, and those weights form an immutable, content-addressable corpus: a sparsely accessed blob store rather than a monolithic tensor. Loom keeps computation node-local and moves only weights. The corpus is sharded into content-addressed expert blocks, replicated across coordinated committees of nodes, verified by Merkle digests at every hop, and paged into the token loop on demand. Because only immutable weights cross the network, a lost peer costs a re-fetch from a replica rather than a broken pipeline.

What has been demonstrated. The inference engines produce correct output on reference checkpoints: the tinyllamas models, and DeepSeek-V2-Lite, a 15.7-billion-parameter MoE, at Q4_K_M quantization. A node holding

33% of that model's shards produced the expected, token-identical completion while fetching the missing experts from a single LAN peer during inference, with every block digest-verified before use. The same partial-store loop has since been demonstrated across real machines: a two-machine run over a wide-area link (25.8 ms RTT), and a three-machine swarm spanning two continents with nodes holding 100%, 33%, and 20% of the shards, one of them a NAT'd laptop, serving through the node's RPC path with every block digest-verified (Section 10).

What is targeted, and not yet demonstrated. The design goal is serving GLM-5.2-class models (744 billion parameters, about 19,200 experts) on swarms of 16 to 32 GB machines. That model has not been run, and there is no continuous batching yet. The kernels are no longer the boundary: the CPU path is SIMD with int8 activations, and the Metal and Vulkan GPU backends have been measured on real silicon (Section 10). One caveat governs expectations throughout (Section 9): on ordinary gigabit Ethernet, distribution buys capacity, meaning the model fits where it otherwise could not, but not speed. Competitive throughput additionally requires 10-gigabit-class fabric, a pinned hot set, and batched serving.

Glossary

Term	Meaning
Expert	One routed feed-forward sub-network (gate/up/down matrices) in one MoE layer. GLM 5.2 has 256 per layer; 8 activate per token.
Expert shard	The unit of distribution and compute: one expert's weights, about 19 MB at int4. A fetched expert shard is exactly what a MoE matmul consumes.
Resident chunk	A roughly 16 MB piece of the resident bundle, the non-expert weights (attention, embeddings, shared experts, router, norms) that every full node holds in full.
Manifest / version id	The list of shard digests plus layout. Its Merkle root is the model's version identity, which nodes pin requests to.
Full node	Holds the resident bundle and a subset of expert shards, runs local inference, serves shards to peers. The supply side.
Light node	Holds no weights and runs no engine; exposes the same local API and delegates to full nodes. The demand side.
Committee	A group of full nodes that collectively holds the complete shard set, a self-sufficient serving cell.
MLA	Multi-head Latent Attention [1]: attention with a low-rank compressed key/value cache (about 576 values per token per layer), used by the target models.
Mesh	The peer table each node builds from gossip announcements, used for discovery and fallback routing.

---

1. Introduction

Loom targets operators who want to run frontier open-weight MoE models on hardware they already have: a homelab, a research group's workstations, or a small trusted cluster, rather than rented datacenter GPUs. It is not built for interactive single-user chat on a cold model over a slow link (Section 9 explains why), and v1 assumes a cooperative, operator-run swarm rather than an open adversarial network (Section 7).

Three facts frame the problem.

1. **The best open models are MoE.** Kimi K2 (about 1T total, 32B active), DeepSeek V3 (671B, 37B), and the GLM family activate roughly 3 to 5 percent of their parameters per token [2][3][4].
2. **The weights do not fit cheaply.** At int4, the routed experts of a GLM-5.2-class model total about 370 GB, beyond any commodity machine and beyond most workstations.

3. **The activated set is small and skewed.** A single token reads on the order of 600 expert blocks (about 11.4 GB for GLM 5.2), drawn Zipf-like from a fixed corpus of roughly 19,200. This is a sparsely accessed, immutable, content-addressable blob store.

Loom's premise follows. This is a distributed storage and caching problem in the shape of an inference problem. The corpus is stored once across a swarm, replicated by policy, cryptographically verified, and paged into computation on demand, the way a distributed filesystem treats cold blocks rather than the way a tensor-parallel runtime treats a partitioned weight matrix.

---

2. Related work

Systems that run large models across multiple machines can be organized by their unit of distribution and the failure and trust properties that follow from it.

System	Unit distributed	What crosses the network per token	Failure model	Commodity-NIC friendly
Petals [5]	Transformer layers	Activations, every layer	Serial: a slow or lost stage stalls the token	Partially (activations are small but on the critical path)
exo [6]	Layer partitions	Activations	Serial pipeline	Partially
FlexGen [9]	Offloaded tensors (single node)	None (host/device)	N/A (not distributed)	N/A
PowerInfer [8]	Hot/cold neurons (single node)	None (GPU/CPU split)	N/A	N/A
llama.cpp [7]	None (single node) or layer-split	Activations if split	Serial if split	Yes single-node
FreeToken [20]	Expert residency (single node, VRAM over host RAM)	None (PCIe/host split)	N/A (not distributed)	N/A
LocalAI [15]	Whole model (federated) or llama.cpp RPC layer-split	Requests, or activations if split	Serial if split; federated needs a full replica per node	Yes federated, but no capacity gain
Loom	**Expert block (weights)**	**Expert blocks only** (cacheable, immutable)	**Soft: re-fetch from a replica**	**Yes for capacity; speed needs 10 GbE**

Loom reuses the GGUF weight format and GGML quantization layouts [7] and the DeepSeek MLA and sparse-routing model shape [1][2]. It introduces three things: the expert block as the unit of distribution, committee-based redundancy with coverage by construction, and a token-loop fetch that doubles as replication. Its closest philosophical relative is content-addressed storage (Merkle-verified immutable blocks [10]) applied to model weights rather than files.

FreeToken [20] is the same problem one tier down the memory hierarchy: it treats a single machine's VRAM as a small elastic expert cache over a host-RAM expert pool, where Loom treats a machine's RAM as a cache over a swarm-wide pool. Its explicit assumption is that the complete expert pool fits in one machine's host memory; Loom removes exactly that assumption -- the swarm is the host pool -- and the two designs stack naturally. Two of its mechanisms port across the tier boundary and are adopted here (decision log, 2026-08-23): measured-bandwidth division of miss work (its q* policy, reborn as bandwidth-weighted holder selection) and routing-blind full-layer prefill streaming.

Layer-split systems such as Petals and exo are deployed and effective. The distinction Loom draws is structural, not a claim that they do not work. Because they ship activations, which are mutable, per-request, and orderdependent, the network sits on the serial critical path. Loom ships only immutable weights, so a lost peer costs a re-fetch rather than a broken pipeline.

LocalAI [15] deserves its own note because it is the most complete self-hosted OpenAI-compatible server and it does run across machines, in two modes. Its federated mode load-balances whole requests over libp2p, which multiplies throughput but requires every node to hold the entire model; a 370 GB corpus must fit on each box, so aggregate capacity does not grow. Its worker mode reuses llama.cpp's RPC backend to split one model layer- or tensor-wise across machines, which is the activation-shipping architecture of the rows above: each worker must hold its slice fully resident, and a slow or lost worker stalls every token until it is replaced. Neither mode offers a partial store, integrity verification of weights in flight, or churn repair. LocalAI is also much broader than Loom (audio, image, and embedding endpoints over many backends); the comparison here concerns only the multi-machine text-inference axis.

---

3. Core thesis: experts flow, not activations

Every full node runs the complete forward pass locally. The only data that crosses the network during inference is an expert block: immutable, content-addressed, cacheable, and identical for every user of the model. Three properties follow.

- **Soft failure.** A lost peer costs a re-fetch from a replica, not a broken pipeline. A lone node still functions, streaming experts from its own disk.
- **Compounding cache.** Frequently used experts come to reside on many nodes, so the network is touched only on the cold tail of the access distribution.
- **KV stays home.** The target models use MLA [1], which compresses the key/value cache to about 576 values per token per layer. Distributing that mutable, latency-critical, per-session state would put it on the network for a negligible capacity gain, so Loom does not.

The governing caveat, developed in Section 9: on gigabit Ethernet the swarm buys capacity, not speed. Loom is a serving-throughput architecture first and an interactive-latency one second.

Figure 1. Inference data path (one full node). Only routed-expert reads may leave the machine; everything else is local.

```
prompt
  |
  v
embed --> for each layer:  RMSNorm -> MLA attention -> (local KV cache)
                                          |
                                          v
                             router -> top-k experts --+
                                                       | held?  -- yes --> local pread --+
                                                       v                                 v
                                       expert access tier          -> SwiGLU matmul -> residual
                                                       | no                               ^
                                                       +--> fetch from peer --> verify ----+
                                                            (committee then mesh)  + persist
  |
  v
final norm -> logits -> sampled token
```

---

4. System architecture

Loom is a single binary written in Zig. It separates a data plane (weights at rest and in flight, and the local forward pass) from a control plane (membership, gossip, repair, metering). Two inference engines share a common set of quantized-matmul kernels.

- The **MLA** engine (`deepseek2`), covering the DeepSeek/Kimi family: MLA attention, YaRN context extension [11], and a byte-level BPE tokenizer read from GGUF metadata. Validated on real DeepSeek-V2-Lite weights (Q4_K_M).

- The **GQA** engine, covering `llama` (including Mixtral), `qwen2moe`, `qwen3moe`, `glm4moe` and `gpt-oss` in one forward pass. These architectures differ only in optional pieces over a shared skeleton, so the engine detects them from the tensors a file contains rather than from per-architecture assumptions: QKV biases, per-head Q/K normalization, dense or mixture-of-experts FFN, an optionally sigmoid-gated shared expert, leading dense layers, a post-attention norm in place of `ffn_norm`, trailing MTP blocks that are skipped, and the gpt-oss family's additions -- alternating sliding-window layers, per-head attention sinks, per-expert FFN biases with the clamped `swiglu_oai` activation, YaRN-scaled NEOX rope, and a select-then-softmax router. Only the rotary-embedding style is pinned per architecture, because an incorrect choice yields fluent but wrong output rather than an error. The dense llama path is validated against the tinyllamas reference models at F32, Q4_0 and Q8_0.

Both engines share one router (sigmoid or softmax gating, selection bias, top-k with optionally renormalized and scaled gates, shared experts), so a new architecture requires an attention variant rather than a new engine. The distribution plane was never architecture-specific: expert sharding keys on the GGUF tensor-naming convention that every mixture-of-experts conversion follows.

Weights remain in their GGML quantized formats in a read-only memory map, and each matmul is a fused, vectorized kernel over the raw bytes, distributed across a worker pool by row, so no tensor is dequantized wholesale. Two families are supported. *Affine* formats (F32/F16/Q4_0/Q5_0/Q8_0/Q2_K/Q3_K/Q4_K/Q5_K/Q6_K) reconstruct a value arithmetically. *Codebook* formats (IQ1_S, IQ1_M, IQ2_XXS, IQ2_XS, IQ2_S, IQ3_XXS, IQ3_S, IQ4_NL, IQ4_XS, and MXFP4) instead store an index into a static grid table, so a transcription error in that table would silently corrupt weights rather than fail; every codebook decoder is therefore checked bit-for-bit against the reference implementation's output on stored vectors. The source layout appears in Appendix B.

Shared retrieval plane (RAG). Alongside weights, a node can carry a swarm- shared store of text chunks for retrieval-augmented generation (`--rag`). A chunk ingested on any node (via `POST /v1/rag/chunks`) reaches the whole network by gossip: nodes advertise a rotating window of chunk hashes, pull what they lack, and push what a peer wants, with a pull direction so a NAT'd node that can only dial out still receives everything. Only text travels. This works because a network serves exactly one model (Section 6's network id), so every node computes the identical embedding (mean-pooled token embeddings, L2-normalized) from the same text; vectors never cross the wire and cannot disagree. Chunks are deduplicated by content hash, stored brotli-compressed at rest, and searched before each generation; retrieved chunks are prepended to the prompt as context.

What this buys the swarm is a knowledge base with the same economics as the weight store: ingest once anywhere, available everywhere, no central index. On the three-machine run (Section 10), facts ingested only on the NAT'd laptop were answered correctly by the origin, where the same question minutes earlier produced a hallucination. The trust model matches v1's: a peer cannot corrupt a chunk in flight (content addressing), but nothing yet vets what the text says; v2's provenance work applies here too. Chunks currently never expire. Growth is bounded by a store cap, and the planned mechanism for removal is an optional ingester-stamped expiry in the chunk envelope, which expires consistently network-wide without tombstones; a bare delete cannot work under gossip, since any peer still holding the chunk would re-spread it.

---

5. Expert-aligned sharding

The unit of distribution equals the unit of computation. An expert shard is one expert's gate/up/down matrices in one layer, described as a byte-extent list over the GGUF file's three-dimensional expert tensors. A fetched expert shard is precisely what a MoE matmul consumes: no reassembly, no erasure coding, no partial reads on the token path.

Everything that is not a routed expert (attention, norms, shared experts, embeddings, router weights, the file header) forms the resident bundle, split into roughly 16 MB resident chunks that every full node holds in full. This is what keeps compute node-local. Any node can run the dense path and the router by itself; only routed-expert reads may leave the machine.

Every byte of the file belongs to exactly one shard, either an expert shard or a resident chunk. Each shard carries a SHA-256 digest, and the Merkle root over the digest list, together with the layout, is the model's version identity, which nodes gossip and pin their requests to. Integrity is therefore free at every hop: a shard from any source is checked against the local manifest before it reaches disk or a matmul.

Sizing for the GLM-5.2-class target:

Quantity	Value
Expert shards	19,200 (75 MoE layers x 256 experts)
Expert-shard size	about 19 MB (int4)
Routed corpus	about 370 GB, stored once across the swarm
Resident bundle	about 10 GB, on every full node
Logical file	about 380 GB; each node's copy is sparse (about 60 GB of real bytes at defaults)
Manifest / holdings bitmap	about 1 MB / 2.4 KB

Measured on real DeepSeek-V2-Lite Q4_K_M (10.4 GB): 1,664 expert shards of 4.98 to 6.02 MB, plus 73 resident chunks totaling 0.78 GB; manifest 204 KB; holdings bitmap 218 B.

---

6. Distribution protocol

This section states the protocol's shape and invariants. Field-level detail is in SPEC.

Component	Role	Key property
Bootnode	Onboards joiners; assigns each a committee and a least-covered-first want-set	Coverage by construction; not in the inference path; trusted in v1
Committee	A group of full nodes that collectively holds every shard	Completeness means at least one holder per shard; filled toward redundancy R (default 2)
Gossip mesh	Every node announces its holdings every 3 s; peers merge into a peer table	Transitive discovery; carries committee membership
Query path	Fetch a missing shard: committee members first, then the mesh; first choices spread by each holder's measured throughput (smooth weighted round-robin over per-peer EMAs)	Digest-verified; fails only if no reachable holder exists anywhere; a fast peer absorbs proportionally more of a layer's parallel misses so all finish together
Eager repair	A 2 s loop re-fetches any wanted-but-missing shard from all known peers	Dead peers are retried, not forgotten; a returning holder is drained within one tick
Striped repair	Systematic RS parity (k=8+m=2 per range group) rebuilds small holes from any full-group holders	Deterministic Cauchy parity; rebuilt shards digest-verified; propagation plane only
Liveness	First-hand contact only; 30 s TTL gates holder claims, never addresses	Hearsay cannot revive a dead peer, so death is detectable without committee membership
Wire messages	Binary frames (Heartbeat, Announce, ExpertRequest/Response) with adaptive Snappy [13]	Requests pin the version, so a cross-version peer is refused (the hardfork guard)

Two design choices hold the protocol together.

Coverage by construction. Rather than hope random placement covers every shard, the bootnode assigns each joiner the shards its committee currently covers least. Completeness (every shard has a holder) and redundancy (R holders) are distinct thresholds a committee crosses as it fills. When all committees are saturated at R, the next joiner opens a new one.

Membership is gossip-derived, not static. Announcements carry committee ids, so earlier members learn later joiners automatically and the committee view survives the bootnode's departure. Heartbeats (5 s) carry liveness plus the manifest version, a monotonic holdings sequence number, a bitmap digest, and a load hint. That is enough to detect staleness and spread load without shipping the full bitmap on every beat.

Liveness is first-hand only, and it gates holder claims rather than addresses. Two properties are easily conflated and must not be. An address is useful indefinitely; the eager-repair policy never forgets one, so a returning holder is picked up without re-bootstrapping. A *holdings claim* is only as good as the evidence that its owner still answers. Loom therefore tags every peer-table update with how it was learned: `first_hand` (we exchanged with that peer, or it connected to us) or `hearsay` (another peer listed it). Only first-hand contact refreshes a peer's last-seen time; a peer unheard-from for 30 s (ten gossip rounds) stops being offered as a holder while its address is retained.

Striped repair puts the first erasure coding in the propagation plane. Consecutive manifest ranges form groups of eight; each group defines two parity pieces under a systematic Reed-Solomon code over GF(2^8) whose Cauchy generator depends only on the geometry, so every honest holder computes byte-identical parity and pieces from different peers compose (the construction follows commonware's stripes design [19]). Both bootstrap and the repair loop end with a parity pass: a group missing at most two ranges is rebuilt from local data plus parity rows fetched round-robin across full-group holders, and the rebuilt ranges pass the same manifest-digest check as any fetched shard. That check is the whole trust story — a wrong or malicious parity piece costs one skipped group, never a corrupt store, which is why plain RS runs against untrusted peers here without the homomorphic-hash machinery that recoding (RLNC) demands. Two honest scope limits: under the pull model, a peer able to serve a group's parity necessarily holds its data and could serve that directly, so today the pass spreads load across holders rather than adding availability; the availability gain arrives with parity-only holders (an EC'd cold tier at 1.25x overhead instead of 2x replication) and coded push rollout, which reuse this wire format. And per principle 7 it stays out of the token loop: the per-token fetch path still reads only original, directly-addressed blocks.

Without the distinction, no death is detectable in a swarm of three or more, because the survivors keep echoing the dead node's entry to one another and each echo renews it. Measured on a three-node run, an origin holding the full model was killed and both survivors still advertised it as a live holder six minutes later. Nor can liveness be delegated to the committee heartbeat, which watches committee members only: the origin is the bootnode and sits in no committee, so the node that was the sole holder of 332 of 1,664 expert shards was monitored by nobody.

Under-replication is reported, not inferred. A node's status line names the number of shards fewer than R live peers hold. Redundancy is an arithmetic consequence of membership, not a setting that can be asserted: two joiners at `--hold-fraction 0.40` cover 80% of one copy and cannot reach R=2 however correctly the shards are assigned. Measured on the same run, exactly 332 shards sat at one holder under `--r-target 2`, and before this was surfaced nothing said so.

Figure 2. Committee and mesh.

```
       +----------+   JOIN gives a committee id and a least-covered want-set
join ->| BOOTNODE |   (out of the inference path)
       +----+-----+
            | manifest + initial shards
   +--------+-------- committee 0 (complete: every shard has >=1 holder) -------+
   |  full node A <--- 5 s heartbeat ---> full node B <--> full node C          |
   |   shards A..M        shard fetch          N..Z           A..K              |
   +---------------------------+-----------------------------------------------+
                               | global gossip mesh (3 s: addr, version,
                               | committee, holdings bitmap)
   +---------------------------+-------- committee 1, light clients -------------+
   |  full node D        full node E        light node (delegates to a full)    |
   +---------------------------------------------------------------------------+
```

7. Trust and security model

v1 assumes a cooperative, operator-run swarm and provides cryptographic integrity relative to a trusted manifest.

Assumptions. The bootnode is operated by the swarm operator, peers report their holdings honestly, and the transport is a trusted LAN.

Guarantees. Every shard is checked against its manifest digest before disk or matmul, so corruption, bit-rot, or a wrong-bytes peer is caught at every hop. The manifest's version id binds the shard layout, so a manifest whose extents do not tile the file exactly is rejected on parse. Version pinning on every request isolates model versions and, later, hardforks.

Non-guarantees. Which manifest is trusted is not free. Content addressing secures integrity within a manifest, not the choice of manifest. A node adopts its Merkle root on first contact (trust-on-first-use) or from operator config, and a hostile first contact could pin an alternate, internally consistent but poisoned root. Holdings claims are also unverified: a peer can advertise shards it lacks. This is caught reactively, when a fetch returns wrong or absent bytes and the requester falls through to the next peer, so committee completeness is a construction-time property under honest participation, not a cryptographic invariant. v1 does not defend against availability attacks (service refusal, mesh poisoning with fake holders, request flooding).

v2 (designed, not built): untrusted peers. The planned untrusted-peer layer adds rateless (RLNC) wide-area propagation gated behind homomorphic-hash pollution defenses [14], redundant recompute with M-of-N voting on sampled tokens, and TEE attestation. Erasure and network coding are confined to the propagation and durability planes; they are never permitted on the token path, where only direct fetch of an original block is allowed.

Known limitations of the current implementation are consolidated in Section 11.

---

8. Node classes and compensation

Loom defines two node classes.

Full nodes hold the resident bundle and their assigned expert shards, join committees, serve shards to peers, and run inference. They are the supply side. The "compute node-local, only expert blocks cross the network" property of Section 3 applies to the full-node inference data path; request, response, and control traffic naturally also cross the network.

Light nodes hold no weights, no store, and no engine, a footprint of a few megabytes suitable for a low-memory device. A light node exposes the same local API a full node does, so applications are unaware of the distinction, and delegates each request to a full node with round-robin failover across its configured backends. It stamps its own client identity on every request, and a caller-supplied identity is dropped. Light nodes are thin clients, not swarm participants: they contribute no storage or serving, so capacity planning must size the full-node fleet for aggregate light-node demand.

Compensation. Full nodes are meant to be compensated by light nodes for the inference they serve. v1 implements the accounting mechanics and leaves settlement as an integration seam. Each full node keeps a per-client ledger, where allowance equals free quota plus purchased credits minus token usage. A metered response carries its cost and the remaining balance, and an exhausted client is refused with `payment_required` before any compute. Credits are added through an admin-gated `credit` operation whose proof field a real payment rail would verify. This is scoped as a trusted-operator accounting demonstration, not a settlement system. Its limitations are enumerated in Section 11.

---

9. Deployment and performance model

Held shards are a disk budget served by `pread`. RAM carries only the compute working set.

Resource	Minimum	Recommended
RAM	16 GB (about 10 GB mmap'd resident bundle, 0.7 GB MLA KV, 2 to 3 GB expert cache)	24 to 32 GB
Disk	75 GB NVMe (resident bundle plus about 50 GB expert shards)	150 GB NVMe
Experts held	about 1,300 (25 GB)	about 2,600 (50 GB), 13.7% of GLM 5.2

Resource	Minimum	Recommended
Network	1 GbE (capacity only)	10 GbE+ (also latency)

The 16 GB minimum is conditional on the resident bundle. A per-tensor-class breakdown from the real converted GLM 5.2 file is required before it is asserted as a default rather than a target. For DeepSeek-V2-Lite the measured resident bundle is 0.78 GB.

Swarm sizing against redundancy R over the 370 GB corpus (R = 3 target, R = 2 floor): at 50 GB per node, completeness (R = 1) needs at least 8 nodes, R = 2 needs at least 15, and R = 3 needs at least 22. Eight nodes at 100 GB reach R = 2.

Performance model (analytical, not yet measured). Per-token time is about `t_compute + misses * t_fetch`, where `misses` is the number of a token's routed experts absent from the local store and page cache, and `t_fetch` is about `RTT + shard_size / bandwidth` per miss. Fetches within a layer parallelize, so the layer cost is the maximum, not the sum, over its 8 or fewer misses. A cold 19 MB miss costs roughly 160 ms on 1 GbE and 16 ms on 10 GbE before protocol overhead. The consequence is direct. With a cold working set on gigabit Ethernet a single stream is seconds per token, unusable interactively. Serving becomes viable insofar as pinning, the Zipfian access skew, and organic replication drive the steady-state miss rate toward zero, and batching amortizes what remains. The `t_compute` term has since been driven down by the SIMD CPU path and the Metal and Vulkan backends, each behind one compute interface (Section 10); a CUDA backend remains open. No network-tier throughput measurements exist yet; Section 10 lists them as the measurement agenda.

Positioning follows from the model: pool commodity machines so a frontier MoE fits where it otherwise could not; speed is a separate problem, bound by network and pinning. The differentiators against layer-split systems are structural (weights not activations, soft failure, verified weights, self-healing membership). The differentiator against single-box streaming is aggregate capacity.

10. Preliminary results

These are early results, not a full throughput evaluation, but they now span real machines and real links: single-machine runs, localhost multi-node runs, a two-machine wide-area run, and a three-machine swarm across two continents. The evidence and its boundaries:

Claim	Configuration	Evidence	Boundary
Engine correctness	DeepSeek-V2-Lite Q4_K_M; tinyllamas F32/Q4_0/Q8_0	Coherent, expected completions on real weights (validates kernels, MLA, RoPE convention, K-quants, BPE, YaRN)	Not a factual-accuracy benchmark; single-sequence
CPU kernel performance	TinyLlama 1.1B Q4_K_M, 10-core M5	57.8 tok/s, 53x the original scalar path (SIMD, 8 threads, int8 activations, vectorized attention); row-splitting bit-identical by test	One model size; one machine
GPU backends	DeepSeek-Coder-V2-Lite Q4_K_M	Metal (Apple M5): 39.7-44.4 tok/s single-stream decode, ~1.8 command buffers/token. Vulkan (RTX 3060): ~32-34 tok/s end-to-end, ~70 marginal, every ker-	Single-stream; three devices measured

Claim	Configuration	Evidence	Boundary
		nel oracle-pinned. Vulkan (RTX 3090): 8.0 ms median GPU frame (~125 tok/s marginal), no-barrier floor 4.9 ms, llama.cpp reference 172 tok/s -- the gap decomposes as ~2.2 ms kernel arithmetic + ~1 ms sync differential (both barrier-drain fusions measured slower; see decision log)	
Expert-aligned sharding	Real 10.4 GB model	Round-trips to 1,664 expert shards plus 73 resident chunks; layout partition property-tested	Fully covered
Partial-store inference	33% store (573 of 1,737 shards), localhost, one peer	Token-identical to a full-copy run while fetching 641 experts (3.5 GB, about 29 ms/fetch, 0 failures); grew to 1,214 shards held	Loopback link
Wide-area partial store	Two machines (Mac <-> Hetzner), 25.8 ms RTT, 29 MB/s	A node holding 3.6% of shards produced token-identical output, fetching 1,063 experts across the WAN, each digest-verified; also surfaced a bulk-sync framing failure that only a real link exposes	Throughput matches Section 9's bandwidth table: a WAN buys capacity, not speed
Three-machine swarm	Origin (100%), GPU node (33%), NAT'd laptop (20%); two continents; served through the node RPC path	Expert-sharded serving with manifest-root verification on all three; wrong-network-id node refused for the full 45 s it kept dialing; RAG chunks gossiped in every direction including the NAT pull path; facts ingested only on the laptop answered correctly by the origin (28 -> 81 prompt tokens with retrieved context) where the same question minutes earlier hallucinated; origin OOM-killed mid-test and both joiners re-	Three nodes; not adversarial; not a throughput evaluation

Claim	Configuration	Evidence	Boundary
		formed the network unaided	
Committee and repair behavior	Live multi-node runs	Observed: bootstrap, gossip discovery, committee formation and saturation, heartbeat death detection, eager repair, mesh fallback	Not adversarial availability
Weight-integrity defenses	Live	A corrupted on-disk shard is caught on store-open, cleared, and re-fetched; malicious non-partition manifests rejected on parse	Fully covered
Metering	Live, two full nodes and one light node	Transparent delegation with cost and balance; per-provider ledgers; deterministic `payment_required` on exhaustion; resume after credit	Trusted-operator only (Section 11)
Target frontier deployment	GLM-5.2-class	Analytical sizing only	Not run

The abstract's headline, the "Paris." completion, should be read as expected, token-identical output under a partial store, demonstrating the fetch-verify-persist loop and engine correctness, not as a factual-accuracy evaluation.

Measurement agenda (not yet done). Tokens per second as a function of miss rate and NIC tier on a fast LAN fabric; repair time after killing a sole holder; the committee-saturation join sequence at scale; multi-stream serving under continuous batching.

---

11. Limitations

Consolidated so a reader sees the full picture in one place.

Compute and coverage.

- Single-sequence only; no continuous batching.
- GLM 5.2 itself has not been run: its `glm-dsa` attention variant (sparse indexer, MTP head) is not implemented; deepseek2/glm4moe are the architectural proxies. Split (multi-file) GGUF checkpoints are not yet readable, which any frontier-scale download will be.
- The Vulkan path is ~60-65% of llama.cpp on the same card; the remaining gap is localized (barrier drains) but not yet closed.
- Wide-area behavior is measured for correctness, not under contention or churn at scale; only three simultaneous machines have been run.

Trust (v1 is operator-trusted).

- Manifest choice is trust-on-first-use; a hostile bootstrap can pin a poisoned root.
- Holdings claims are unverified; committee completeness assumes honesty.
- No transport encryption or authentication (plaintext TCP), which is LAN-appropriate, not public.
- Availability attacks (service refusal, mesh poisoning, flooding) are not defended.

Metering (trusted-operator accounting demo).

- Client ids are self-asserted strings, so rotating an id resets the free quota.
- Ledgers are per-provider, so round-robin across N full nodes yields N times the free quota.
- `credit` is admin-gated but its payment proof is unverified in v1.
- No signed receipts, so neither party can prove the other's ledger wrong.
- Prerequisites for real compensation: cryptographic client identity, payment-proof verification, signed usage receipts.

Operational.

- The RAG chunk store is memory-only; a crash or OOM discards every chunk (persistence is queued).
- ENR advertising and hardfork coordination are designed but unimplemented.

---

12. Roadmap

Milestones with exit criteria; full tracking in ROADMAP.

1. **Hardware-tailored kernels** (issues #10 to #14). Done for CPU, Metal and Vulkan: measured tokens/sec over scalar on DeepSeek-V2-Lite (Section 10), each kernel behind a backend interface with a differential oracle. CUDA (#14) remains open.
2. **Serve the distributed engine through node RPC.** Done: `loom node` serves the distributed GGUF (deepseek2) engine over its RPC and OpenAI surfaces, answering inference requests via the token-loop peer fetch, not only `loom gguf run`. A partial-store node's output is byte-identical to a full-store node's for the same prompt.
3. **Network-tier evaluation.** Exit: a published table of tokens/sec vs miss rate across 1 and 10 GbE, and repair time after a sole-holder failure.
4. **Hardfork coordination.** Exit: a majority of nodes adopting a new manifest version, with the existing version guard enforcing isolation during transition.
5. **v2 trust layer.** Exit: untrusted-peer serving with pollution-resistant propagation and sampled compute verification.
6. **Distributed speculative decoding** (inspired by DSD [16]; first slice implemented -- see the decision log). Today a cold node below the holdings threshold delegates the *whole* generation to a warm peer and relays the answer. DSD's edge-cloud split suggests the middle rung: the cold node drafts tokens locally with what it already holds (the MTP head and the hot head of the store) and ships only the draft window to the warm peer, which verifies the whole window in one batched forward -- per-request traffic becomes tokens, not experts, and the warm peer's cost per accepted token drops by the acceptance rate. DSD's measured crossover (distributed drafting wins below ~50-60 ms RTT, local wins above) maps directly onto loom's measured-tier-order principle: the same startup probe that orders fetch tiers decides draft-locally-vs-delegate-wholesale. Their stabilization findings carry over too -- clamp the speculation window, smooth it, and make the local/delegated mode switch sticky so it cannot flap. Their learned window controller (a DNN predicting window size) is noted and deliberately not adopted: at loom's scale the same signals (recent acceptance rate, RTT, queue depth) feed a threshold heuristic that DSD's own ablation places within a few percent of the learned policy. Exit: draft-remote-verify measured on the devnet against delegate-wholesale on the same prompts, with the mode decision driven by probed RTT and measured acceptance.
7. **Cross-model KV transfer for a two-tier model family** (research lever, gated on the 235B tier; inspired by [17]). The devnet's current and target models are a matched-KV pair within one family -- verified from both GGUF headers: Qwen3-30B-A3B and Qwen3-235B-A22B share 4 KV heads at head-dim 128 (48/94 layers, 32/64 query heads differ, which the method tolerates). [17] shows that within such a family, a closed-form per-head ridge mapping (no training, fit offline from ~500 calibration sequences) converts the small model's KV cache into the large model's, letting the large model decode without re-prefilling -- retaining 73-98% of standalone accuracy on its Qwen3 pairs, stable over multi-turn handoff. For loom this attacks the most

expensive operation the network has: the big model's prefill is not FLOPs but WAN expert fetches, so the avoided cost here is larger than in the paper's single-box setting. Three uses, in order of leverage: (a) cross-model DSD -- a peer holds ALL of the 30B (~11 GB) as a full-quality drafter and maps its KV so the 235B verifier never prefills; co-locating drafter and mapper on 235B holders makes the KV transfer a local matmul, and the mapper application is exactly the batched per-head matmul the unified K-quant kernels serve; (b) cost-quality cascading with mid-conversation upgrade to the 235B without re-prefill; (c) session continuity across the 30B-to-235B hardfork. The mapper itself is a 4-12 GB immutable parameter blob -- content-addressed and distributed over loom's own weight layer like any other artifact. Honest gates before any build: [17]'s scope is dense full-attention models and its own Tier-2 result (matched-KV Ministral pairs collapsing to 42-44% retention) proves matched KV does not guarantee transfer, so the MoE pair must be measured; calibration must run against the quantized (Q2_K) models' actual KV; and the paper's attention-output cosine diagnostic (correlates with retention at r=+0.57) is the cheap go/no-go screen to run before investing. Exit: mapper fitted on quantized-KV calibration data, cosine probe passed, and measured HellaSwag-class retention plus end-to-end prefill-avoided latency on the devnet pair.

8. **Commitment-weighted verifier expert selection** (research lever; extends lever 6's DSD; inspired by AcceptMoE [18], evaluated on Qwen3-30B-A3B -- the devnet model). DSD's batched verification pays for the expert UNION across the draft window even though only the accepted prefix matters: on loom's verifier every union member that is not held or cached is a disk page-in or a WAN fetch, so the union -- not the token count -- is the bill. AcceptMoE weights each draft position's router demand by its probability of reaching the accepted output, self-sizes the eligible expert set by the demand distribution's effective rank (the always-committed root token keeps its natural top-k untouched), and prunes low-demand NONRESIDENT experts against the current cache state under a rerouting budget -- eligibility becomes a function of residency, which is precisely loom's fetch-tier model. Measured (SGLang, expert-offloaded): expert-weight traffic −38.6 to −48.6% from residency pruning alone, host-to-device transfers −73.6 to −77.1%, 2.06x end-to-end throughput, -0.27 pp mean accuracy. Loom's fit is unusually direct: our draft window is a CHAIN (gamma <= 8), so commitment probability is acceptance^depth and the DSD gamma controller's live acceptance EMA supplies it -- no offline trace collection -- and shard residency is already tracked per fetch. Two gates, stated plainly: masked verifier routing is NOT distribution-preserving, so this ships as an opt-in mode with the default path keeping the drafted-loop byte-identity test gate; and loom's window (<= 9 lanes) is far smaller than the paper's 64-token trees, so absolute union savings shrink and the win must be re-measured in loom's regime, where the per-expert cost (WAN/disk) is however far larger than their host-to-device copies. Exit: opt-in restricted-routing verify measured on the devnet against the default, reporting verifier expert-fetch count, tok/s, and drafted-output divergence rate on the same prompts.

9. **Model co-design: training for distributable activation shape** (research lever; the inverse of every other lever). Levers 1-8 take the model as given and optimize loom around its routing behavior; this lever asks what a model trained FOR loom would look like. The observation: loom's steady-state economics reduce to expected fetch-bytes per token under a cache budget, and that quantity is a property of the model's activation shape -- which today's MoEs actively pessimize, because their load-balancing auxiliary losses push expert usage toward uniform, the worst case for any cache. Four trainable properties, in leverage order: **temporal stickiness** (adjacent tokens reuse experts; a routing-consistency regularizer makes the LRU work), **routing predictability** (pre-gated routing decides layer L+1's experts at layer L, turning the speculative-prefetch lever from a gamble into a pipeline; Pre-gated MoE demonstrated this for offloading), **global skew** (weaken the load-balance loss so a pinned hot set covers most activations), and **fetch granularity** (routed experts as low-rank deltas over a resident shared backbone -- DeepSeek's shared-expert design taken to its conclusion -- shrinking a fetch from ~1.5 MB to kilobytes). Staged cheapest-first, each stage a go/no-go: stage 0 instruments the devnet model's actual shape (`LOOM_EXPERT_TRACE` + `scripts/expert-trace-stats.py`: coverage, stickiness, window-union, cross-layer predictability, and cache-replay fetch-MB/token); stage 1 replays traces against cache/prefetch policies to bound what systems work alone can extract; stage 2 retrains ONLY the routers of an existing small MoE with the regularizers and measures the exchange rate -- perplexity paid per fetch-byte saved -- which is the lever's whole verdict; stage 3, only if the rate is favorable, pretrains a ~1B-total toy with the loom-native architecture (MLA, heavy shared expert, low-rank routed deltas, pre-gated routing) and benchmarks it end-to-end on a partial-holder devnet node, tok/s and fetch traffic against a same-budget conventional MoE.

Honest gates: routing constraints spend model quality (the load-balance losses exist because unconstrained routers collapse), no published measurement of this exchange rate exists for the distributed-cache objective, and stages 2-3 are training projects with real compute cost -- which is why the lever is staged to die cheaply at the first unfavorable number. Exit: stage-2 exchange-rate curve (perplexity delta vs fetch-MB/token at fixed cache budget) on an open small MoE, published with the traces.

---

13. Conclusion

Loom reframes frontier-MoE inference as a distributed-storage problem: store the expert corpus once across a swarm, verify it cryptographically, and page it into node-local computation on demand. The consequences (soft failure, a compounding cache, and self-healing committee membership) follow from moving immutable weights rather than mutable activations. The implementation demonstrates the core loop end-to-end on real weights under a partial store, now across real machines and a wide-area link with GPU-rate local decode, and is explicit about the boundary: cross-node distribution is validated for correctness and capacity, not yet for throughput at scale. Pooling ordinary machines lets a frontier MoE model fit where it otherwise could not. Making it fast is a separate problem, bound by network and kernels, and the roadmap treats it as such.

References

1. DeepSeek-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model.* 2024. https://arxiv.org/abs/2405.04434 (Multi-head Latent Attention).
2. DeepSeek-AI. *DeepSeek-V3 Technical Report.* 2024. https://arxiv.org/abs/2412.19437
3. Moonshot AI. *Kimi K2.* 2025. https://github.com/MoonshotAI/Kimi-K2
4. GLM Team, Zhipu AI. *ChatGLM / GLM-4 family.* 2024. https://arxiv.org/abs/2406.12793 (GLM 5.2 is the design target; the GLM-4 report is the cited real family.)
5. Borzunov et al. *Petals: Collaborative Inference and Fine-tuning of Large Models.* 2022. https://arxiv.org/abs/2209.01188
6. exo-explore. *exo: run your own AI cluster at home.* https://github.com/exo-explore/exo
7. Gerganov et al. *llama.cpp / GGML / GGUF.* https://github.com/ggml-org/llama.cpp
8. Song et al. *PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU.* 2023. https://arxiv.org/abs/2312.12456
9. Sheng et al. *FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU.* 2023. https://arxiv.org/abs/2303.06865
10. Merkle, R. *A Digital Signature Based on a Conventional Encryption Function.* CRYPTO 1987 (Merkle trees and content addressing).
11. Peng et al. *YaRN: Efficient Context Window Extension of Large Language Models.* 2023. https://arxiv.org/abs/2309.00071
12. Ethereum. *EIP-778: Ethereum Node Records (ENR).* https://eips.ethereum.org/EIPS/eip-778 ; libp2p gossipsub. https://github.com/libp2p/specs/tree/master/pubsub/gossipsub
13. Google. *Snappy compression.* https://github.com/google/snappy (Zig binding: https://github.com/blockblaz/zig-snappy)
14. Krohn, Freedman, Mazieres. *On-the-fly Verification of Rateless Erasure Codes for Efficient Content Distribution.* IEEE S&P 2004 (homomorphic hashing).
15. Di Giacinto, E. et al. *LocalAI: The free, open-source OpenAI alternative.* https://localai.io (federated and worker modes: https://localai.io/features/distribute/)
16. Yu, F., Li, L., McDanel, B., Zhang, S.Q. *DSD: A Distributed Speculative Decoding Solution for Edge-Cloud Agile Large Model Serving.* 2025. https://arxiv.org/abs/2511.21669 (adaptive window control, edge-draft/cloud-verify split, RTT crossover).
17. Heo, T., Shafipour, R., Zhao, R., et al. (NVIDIA). *Cross-Model KV Cache Transfer in LLM Families: A Closed-Form Linear Mapping for Prefill Reuse.* 2026. https://arxiv.org/abs/2608.03893 (per-head ridge mapper, RoPE-stripped content space, matched-KV pairs, attention-output-cosine retention predictor).

18. Liang, S., Chen, H., Mo, Z., et al. (Imperial College London / Tile-AI). *AcceptMoE: Commitment-Weighted Self-Sizing Verifier Expert Sets for Efficient MoE Speculative Decoding.* 2026. https://arxiv.org/abs/2608.02989 (verifier-side expert selection, commitment-weighted demand, effective-rank self-sizing, residency-aware pruning; evaluated on Qwen3-30B-A3B).
19. Commonware. *Earn Your Stripes.* 2026. https://commonware.xyz/blogs/earn-your-stripes (leader-side RS striping for block dissemination: distinct shard per validator, reconstruct from any k, commitment over shards).
20. Yang, S., Fan, X., Pan, M., et al. (UC Berkeley / UT Austin). *FreeToken: Efficient Edge-Native MoE Serving with Bandwidth-Adaptive Execution.* 2026. https://arxiv.org/abs/2608.16157 (single-node VRAM-over-host-RAM expert cache; measured-bandwidth q* miss splitting, full-layer double-buffered prefill streaming, semantic-anchor state caching).

Appendix A: Decision log

For contributors. This append-only log records design decisions with dates and rationale. It is maintained alongside the code and is not part of the paper's narrative. Superseded decisions are struck through, not deleted. Where this log, the spec, and the roadmap differ, the spec governs the p2p protocol and the roadmap governs status.

Date	Decision	Rationale
2026-07-11	Distribution unit is the expert, not the layer; compute node-local; never distribute KV	MoE sparsity plus MLA's tiny KV; avoids activations in the serial network path
2026-07-11	Content-addressed shards plus a Merkle-rooted manifest	Free integrity and poisoning check at every hop; root doubles as version id
2026-07-11	No coding (EC or RLNC) in the token loop, ever	Decode-before-use adds latency where it cannot be hidden; coding stays in propagation and durability
2026-07-12	Target toolchain: Zig 0.16.0, matching the sibling zeam project	Shared toolchain plus reusable deps (ENR, SSZ, snappy)
2026-07-12	v1 direction: GGUF interchange format; ENR, gossip, and request-response p2p; majority hardforks; boot-time peer sync	Requirements of record; supersedes an earlier Hyperswarm/Hyperbee sketch
2026-07-12	Weight advertising: both ENR (compact summary within the 300 B limit, a bitmap digest plus seq) and a global gossip topic (full bitmap)	ENR for discovery-time selection; gossip for live freshness
2026-07-13	Churn repair is maximally eager; an always-on loop retries all known peers and dead peers are retried, not forgotten	Chosen over lazy repair-on-miss
2026-07-13	Gossip is epidemic announce-and-exchange over the peer table; transport swappable for gossipsub later	Table semantics are transport-independent
2026-07-19	deepseek2 rope is NORM (adjacent-pair), not NEOX	Empirical: DeepSeek's (d/2,2) transpose before rotate_half nets to adjacent-pair on the stored layout; NEOX degenerates
2026-07-19	Response payloads carry no digest; the requester verifies against its own manifest	The manifest is the only trust root
2026-07-19	Shard equals one expert block plus mandatory 16 MB resident chunks; about 19 MB per expert gives 19,200 shards for GLM 5.2	Shard unit must equal the matmul's consumption unit; metadata stays trivial

Date	Decision	Rationale
2026-07-19	Bootnode assigns committees least-covered-first toward R = 3 target, R = 2 floor; saturated committees spawn new ones	Coverage by construction; bootnode out of the inference path
2026-07-19	Fetched shards are persisted, not evicted, so fetch-on-demand is organic heat replication	Disk is the cheap resource; hot experts gain holders by being used
2026-07-20	Wire messages v1: binary frames, adaptive snappy; heartbeat carries seq, digest, and load but not the bitmap	Quantized payloads are incompressible; heartbeats stay small; staleness detected cheaply
2026-07-20	Committee membership is gossip-derived, since announces carry committee id	Fixes static-at-join membership; survives bootnode death
2026-07-20	Two node classes: light nodes (no weights or engine; delegate) and full nodes (shards plus inference)	Local inference for low-memory devices without weakening supply
2026-07-20	Compensation: per-client metering ledger on each full node; `payment_required` enforcement; `credit` op as the settlement seam	Accounting precedes payment rails; ledgers are per-provider
2026-07-20	Redundancy R = 3 target, R = 2 floor (default 2); completeness means R >= 1	Completeness and redundancy are distinct thresholds
2026-07-20	Placement is least-covered-first committee assignment; random `--hold-fraction` is the no-bootnode fallback	One live placement policy
2026-07-20	Binary frames are normative; the line protocol is legacy and debug during migration	Prevent forked implementations
2026-07-20	Hardware-tailored compute backends behind one `Backend` seam (scalar oracle; CPU SIMD then GPU); planned as issues #10 to #14	Per-node throughput gates the distribution story
2026-07-20	Code-audit hardening: version root binds layout; hot-path and serve re-verify; store-open re-audits held shards; resident completeness gate; publish-after-verify cache; credit admin-gated; light node forces client id; token clamp; account cap; snappy decode cap; bounded peer table; monotonic holdings seq; connection caps; death triggers wanted re-replication	Close the gap between "verified before use" and the code; ship the availability bounds the spec named
2026-07-20	Documents restructured to standard whitepaper format (front matter, TOC, glossary, related work, references, claim-to-evidence table, limitations); terminology split (expert shard vs resident chunk); decision log demoted to a contributor appendix	Editorial reviews: read as a whitepaper, scope claims precisely, cite external work
2026-07-20	Add an OpenAI-compatible HTTP API as the north-facing client surface (`--openai-port`); client identity moves to `Authorization: Bearer`; MCP is not the serving path (a node is more naturally an MCP client)	Ecosystem clients work with no adapter; bearer identity is out-of-band and not prompt-forgeable, unlike the native RPC `client` field. Shipped as a skeleton (`src/node/openai.zig`): transport, routing, shapes, identity; generation/streaming TODO

Date	Decision	Rationale
2026-07-21	A `Generator` abstraction (`src/node/generator.zig`) unifies the loom-format and distributed GGUF (deepseek2) engines; both RPC and OpenAI serve through it; the node auto-serves the GGUF engine when an expert-sharded store is attached and its resident bundle is complete	Serves the distributed engine (token-loop peer fetch) through the node, not only `loom gguf run` (roadmap item 2). Store mutation from serving-fetch and eager repair serialize on one engine mutex
2026-07-21	Light nodes delegate the OpenAI surface too (`src/node/light_openai.zig`), a metered reverse proxy to full-node OpenAI endpoints; the light node forces its client id via `Authorization: Bearer` (caller bearer dropped)	OpenAI clients can point at a low-memory light node; identity is forced so a light node cannot be an open proxy, parallel to the native-RPC rule
2026-07-22	SSE streaming for `stream:true` (`src/node/openai.zig`): OpenAI `text/event-stream`, one chunk per token then `[DONE]`, over both engines. Backed by an optional per-token callback on `engine.generate` and a `generator.TokenSink`	Chat clients that default to streaming work; client disconnect aborts generation, metering still charged
2026-07-22	Per-model chat templates (`src/gguf/chat_template.zig`): detect the format from GGUF `tokenizer.chat_template` (or arch), render `messages[]` per format (deepseek/chatml/llama2/llama3/gemma/mistral/generic), `--chat-format` override. Format detection plus built-in renderers, not a Jinja engine	Real chat models need their own prompt format. DeepSeek-V2 (the validated target) is faithful
2026-07-22	Special-token-aware tokenization (`src/gguf/special.zig`): both BPE and SPM splice control (type 3) / user-defined (type 4) tokens to atomic ids, longest-match with a first-byte filter, splitting the input before normal encoding	Chat markers (chatml/llama3/gemma) and control tokens tokenize exactly instead of being split. No change when no special appears (no regression). Caveats: parse_special always on (injection); SPM dummy-prefix on the first segment only
2026-07-24	Parse-special injection hardening: `parse_special` threaded through the tokenizer chain; off for raw prompts and text-marker chat formats (deepseek/llama2/mistral), on only for special-marker scaffolds (chatml/llama3/gemma)	Untrusted input can no longer inject a control token on the raw-prompt or RPC path, nor via text-marker chat content (the validated DeepSeek-V2 target is fully safe). Segment-encoding content for special-marker formats is the remaining follow-up
2026-07-27	Codebook quantizations implemented (`src/gguf/iq.zig`): IQ1_S/M, IQ2_XXS/XS/S, IQ3_XXS/S, IQ4_NL/XS and MXFP4, with grid and sign tables transcribed mechanically from llama.cpp (`src/gguf/iq_tables.zig`)	Most modern GGUF repositories publish mostly IQ variants, so the affine-only kernel set excluded the majority of available checkpoints. MXFP4 additionally covers microscaling checkpoints. Verified bit-for-bit against llama.cpp's own `dequantize_row_*` on stored vectors (`src/gguf/iq_vectors.zig`), because a wrong codebook entry corrupts weights silently instead of failing

Date	Decision	Rationale
2026-07-27	MoE routing extracted to `src/gguf/moe.zig` and shared by both engines, including the expert-to-shard binding used for distributed fetch	The routing was never DeepSeek-specific: upstream funnels every MoE architecture through one routing function differing in four knobs (gating function, selection bias, gate renormalization, constant scale). Sharing it means a new architecture needs an attention variant, not a new engine
2026-07-27	The llama engine generalized into one GQA engine covering `llama` (Mixtral), `qwen2moe`, `qwen3moe` and `glm4moe`; optional features detected from the file's tensors, with only the RoPE style pinned per architecture	Distributed serving previously required `deepseek2`, even though sharding, sync and repair were already architecture-agnostic: a Mixtral store distributed correctly but could not be served. Feature detection follows the checkpoint rather than a table of beliefs about each architecture, which is also what makes the qwen3 explicit `head_dim` and the glm4 NextN skip fall out naturally. Verified end to end: for all five architectures a node holding roughly 30% of shards serves with hit rate below 1 and produces output token-identical to a full-copy origin
2026-07-28	A node serves a GGUF locally when it is not expert-sharded, instead of falling back to the synthetic checkpoint	A dense model has no routed experts, so it shards into fixed ranges and cannot be served *distributed*; but it is a complete model file, and answering from an unrelated synthetic checkpoint reads as a broken model rather than as an unsupported topology
2026-07-29	The 1.9x whole-layer result is validated on TinyLlama-1.1B only; Mistral-7B on the same machine is memory-bound and tests nothing. The device KV cache is released when calibration declines every reader	Mistral-7B runs at 0.27 tok/s here with the process at 116% CPU and 2.75 GB resident against a 4.37 GB model, stalling on page faults. Calibration measured 3442 ms/tok against 3414 and correctly declined, since a ~262 us command buffer is invisible against 3.4 seconds. So the GPU result holds for cache-resident tensors (TinyLlama's are 6.5 MB) and is unproven at DRAM-bound sizes, which is also where ZINC's kernel advantage disappears. This is the same constraint the distributed thesis exists for, arriving from the other direction: a 16 GB box cannot hold a 7B model and its caches, let alone a large MoE
2026-07-29	The recorded whole-layer path is the default on a GPU machine, chosen by timing a real token both ways at load (25% margin, `--no-gpu-layers` to force the host path)	The unit of measurement has to be the whole token: nothing smaller is honest about submission cost, because timing one operation in a tight loop lets successive commits pipeline while the engine serialises them. The per-operation calibration this replaces reported the fused FFN block at 1.107 ms against a CPU 9.837 ms, when the same CPU block measured 0.489 ms elsewhere, and acting on it took

Date	Decision	Rationale
		decode from 56 to 9.1 tok/s. The whole-token measurement predicts 119 tok/s where 109 is observed. A build with no GPU backend never enables it: `layerBlock` declines, both timings match, and the margin is not met
2026-07-29	A frame abstraction on the compute seam (`beginFrame`/`layerBlock`/`endFrame`) records a whole GQA layer per command buffer; `--gpu-layers` measures 110.3 tok/s against 57.7 on the CPU path, with correct output	The seam's one-shot entry points each created a command buffer, committed, waited and copied back, so a token cost ~150 submissions at ~262 us regardless of kernel quality; the same kernels behind a recorded path are 1.9x faster. ZINC's Metal on the same model measures 52.9-56.1. Recording introduces failure modes that immediate submission does not, all of which produce fluent, wrong output: host writes to a shared scratch buffer are no longer ordered against the recorded reads (twenty-two layers read the last layer's norm weights); and a dispatch grid must follow each kernel's rows-per-SIMD-group, since Q4_K takes two rows per group and Q6_K one, so dispatching Q6_K with the Q4_K count silently computes half the output; TinyLlama's `ffn_down` is Q6_K. A unit test passed throughout because it ran only `pos=0`, where RoPE is the identity and single-position softmax makes attention a passthrough: the fixture had switched off both things most likely to be wrong
2026-07-29	The Metal Q4_K matvec is at the machine's DRAM bandwidth ceiling and level with ZINC's kernel; the remaining GPU gap is command-buffer submission, not kernel quality	Measured amortized (many dispatches per command buffer, as a kernel microbenchmark must be), loom reaches 114-116 GB/s from 32k rows up against a ~110 GB/s streaming-read ceiling and ZINC's reported 109 GB/s at 27 MB. An earlier reading of ZINC's numbers as ~3x faster compared loom's per-dispatch timings against ZINC's amortized ones; ZINC reports 259.81 us for a shape where one command buffer alone costs ~262 us, so its figures cannot include one. The same kernel takes 0.504 ms per-dispatch and 0.323 ms amortized at 32k rows: ~180 us of submission, which is the whole difference between losing to the CPU and beating it 1.3x. ZINC remains genuinely ahead at cache-resident shapes (239 GB/s at 2.25 MB vs 67), where matvecs are latency-bound and its register-level work pays; adopting three of those techniques changed nothing at DRAM-bound sizes, consistent with there being no headroom there
2026-07-29	Metal gains Q6_K dmmv, a batched Q4_K gemm for prefill, and fused grouped-query at-	Each kernel is correct and each loses. The reason is not kernel quality but that calibration,

Date	Decision	Rationale
	tention over a device-resident KV cache. All are validated against exact CPU references and all are disabled by default, behind `--gpu-ops`	which times one operation in a tight loop, systematically overestimates the GPU: consecutive submissions pipeline there and are serialised in the engine. Acting on that verdict took decode from 56 to 9.1 tok/s: a 6x regression chosen by a measurement that looked rigorous. Two further traps were found and fixed: a device KV mirror written by decode but not by batched prefill left zeros at every prefilled position, and attention over them produced fluent, wrong text; and maintaining that mirror when nothing reads it cost 12,672 device writes over a 576-token prefill, taking decode to 3 tok/s with the GPU otherwise unused. The verdict is still computed and printed, because seeing it is useful; it is not obeyed until calibration measures a whole layer as the engine issues it
2026-07-29	Metal is built by default on macOS; the CPU/GPU choice per operation is measured on the loaded model's own tensors at startup (25% margin, tie resolves to CPU) rather than compiled in; the fused FFN block is exported through the compute seam	A shipped binary that is CPU-only cannot use a GPU backend however good it is, and a compiled-in row threshold is a measurement taken on someone else's machine; the crossover is a ratio between a given box's GPU and its CPU cores. Measured against ZINC (Metal) on TinyLlama-1.1B Q4_K_M on an M5: loom 55.7-56.2 tok/s decode, ZINC 52.9-56.1: loom matching a Metal engine while running entirely on the CPU, because calibration found the GPU slower at every shape this model issues. This refutes the inference that equal throughput implies equal GPU optimisation, and bounds the remaining GPU prize: on unified memory the measured GPU win is ~1.5x above 65k rows and nothing below it, so the payoff is in large models and prefill, not small-model decode
2026-07-29	The GGUF serving path gains a local RAM tier (LRU of verified expert blocks, sized by the largest *expert* shard rather than the largest shard); MLA attention absorbs W_k into q and accumulates values in compressed space; Q4_1/Q5_1 gain int8-activation kernels; a `LOOM_PROFILE=1` per-phase decode profiler; optional zero-copy weight serving from a read-only mapping (`--mmap-weights`, off by default)	Decode was 1.07 tok/s and the reasons were all outside the kernels. Expert `get` was 54-58% of a token because every routed expert was re-read from disk *and* re-hashed on every access with nothing cached; MLA was decompressing K and V per head per cached position, O(seq) matvecs where the whole point of MLA is that K and V are never materialised; and `ffn_down_exps` in this Q4_K_M checkpoint is Q5_1, so a third of every expert fell to the exact dequantize path. 584 -> 272 ms/token, 1.07 -> 3.03 tok/s end to end. The profiler is itself a decision: two well-argued theories together moved the number by 20%, and a hypothesis that survives a measurement which should have

Date	Decision	Rationale
		refuted it means the measurement is what needs fixing
2026-07-29	Measured, on a 16 GB machine serving an 8.9 GB model, single-node decode is bound by getting ~1.16 GB of expert weights per token off disk, not by compute. Four residency strategies (no cache, 2 GB heap, 8 GB heap, zero-copy mmap) all land within 480-362 ms/token and merely move the cost between `get` and the matmul	This is the regime the distributed thesis is *for* (the whitepaper's own bandwidth table says a single commodity box cannot hold a large MoE's working set), but it also means single-node numbers on an oversized model measure the storage path, not the engine. Engine-efficiency comparisons against other runtimes need a model whose expert corpus fits in RAM, or the comparison measures page-cache policy
2026-07-28	The Metal Q4_K matvec reads four quant bytes per lane instead of one, with a branchless 6-bit scale unpack; `MIN_GPU_ROWS` drops 100,000 -> 65,536; the GPU scaling benchmark reports best-of rather than mean	The previous "61 GB/s is the unified-memory wall" conclusion was drawn from our own kernel's achieved bandwidth, never from the machine's. A pure streaming-read probe reaches ~110 GB/s on the same M5, so the wall was ours: the kernel was coalesced but quarter-width, with too few loads in flight to cover their latency. Widening alone is a regression below 131k rows because the sub-block index then varies per lane and the scale unpack's branch diverges within the SIMD group; both changes are needed. Result 63 -> 94 GB/s at 131k rows, 1.5-1.6x. Decode on a 1.1B model is still CPU territory; its largest tensor is 32,000 rows and a command buffer costs ~262 us fixed. Means were replaced by best-of because GPU contention with the window server moved repeated measurements of an unchanged kernel by 50%, which had briefly made a regression look like a 1.4x win
2026-07-28	Peer liveness is evidence-tagged: only `first_hand` contact refreshes last-seen, `hearsay` (a peer appearing in another peer's table) does not; a peer stale for 30 s is withheld from holder selection and from the peer count while its address is retained for eager repair. Nodes report the number of shards held by fewer than R live peers	Liveness previously came only from the committee heartbeat, which watches committee members; a bootnode/origin belongs to no committee, so the node solely holding 332 of 1,664 expert shards was monitored by nobody. Independently, hearsay refreshed last-seen, so in any swarm of three or more the survivors' echoes kept a dead peer alive indefinitely: measured, an origin killed at t=0 was still advertised as a holder at t+6 min. Separating address (keep forever) from holdings claim (expire) preserves the eager-churn policy while making death detectable. Under-replication is surfaced because R is an arithmetic consequence of membership, not a setting: n nodes at hold-fraction f cannot exceed R = n*f
2026-07-28	A chat UI is compiled into the binary and served on its own listener (`--ui-port`, default	Nothing in the system reported its own state: exercising a node meant hand-writing HTTP,

Date	Decision	Rationale
	8555), sharing the HTTP implementation and generator with the OpenAI API; plus a periodic console status line (`--status-secs`)	and membership and holdings move with no request arriving, so an event-only log made a churning node look idle. Serving the page same-origin with the API removes any CORS or host configuration. The UI separates decode speed from time-to-first-token, because on a partial node the first token also pays for the prefill's expert fetches
2026-07-28	`/health` reports whether the served weights are one of loom's random-weight fixtures, and the UI says so	A fixture answers with meaningless text by construction; unlabelled, that is indistinguishable from a broken model and was the first thing a new user saw
2026-07-28	Q4_1 and Q5_1 implemented, completing every non-ternary GGML type	Three Q5_1 tensors out of 377 blocked an entire 8.9 GB checkpoint. llama.cpp quant *mixes* fold a handful of legacy types into otherwise K-quant files, so one missing format costs a whole model
2026-07-28	SIMD kernels via `@Vector` plus row-parallel matvec over a worker pool (issue #11)	Inference ran at roughly 1% of the machine's memory bandwidth on one of ten cores: the limit was scalar single-threaded code, not the absence of a GPU. Measured 23.6x on a 10-core M5 (1.1 -> 26.0 tok/s, TinyLlama 1.1B Q4_K_M). Row-splitting is exact rather than approximate, since each output row is an independent dot product with no reassociation, so results stay bit-identical to the serial path and the tests assert it. The pool is scoped to a generation because Zig 0.16 moved condition variables under `Io`, which the kernels deliberately do not depend on, so parked workers would otherwise spin
2026-07-28	int8-quantized activations for the Q4_K, Q6_K and Q8_0 kernels	Profiling showed 76 to 92 percent of a quantized matvec was the dequantize step, not the dot: each block was expanded into a 256-float scratch buffer and pushed through L1 only to be read back. Quantizing the activation once per matvec and dotting against packed weights as integers removes the buffer and uses lanes four times as wide. Measured 1.1 -> 51.9 tok/s overall (47x from the original scalar path). Unlike the earlier steps this is an approximation, about 0.4 percent per activation element, so results are no longer bit-identical to the dequantize path and the tests bound the error against the mass of the summed terms rather than against the heavily cancelled result
2026-07-29	~~A whole MoE layer is recorded as one command buffer (`moeFfnBlock`: zero the accumulator, then every routed expert's gate/~~	~~Per-expert dispatch cost one ~262 us command buffer per expert, seven per layer, against expert FFN kernels of ~250 us; the missing

Date	Decision	Rationale
	~~up/SwiGLU/down and its weighted add, one submission for the layer), plus a Q5_1 matvec kernel so the DeepSeek checkpoint's `ffn_down_exps` no longer forces the layer back to the host. It is used only when the expert source hands back pointers that stay valid across later fetches -- a mapping, or an LRU with at least as many slots as the layer selects~~	Q5_1 kernel was worse than its own tensor's cost, because declining one tensor declines the whole layer. The measured outcome is that it changes nothing end to end, and the reason is the more useful finding: on a 16 GB machine serving a 9.4 GB store, `expert get` is 36-40% of a token and varies by more than the entire FFN bucket between processes -- four alternating warmed runs gave 1.31/2.01 tok/s batched against 2.36/1.58 per-expert, ranges that overlap completely. Correctness is established instead of speed: the block matches the CPU expert loop numerically in test, and 48 greedy tokens are identical either way. Calibration on this model still selects the host path for every operation, so the code ships disabled here and is kept for the regime it was built for -- a model whose experts are resident, where fetch stops dominating~~
2026-07-29	The compute seam exports `moeFfnBlock` and `ExpertRef`; the engine's batched MoE path actually runs	`deepseek.zig` guarded the branch with `@hasDecl(backend, "moeFfnBlock")` against the *seam*, which did not re-export it. `@hasDecl` is comptime, so the guard was comptime-true, the whole branch was never analysed, and it compiled and ran as if the feature existed. A capability probe against the wrong module is indistinguishable from a backend that declines
2026-07-29	A weight mapping is handed to the compute backend as device allocations and made resident (`registerArena` / `materializeArenas`, MTLResidencySet), split at the device's `maxBufferLength`. Measured 4.1-5.4 tok/s against 3.5-3.6 for the 8 GB heap cache, page-ins over a generation 6.9 GB -> 1.4-2.2 GB	Two earlier conclusions were wrong and are corrected here. "This machine cannot make a 10.4 GB model resident"; it can: llama.cpp (build 34ce48d, Metal) on the *same file and machine* maps it into one 9,880 MiB buffer, pins it with a residency set, offloads 28/28 layers and runs at 46.8-58.6 tok/s with 249 MB of page-ins. And "on unified memory the GPU win is ~1.5x above 65k rows and nothing below" was measured on TinyLlama, where the model is 6.5 MB and only kernel speed is in question; on a 10.4 GB MoE the GPU also buys residency. What was *right*: loom's CPU engine is at parity with llama.cpp's (2.52 vs 2.60 tok/s), so the kernels were never the problem. Three failures were needed to get here, each silent: registering into a context the node had not built yet; wrapping without `requestResidency`, which leaves the pages as evictable as any other mapping; and asking for 9.7 GB in one allocation when `maxBufferLength` is about half of RAM. None of them failed loudly, which is

Date	Decision	Rationale
		why the banner now prints the resident byte count and the reason when there is none
2026-07-29	Two pre-existing defects, both found by the residency work rather than by a test: `Source.get` read a resident chunk into an expert-sized cache slot, and `Store.deinit` never freed the lazily-allocated `verified` bitmap	The first is a heap overflow, 16 MB into a 6.3 MB slot, on the resident-gate loop that every node runs at startup, so it had been corrupting memory on every distributed launch since the RAM tier landed. It only ever surfaced under `-Doptimize=ReleaseSafe`; ReleaseFast wrote past the slab and carried on producing coherent text. The lesson is not about the bounds check but about the benchmark: months of numbers were taken on a build whose safety checks were off, on a path where the overflow was guaranteed
2026-07-29	Metal gains Q5_0 and Q8_0 matvec kernels, and a test that asserts `moeFfnBlock` *accepts* every down-projection type real checkpoints use rather than skipping when it declines	Reading the checkpoint's tensor types rather than trusting the quant mix's name: DeepSeek-V2-Lite Q4_K_M stores `ffn_down_exps`, one of the three tensors in every routed expert, as Q5_0 in some layers and Q8_0 in others, and contains no Q5_1 at all. Neither had a pipeline, so `moeFfnBlock` declined at `pipelineFor(e.down.ty)` for every MoE layer, and the batched path was dead on the one model it was written for. The Q5_1 kernel added earlier was for a type this file does not contain. It survived review because the block's existing test treats a decline as `error.SkipZigTest` (a backend that refuses everything passes a suite built that way), so the new test fails instead, and was checked by removing the pipeline again
2026-07-29	The backend counts command buffers and the profiler reports submissions per token; the profile is a rolling window rather than a cumulative total	Counted rather than timed, because a count is deterministic and a wall-clock number on a swapping machine is not: a command buffer costs ~262 us fixed whatever it holds, so `count x 262 us` is a floor on the token that no kernel work can move. The measurement then refuted the optimisation it was taken to justify. The MLA engine has no recorded-layer path where the GQA engine does, and the plan was to build one; but submissions are 26-28 per token, ~6.9 ms of 128, so the whole recorded path was worth at most 5% and was not built. What the count did confirm is that `moeFfnBlock` now runs: 26 submissions across 26 MoE layers is one per layer, where per-expert dispatch would be 500+. Windowed rather than cumulative because the first touch of an expert hashes its whole 6 MB shard to verify it, so a short profile is mostly that transient and a long one averages

Date	Decision	Rationale
		it away; neither says what the node is doing now. Watching the windows converge (178.6 -> 157.8 -> 147.4 -> 136.6 -> 129.5 ms/token) separates the two for the first time
2026-07-29	Releases, the container image and `install.sh` build ReleaseSafe, not ReleaseFast; CI additionally runs the suite optimized	A node accepts weight shards from peers and writes them into buffers, so the difference between the two modes is whether an out-of-bounds write traps or executes. It was not hypothetical: a cache slot sized for a routed expert was handed a larger resident chunk on the startup path every node runs, and ReleaseFast wrote 16 MB into a 6.3 MB buffer and carried on producing plausible text for however long it had been there. Measured cost, alternating runs on DeepSeek-V2-Lite decode: 80.8/82.6 ms/token ReleaseFast against 91.0/91.8 ReleaseSafe, about 11%, and *none* of it in the GPU kernels: the expert FFN block is 41.1 ms in both, so what is being paid for is bounds checking on host loops. 11% of throughput is a fair price for a daemon on a network; ReleaseFast remains available and is the honest choice for a kernel benchmark, which is why the docs now say which mode a number came from. The suite runs optimized in CI as well, because Debug and ReleaseSafe differ in what the optimizer may assume, and a bounds check only Debug performs is not one this project ships
2026-07-30	Two-machine run over a real WAN (Mac <-> Hetzner, 25.8 ms RTT, 29 MB/s): a node holding 3.6% of shards produced output token-identical to a full copy, fetching 1,063 expert shards from the peer, each digest-verified. 24 tokens in 429.6 s	The correctness and integration claim holds across machines and across a wide-area link, not just on localhost, which is what this test existed to check. The throughput is what the bandwidth table in Section 9 predicts and is not the point
2026-07-30	**`--hold-fraction` is not a capacity cap, and the storage half of the thesis is therefore unenforced.** It bounds the bootstrap fetch only; a shard fetched at token time is verified, persisted and marked held, and nothing evicts it	Found by running the capacity test rather than by reading the code: the node above went from 3.6% to 93.1% of the corpus, 1.5 GB to 9.0 GB on disk, in the course of generating 24 tokens. The whitepaper's claim that the swarm stores the corpus *once* (sharded and hot-replicated instead of copied per node) does not survive this: every serving node drifts toward a full replica, so N nodes converge on N copies and the capacity benefit that distinguishes Loom from per-node streaming evaporates. Fetch-on-demand as organic heat replication is a deliberate and defensible design (`expert_fetch.zig`), but without an eviction policy it has no upper bound. What is needed is a real resident-set cap with eviction, at which

Date	Decision	Rationale
		point `--hold-fraction` becomes the knob the documentation already claims it is. Until then the honest statement is that Loom demonstrates distributed *fetch*, not distributed *capacity*
2026-07-30	`--hold-fraction` becomes a real cap: holdings are enforced continuously, the least-recently-used expert is evicted once the count is exceeded, and its blocks are hole-punched back to the filesystem. Resident chunks are never evicted. Measured on the same two-machine setup: holdings pinned at 489/1737 (73 resident + 416 experts, exactly the cap) and stayed flat while serving, where the uncapped node had run to 93.1%	This is what the storage half of the thesis requires: without it every serving node converges on a full replica and N nodes hold N copies. Two things had to be right and only the first was obvious. Clearing the bit makes the cap true, but punching the hole is what makes it true *on the disk*; APFS refuses an unaligned punch outright, so the first version freed nothing while the holdings count sat correctly at 28.2% and the store grew 1.8 -> 7.6 GB anyway. Extents start and end at arbitrary offsets, so the range has to be aligned inward to whole blocks, losing at most one block at each end. The test asserts allocated blocks rather than the bitmap, and was checked by reverting the alignment. The cost is now visible: at a 25% cap over a 29 MB/s link the node re-fetches evicted experts and 24 tokens exceeded the 900 s generation deadline, so the cap is a capacity knob with a latency price, not a free win
2026-07-30	Bulk range sync fails over a wide-area link (`EndOfStream`, 0 of 905 assigned shards) while the per-token fetch path over the same socket succeeds	The node still reached 93.1% holdings, through 1,063 individual token-time fetches instead of one bulk transfer, which is why the run took 430 s where a bulk sync at the measured 29 MB/s would have taken about three minutes and then served fast. Only a real link surfaced it: the same code path is exercised on localhost by the multi-node walkthrough and passes there, so whatever the framing assumption is, it holds at loopback latencies and does not hold at 25.8 ms
2026-07-30	The p2p server refreshes its deadline between requests; `fetchFromPeer` returns partial progress on error; the RAM-budget default is a quarter of physical memory, clamped to [0.5, 8] GB	Fixed and re-verified on the same two machines: bulk sync now reports `synced 905/905 ranges (5275.3 MB)` where it had reported `EndOfStream, 0/905, 0.0 MB`, and the origin runs at 1.92 GB RSS on the 7.7 GB box that previously OOM-killed it at 7.4 GB, with no `--ram-gb` given. The deadline was taken at accept and never refreshed, so a 30 s budget covered a whole connection instead of idle time: fatal for a ~5.4 GB bootstrap at 29 MB/s, and invisible on loopback where the same sync finishes in seconds. The count was lost separately: `fetchFromPeer` accumulated into a local and returned `!FetchStats`, so an error discarded the tally for everything already

Date	Decision	Rationale
		written, which is why the node claimed 0 while its holdings bitmap disagreed. Two independent defects presenting as one symptom, and only the second was cosmetic
2026-07-30	The shared expert is timed into the `expert ffn` bucket. Folding it into `moeFfnBlock` was tried, measured 6% slower, and reverted	It is a 2816-wide dense FFN on every layer (~225 MB/token, a third of what the routed experts move), and it sat outside both the block and the profiler's timing, so it landed in "everything else": 30% of a token and the largest thing nobody had explained. Attributing it was free. Batching it was not: A/B'd in one binary, folded runs 97.9/92.1 against 92.8/86.2 ms/token not folded. The fold is correct in principle (one command buffer instead of extra dispatches) and loses anyway, because it moves the work from the CPU onto a Metal matvec that is slower at 1408-2816 rows; that is the same verdict calibration reaches on its own, and why `--gpu-ops` is needed to force the GPU path at all. The lever is therefore the kernel at those row counts, not the batching around it: at ~27 GB/s against this machine's ~110 GB/s streaming ceiling, the expert FFN is where the remaining time is, and adding work to a losing kernel makes it worse
2026-07-30	Q4_K is two kernels compiled from one source (four rows per SIMD group below 4096 rows, two above), selected together with their dispatch grid by a single `dmmvFor`. Two correctness bugs found on the way, one of them pre-existing and silent	The scaling sweep started at 2048 rows and so had never covered the shapes an expert FFN actually issues (1408 routed, 2816 shared, 2048 dim), which is where half a token is spent. Extended down, it showed 33-38 GB/s there against 113 at 131k rows: not bandwidth, since 1408x2048 of Q4_K is 1.6 MB and sits in cache, but activation reuse: each group re-reads the whole 8 KB vector to serve two rows. Four rows per group takes 1408-row throughput to ~47-53 GB/s and costs ~7% above 5632 rows, so both variants ship and the shape picks. Pairing the pipeline with its grid in one function is not tidiness: the two must agree, nothing else enforced it, and dispatching one kernel with the other's grid silently computes part of the output vector; the worst bug in this work, twice. The new oracle test then found a third instance immediately: `dispatchThreads` may launch a smaller final threadgroup, in which `simdgroups_per_threadgroup` is not the full count, so `tgid * nsg + sgid` addresses the wrong rows and the tail is never written; `matvec` then copies stale scratch. Latent since the kernels were written, invisible because

Date	Decision	Rationale
		every shape tried divided evenly (65,536 at two per group, 1408 at four); 1409 does not. Grids now round to whole threadgroups. The GPU still loses to the CPU at these shapes (0.034 ms against 0.027 at 1408 rows), so calibration's verdict is unchanged and there is no end-to-end gain yet; the kernel is closer, not ahead
2026-07-30	Kernel variants are compared by an *interleaved* benchmark (each variant timed round-by-round inside one run), and the four-rows-per-group Q4_K kernel is 1.8x the two-row one at the expert-FFN shapes (87-94 against 47-53 GB/s at 1408-2048 rows). Threadgroup staging of the activation vector was built, measured consistently worse, and removed	The instrument had to be fixed before any of this could be believed. Comparing variants across separate runs does not work on this machine: the same binary reported 62.4, 52.9, 42.8 and 46.1 GB/s at 2816 rows on four consecutive runs, a +-20% band that swallows the differences being measured and had already produced two confident wrong conclusions. Interleaving the variants within a run puts whatever else the GPU is doing onto all of them equally, and repeatability goes from +-20% to +−3%. The sweep was also measuring the wrong thing outright: it dispatched `cx.q4k` with a `((rows + 1) / 2)` grid left from when Q4_K covered two rows per group, so against the four-row kernel it launched twice the groups needed and timed the waste: 47 GB/s reported where an honest grid gives 89. Both now go through the same `dmmvFor` the engine uses. The staging idea was well-founded (at 1408 rows the activations are 2.8 MB of traffic against 1.6 MB of weights) and still lost, because the 16 KB threadgroup allocation costs more occupancy than the sharing saves; eight rows per group lost for the same reason at 1408, where 44 threadgroups no longer fill the device
2026-07-30	A MoE layer is recorded as four phases across all experts (every gate and up, then every SwiGLU, then every down, then one fused weighted reduce) instead of a per-expert chain. Expert FFN 56.1 -> 49.1 ms/token, token 98.1/81.7 -> 81.8/76.6	The per-expert form put a barrier at every step, about thirty for a layer, and a barrier is a pipeline drain rather than a fence on one buffer, so at most two dispatches could ever be in flight. Experts are independent until the final sum, so the ordering was never required. Four barriers now, and twelve-way parallelism in the first phase for six experts. The accumulation had to change with it: a `scaled_add` per expert all write the same accumulator and so needed a barrier between each, which one `moe_reduce` over the strided outputs removes entirely. Measured by alternating two binaries in one session, because a cross-run comparison of this size is not resolvable on this machine: the expert-FFN ranges do not overlap (48.1-50.0 against 53.5-58.6) where the token totals nearly do.

Date	Decision	Rationale
		Also worth recording: the 87-96 GB/s the interleaved kernel benchmark reports is *cache* bandwidth, since it reuses one 1.6 MB matrix; the engine streams ~1.1 GB per token from DRAM, so the kernel figure is not the engine's ceiling and the gap between them is not kernel quality
2026-07-30	Three kernel-level probes, all of which refuted the hypothesis they were built for, plus a profiler split that found where the expert-FFN time actually is	The bucket was 48-60 ms/token against a MoE block that measures 11.1 ms for 26 layers, so the question was where the other 5x went. Not DRAM: reading 24 distinct 1.6 MB matrices is *faster* than reusing one (86.2 against 75.0 GB/s), so the kernel is not cold-miss bound at these shapes. Not the untuned quantizations: `ffn_down_exps` is Q5_0 or Q8_0 and those kernels run at 121.6 and 106.6 GB/s against Q4_K's 87.5; the two written last are the fastest. What it is: the shared expert, which the profiler was counting inside `expert ffn` while it ran on the host, is 20.2 ms/token on its own: 34% of the bucket, ~273 MB at about 13.5 GB/s. Splitting it out leaves the routed experts at 39.7 ms against the block's 11.1, so a 3.6x gap remains unexplained and is now the next question rather than a guess. Folding the shared expert into the phase-parallel block was retried on the strength of that number and produced wrong output on mixed widths, so it is not shipped: uniform 1408 and uniform 2816 both agree with the oracle, mixed does not, and the cause is not yet found
2026-07-30	The shared expert joins `moeFfnBlock` as one more expert, with per-expert hidden widths. Expert FFN 60.0 -> 43.2 ms/token, token 121.2 -> 95.8	It is a 2816-wide dense FFN on every layer against a routed 1408, and on the host it cost 20.2 ms/token, a third of the expert-FFN bucket at about 13.5 GB/s. Folding it lost when the block was a per-expert chain, because it moved work onto a path where every step waited behind a barrier; against the four-phase block it is simply a seventh column and wins. The mixed widths this requires were reported wrong in an earlier attempt, which is why every shape handed to a kernel is now per expert and a test covers 1408 beside 2816 against the exact oracle: uniform-1408, uniform-2816 and mixed all agree, at three different router-weight combinations. Isolating each expert with the other's weight zeroed (both correct) is what showed the earlier failure was a transient build rather than the design

Date	Decision	Rationale
2026-07-30	The 3.3x between the MoE block in isolation (~13 ms/token) and in the engine (43 ms/token) is *not* GPU clock ramp and *not* the working set. A 1 ms host gap between calls costs 8%; rotating a 698 MB set of experts so nothing is reused costs 3%	Both were plausible and both are now measured rather than argued. The block was exercised at the shape a layer actually issues (seven experts, 35 MB per call), back to back, with a host gap matching a layer's attention, and against a working set twenty times its own size. It holds 71-85 GB/s in every case. So the remaining difference is neither how often the engine calls it nor what it reads, and the next candidate is the one structural thing left untested: the engine addresses every expert as an offset into a single multi-gigabyte arena buffer where the benchmark uses one small allocation each. That probe is written and does not yet run on this machine, so it is recorded as the open question rather than as a finding
2026-07-30	The profiler counts MoE layers that took the backend block against those that fell back to the host, and reports 26.0 against 0.0: the fast path is taken on every layer. A fifth explanation for the isolation/engine gap, arena buffer size, is also refuted: the same experts placed a gigabyte deep inside a 2 GB mapping run at 77.6 GB/s	The counter exists because a path declining silently has cost this project two long detours: the compute seam that never exported `moeFfnBlock`, and the missing Q5_0 pipeline that made the block refuse every layer of the one checkpoint it was written for. Both looked like slow code and were unused code. It is worth the two lines to never ask that question by inference again. What remains unexplained is the block measuring ~12 ms/token in isolation under every condition tried (back to back, with a host gap, against a 698 MB rotating working set, inside a 2 GB arena, all 71-85 GB/s) and 35.6 to 68.5 ms in the engine across runs. The one structural difference left untested is that the engine's experts are file-backed pages of the store mapping where every benchmark used anonymous memory; the heap-cache configuration that would isolate it cannot be compared without a warm-up long enough to retire the verify transient, which this machine's drift then swamps
2026-07-30	MLA attention has a Metal kernel and a device-resident compressed cache (`mla_attn.metal`, one head per threadgroup on the absorption identity). It runs, it is correct, and it is slower: attention 42.2 ms/token on the device against 17-24 on the host, and the token 114.6 against ~85. Left opt-in under `--gpu-ops`	Built to test a specific claim: that loom runs DeepSeek on the host because MLA attention had no kernel, so the residual stream could not stay on the device and every layer round-tripped activations across the bus. The kernel closes the first half of that and the claim is still not supported: submissions go 26.0 to 53.0 per token and the token gets slower, because each attention layer is its own command buffer ending in `commitAndWait`, which is not residency. Two silent gates had to be removed before it dispatched at all, both the same failure

Date	Decision	Rationale
		already fixed elsewhere in the file: allocating at model load before the Metal context exists, and then being handed the model's native 163,840 context rather than the capped 4,096, so it declined on shape. Each looked exactly like "unsupported shape". The open question is unchanged and now better posed: attention on the GPU was never the argument, keeping the token there was, and testing that needs the recorded whole-token path this kernel is a prerequisite for. Verified against an f64 oracle at 37 positions on a non-zero layer, and checked by dropping the rope term, which only makes attention position-blind and would otherwise read as slightly worse text
2026-07-30	Read from llama.cpp's source rather than inferred: `ggml_metal_graph_compute` encodes an entire graph into n_cb + 1 command buffers (n_cb is 1 at init, so about two per token) and calls `waitUntilCompleted` once per graph, splitting nodes across threads by `n_nodes_per_cb`. loom issues 53 per token and drains the GPU on each	This is the difference, and it was available to look up for the whole of the preceding work. It also explains the result immediately before it: putting MLA attention on the device added 27 more command buffers, each with its own `commitAndWait`, so it added drains rather than residency and duly ran slower. The profiler had been reporting the mechanism as a cost to accept (submissions 26 -> 53, ~13.9 ms) rather than as the thing to remove. The reason llama.cpp *can* encode a whole graph is the second half: ggml computes MoE routing on the device as graph ops, where loom computes it on the host and so must synchronize every layer. The plan that follows is therefore not "port `layerBlock` to MLA": record into the existing `beginFrame`/`endFrame` seam that the MLA and MoE paths currently bypass, move routing onto the GPU so the per-layer sync goes away, and only then is a comparison with llama.cpp like-for-like
2026-07-30	Fusing the absorption into `mlaAttnHeads` (two dispatches, one command buffer, `q_abs` never returning to the host) takes submissions from 80 to 53 per token and 118.8 to 98.5 ms. Still above the host path's ~85, because 53 buffers is still 53 waits	The first change in this direction to move the number the right way, and it confirms the cost tracks submission count rather than kernel quality. What it also shows is that the remaining halving is not more of the same: to put the MoE block in the same buffer as the attention before it, the routing between them must leave the host, and `llama-graph.cpp`'s `build_moe_ffn` says what that actually requires. Top-k there is a device graph op (`ggml_argsort_top_k`), the experts are fetched by `ggml_mul_mat_id(ctx0, w, cur, ids)` (one 3D tensor indexed by an id tensor on the device), and the selected indices

Date	Decision	Rationale
		are never read back. So "routing on the GPU" is not a step on its own: loom's MoE block takes explicit per-expert host pointers, and it would need a `mul_mat_id` equivalent before device-side routing buys anything. The distributed path complicates that further, since its experts are arbitrary offsets into a store rather than planes of one tensor, and would need an offset table on the device or to stay host-routed
2026-07-30	Expert selection moves to the device: `moe_route` (scoring, bias, top-k, renormalization; `ggml_argsort_top_k`'s role) and id-indexed matvec kernels for q4_k, q5_0 and q8_0 (`ggml_mul_mat_id`'s shape: one 3D tensor, planes picked by a device-side id buffer, per-slot activation stride for the down projection). Each verified bit-for-bit against the plain kernels across six out-of-order planes, and the router against `moe.route` across all eight gating/bias/normalization combinations	While the host picks the experts there is a read-back between a layer's attention and its FFN, so the two cannot share a command buffer whatever else is recorded. The earlier q5_0 failure (slot 0 right, slot 1 on wrong) was not the kernel: the test freed and reallocated fixtures at the same address and `wrapFor`'s cache handed the GPU a stale mapping of freed memory. A content check for that was written and reverted as vacuous (the buffer *is* the host memory through the same virtual address), so it is stated as `wrapFor`'s contract instead: wrapped memory lives as long as the context
2026-07-30	W_v joins the attention command buffer as an id-kernel dispatch with identity ids: head h's value rows are a plane at stride (nope+vd) rows, offset nope rows in, its input its own o_latent, which is exactly the per-slot x-stride shape built for the MoE down projection. One dispatch replaces sixteen per-head host matvecs, the last host step inside attention. On the first clean machine of the effort (rebooted; swap had sat at 8-10 GB through every prior measurement): 18.2 tok/s under --gpu-ops against 8.7 on the host path, identical text; attention 24.7 -> 11.8 ms/token, expert FFN 68.0 -> 30.7	The first same-conditions measurement where the GPU path wins end to end, and the margin is the accumulated structure rather than any one kernel: absorb+attention+W_v as one submission, route+experts+reduce as another, everything resident. The oracle test extends through W_v (a q4_k fixture with the real plane layout, f64 reference), so the whole attention buffer is covered by one assertion. Against the 38 tok/s bar (llama.cpp on an M3): 18.2 on an M5 with ~53 submissions per token still standing, and the remaining path (fusing the two per-layer buffers, then the cross-layer frame) is the measured-not-argued route to the rest
2026-07-30	`mlaLayerTail`: a whole MLA layer (absorb, attention, W_v, output projection, residual, FFN norm, router, routed MoE, second residual) as one submission. 54 -> 28 command buffers per token, and warm decode reaches 35.5 ms/token (~28 tok/s) against 54-64 without the tail, identical text. Pinned by a differential test against its own constituent buffers plus host glue	Three more silent gates had to fall before the tail fired at all, the fourth through sixth of the series: `gguf run` never called `mlaInit` (fixed with the capped context, since the native 163,840 declines on shape), never called `uploadAbsorbWeights` (so `li >= mla_wk.items.len` declined every call), and the router is one of the checkpoint's f32 tensors, for which no dmmv kernel existed; a 15-line `dmmv_f32.metal` closes it. Each presented identically: correct text, no error, submission counter unmoved, which is why `cb/tok` is now printed in the compact profile line and `LOOM_NO_TAIL` exists to bisect the tail against

Date	Decision	Rationale
		the two-buffer path inside one binary. Against the 38 tok/s bar: ~28 warm, with the cross-layer frame (28 -> ~2) the remaining structural step
2026-07-30	`mlaTokenFrame`: the whole token, every layer's attention head (norm, q/kv_a projections, cache norm, rope), attention, and FFN, then the final norm and lm_head, as one command buffer. 28 -> 1.8 buffers per token, 27.9-30.4 ms/token against the tail's 38.7-47.6, identical text: ~34.7 tok/s wall-clock including prefill, against the 38 tok/s bar from llama.cpp on an M3	This is `ggml_metal_graph_compute`'s shape reached: the residual stream lives in a device slot for the entire token and only logits return. The head coming off the host needed the previously merged prerequisites (the rope kernel oracle-tested against `ropeApply` itself, strided q in absorb/attention, `copy_f32`) plus the frame writing the compressed cache on-device with the rows mirrored back after each token, so a later fallback can never attend over stale positions. One trap nearly repeated the day's oldest mistake in reverse: the first frame measurement said *slower* (83.6 ms) and was the cold run paying pipeline compiles and every tensor's wrap; warm and alternating, the frame wins by ~30%. The frame declines, and the engine falls back per layer, on anything it cannot express: a router bias, the q-LoRA path, a type without a kernel
2026-07-31	A frame phase-split (`LOOM_FRAME_DEBUG`: each layer as four waited sub-buffers, per-category sums) found the absorb kernel launching sixteen threadgroups, one per head, on a device that wants hundreds, every thread crawling tens of kilobytes serially. Re-gridded to one SIMD group per output element (8,192 groups on the real model): 41.4-42.9 tok/s, past the 38 tok/s llama.cpp-on-M3 bar, identical text	The rewrite briefly reproduced the codebase's canonical bug in a third place: three call sites dispatch the absorb pipeline, the two single-line ones were updated mechanically, and the multi-line one in `mlaAttnHeads` kept the old grid; with a threadgroup of 64, the new kernel computed a fraction of the outputs and left the rest stale, which is a plausible residual stream. Both the f64 oracle and the layer-tail differential failed on it immediately, which is the whole reason they exist. The phase ledger that motivated the change: head 10.0 ms, attn 13.7, proj 9.2, ffn 33.1, lm_head 2.9 in split mode, against ~1 ms of actual matvec work in the head: occupancy, not bandwidth
2026-07-31	Three squeezes from the same ledger: the attention weighted-sum re-gridded to one SIMD group per output element (the fused kernel kept sixteen threadgroups for the bandwidth half of attention; fine at seq 40, collapsing at seq 4,096); W_k stored f16, halving the largest reducible read outside the experts (~113 -> ~56 MB/token); the six per-slot SwiGLU dispatches merged into one and a provably-false barrier removed. Best run 44.4 tok/s, band 39.7-44.4 against the prior 41.4-42.9	At a 40-token context these are ~1 ms effects and the bands overlap; the wsum re-grid is the one that changes the long-context story, since the O(seq*kvr) sum is what grows. The attention split reproduced the canonical bug an eighth time: the layer tail's own attention dispatch, a third call site again, kept the fused semantics and fed probabilities to W_v as if they were o_latent; the layer-tail differential caught it before any manual run, same as the seventh. f16 rounding of W_k passes the f64 oracle unchanged: rounding a weight already

Date	Decision	Rationale
		quantized to ~4.5 bits is noise against the quantization itself
2026-07-31	The Vulkan backend exists (issue #13): a dlopen-loaded device layer (~25 functions, hand-declared; no SDK at build, no driver required at run, declines cleanly without either), a GLSL Q4_K matvec compiled to committed SPIR-V, and a seam-complete backend that resolves everything to the CPU except that one kernel. All 83 tests pass on llvmpipe on the Hetzner box, including the f64 oracle with silent-fallback detection	Correctness only, by explicit agreement: llvmpipe is software Vulkan and the only Vulkan this project currently has, so every performance decision waits for physical hardware. The bring-up deliberately repeats the Metal backend's day one (hottest kernel first, oracle before use, everything else declining to the CPU) because the expensive lessons (silent gates, fallback detection, pinned scales, alignment) were already paid for there; the SPIR-V alignment panic on the first run was caught by the test in one round for exactly that reason. dlopen rather than -lvulkan is what lets the macOS host cross-compile the Linux test binary it cannot itself run
2026-07-31	Vulkan grows to the full dmmv family (q5_0, q8_0, q6_k and f32 GLSL kernels alongside q4_k, ported from the token-exact CPU/Metal arithmetic), and weights become device-persistent: a pointer-keyed upload-once cache (length mismatch re-uploads; mmap'd tensors make staleness impossible) replaces the bring-up's re-upload-per-call. The oracle test now loops all five types and uses the submission counter, not output comparison, as fallback detection	Pre-rental threshold work: these are the kernels DeepSeek-V2-Lite's token path actually touches (q4_k/q5_0/q8_0 experts and projections, q6_k lm_head, f32 router), so a green fused-layer differential on llvmpipe, not a rented GPU, is the gate for renting one. Memory-type choice stays behind `Device.alloc`, the single seam where DEVICE_LOCAL + staging lands when real hardware exists to measure it
2026-07-31	The Vulkan routed-MoE chain is complete and differentially verified: `moe_route` (score, bias-shifted top-k, unbiased gates, f16-clamped renorm), id-indexed matvecs for q4_k/ q5_0/q8_0 (`ggml_mul_mat_id`'s role), device SwiGLU and gated reduce, shared expert; ids and gates never return to the host between route and reduce. Verified against the shipped host router and an f64 dequantize-everything reference exercising all three id kernels plus the shared expert; 85/85 on llvmpipe. This meets the pre-rental threshold	One bring-up difference from Metal, deliberate: each step is its own submit-and-wait rather than one recorded command buffer, because the minimal device layer only has `dispatchWait`. The kernels are what a real GPU will run; the submission shape is not, and fusing submissions is a performance decision that waits for hardware. With the differential green, renting an NVIDIA box is now justified: first hardware task is DEVICE_LOCAL memory + staging behind `Device.alloc`, then measuring
2026-07-31	**MLA attention on Vulkan**: the compressed-cache attention (absorb, scores+softmax, weighted sum, W_v as an id dispatch with identity ids) and `mlaLayerTail`: attention through output projection, residual, FFN norm, router and the routed MoE chain as one recorded submission per layer. New GLSL: mla_absorb, mla_attn, mla_wsum, rmsnorm, add_vec; id kernels grew a `base` push constant (Vulkan binds whole buffers, so the W_v row offset and	The submission ledger, as always: the tail collapsed a layer's ~15 waits into one, and the decode profile now splits ~75% attention-bucket (26 tail submissions x ~2 ms each of submit-and-wait overhead); on this discrete GPU a submission costs ~2 ms against Metal's ~260 us, which makes cb/tok 59 the whole story. The two remaining ports are known quantities: the whole-token frame (Metal: cb/tok 1.8) to collapse 59 submissions to ~3, and GPU prefill

Date	Decision	Rationale
	per-layer cache offsets ride in push constants; the c/k_rope caches share one buffer for the same reason). Differentials: attention vs an f64 dequantize-everything reference with decoy rows in the other layer's cache region; tail vs its verified constituents. 87/87 on both the RTX 3060 and llvmpipe. End to end on the 3060: 7.1 tok/s over 128 tokens (2.3x CPU), ~115 ms/token marginal decode, cb/tok 59	(the prompt still runs the CPU batched path at ~690 ms/token, which is why short generations read slower than the marginal rate)
2026-07-31	**The whole token as one Vulkan submission** (`mlaTokenFrame`, the port of Metal's): per layer the attention head (attn norm, q/kv_a projections, kv_a norm straight into the cache row, rope on q and on the cached k_rope; new `mla_rope` and `copy_f32` kernels, ropeApply-faithful with YaRN), the compressed-cache attention, projection/residual, and the FFN (routed-with-shared-expert or dense), then final norm and q6_k lm_head. cb/tok 59 -> 1.0; 3060 end-to-end 7.1 -> 9.7 tok/s (tail path same binary via LOOM_NO_FRAME: 6.0), ~90 ms/token marginal at 90%+ GPU utilization, weights fully VRAM-resident (10.1 GB). Verified by a whole-token f64 differential (two layers, dense + MoE-with-shared-expert, cache rows checked because a frame that attends right but caches garbage only fails on the *next* token) plus a determinism probe; 89/89 on the 3060 (30-run soak, 0 failures) and llvmpipe (15 runs)	Two lessons paid for. First: an intermittent 1-in-5 wrong-output on the 3060 was misread as a GPU barrier race for a day: widening the barrier to ALL_COMMANDS "fixed" a 10-run sample, and a 30-run soak then showed 6 failures anyway. The real bug was the pointer-keyed weight cache serving stale device tensors when TESTS free and reallocate same-shape buffers at recycled addresses; mmap'd model weights cannot hit it, tests now clear the cache, the barrier went back to its spec-correct compute scope, and the soak is the only sample size that counts. Second: with one submission per token the profile stops being about submission overhead at all: 90%+ utilization at ~90 ms/token means the correctness-shaped kernels themselves (scalar loops, one workgroup per row) are now the frontier, along with CPU prefill. That is where Metal's occupancy lessons finally apply to Vulkan, with hardware to measure them on
2026-07-31	**Vulkan kernel-efficiency pass**, driven by a ported LOOM_FRAME_DEBUG phase split (head/attn/proj/ffn/lmhead sums per token). Round one: the whole dmmv family vectorized: weights as u32 word loads (an unaligned-tail-safe helper for the 22/34/210-byte block formats, buffers padded a word), activations as vec4; 9.7 -> 15.6 tok/s, ffn phase 54 -> 23 ms, lm_head 5.5 -> 2.0. Round two: absorb and wsum re-gridded from one-workgroup-per-output-element (every read a 64-way scatter) to one workgroup per (head, 64-output tile) with threads walking rows serially (the coalesced shape), plus vec4 attention dots and a 256-thread rmsnorm: attn phase 14.1 -> 5.4 ms. End to end: 19.8 tok/s over 256 tokens; marginal decode 36.6 ms/token = 27 tok/s. 89/89 on the 3060 and llvmpipe	The two rounds are the two classic GPU sins, caught in order of cost: uncoalesced/narrow loads (byte-at-a-time from a 360 GB/s device) and scatter-shaped grids (the same re-grid Metal paid for at 16-threadgroups-per-absorb). The oracle suite is what made both rewrites safe to do in an afternoon: every kernel's arithmetic is pinned by an f64 differential, so a vectorization slip fails a test instead of shipping plausible text. Remaining, in measured order: the MoE id kernels still run ~57 GB/s effective (row-batching, NR0=4 as Metal does, is the known next step), and prefill still runs the CPU batched path
2026-08-02	**Three pre-configured networks: devnet / testnet / mainnet** (ids 1337 / 2 / 1, protocol	One-network-one-model graduates from convention to configuration: the registry makes the

Date	Decision	Rationale
	constants now): `--network devnet` resolves the stable id and the canonical model from an in-binary registry, Ethereum's chain-registry shape for LLM networks. devnet = GLM-4.5-Air (106B-A12B, ~66 GB Q4; verified "loads and runs", distributable across the existing three-machine fleet with zero new hardware); testnet = GLM-4.6 (357B-A32B, ~200 GB Q4, verified likewise on the existing glm4moe engine); mainnet = GLM 5.2 (glm-dsa, gated on the engine variant), with Kimi K3 the recorded alternative. Arch policy enforced at startup: testnet/mainnet REFUSE a mismatched model, devnet warns	invariant that RAG text-only gossip and expert fetch both lean on into something a node enforces before serving a single token. The phase ladder is deliberate: every claim validates on devnet at 106B on hardware already paid for, performance proves out on testnet at 357B on one 256 GB box, and mainnet waits for exactly two named gates (the glm-dsa engine, task #28, and a hardware approval) rather than an open-ended "someday"
2026-08-02	**GLM 5.2 readiness audit** (docs/GLM52-READINESS.md, run with loom's own `gguf check` against live checkpoints): GO with one bounded engine variant. GLM 5.2 GGUFs exist (2-bit ~239 GB to 4-bit ~475 GB); the arch is `glm-dsa`, expert-aligned; loom's sharder distributes it already; llama.cpp mainline itself runs it with a dense-attention fallback, so loom's delta is a deepseek2-family variant (MLA, noaux_tc, shared expert all in hand) that maps the tensors and skips the DSA indexer + MTP at load. And the bridge is free: DeepSeek-V3-0324 Q4_K_M (671B, 256 experts) passes `loom gguf check` with "loads and runs" on the existing deepseek2 engine: thesis-scale distribution with zero engine work, awaiting only a hardware decision (512 GB box for 4-bit, 256 GB for 2-bit)	The audit method is the point as much as the result: fifteen minutes with `gguf check --header-only` against remote URLs answered what could have been a week of speculation; the tool built for integrity checking doubles as an architecture-compatibility probe. DSA sparse attention is deliberately deferred as a speed feature, exactly as llama.cpp mainline treats it
2026-08-02	**The three-machine test** (Hetzner CPU origin 100%, Canadian RTX 3060 node 33%, NAT'd macOS laptop 20%, v0.12.0 binaries): expert-sharded serving verified against the manifest root on all three; wrong-network node (--network-id 999) dialed the origin for 45 s and never entered the mesh; RAG chunks propagated in every direction including the NAT pull; and the full loop closed: facts ingested only on the laptop were gossiped out, retrieved by the origin (prompt 28 -> 81 tokens with prepended context) and answered correctly by the distributed model, where the same question minutes earlier had produced a confident hallucination. Churn recovery observed live: the origin was OOM-killed mid-test and both joiners redialed and re-formed without intervention	Four findings, each now owned: (1) the network-id auto-derivation ran before the store existed, so every node silently derived id 0; consistent, so the mesh still gated correctly, but the manifest default was dead; fixed, computed after the store is final. (2) The RAG store is memory-only and the OOM restart wiped every node's chunks; persistence filed. (3) The origin footprint at --ram-gb 3 re-confirmed the documented 0.5 guidance for small boxes. (4) Cross-continent expert fetch inside the token loop is as slow as the design doc's fabric table predicts: WAN distribution buys capacity, not speed; the RAG-augmented longer prompts amplify prefill fetch cost, which argues for the shared-prefix-KV feature already sketched

Date	Decision	Rationale
2026-08-02	**RAG pull direction (RagInvReq, 0x23)**: planning the three-machine test surfaced a protocol hole: the exchange was push-on-dial only, so a NAT'd node (a laptop) could send chunks on its outbound dials but never receive any. One dial now converges both directions: push (Inv -> Want -> Push) then pull (InvReq -> Inv -> Want -> Push), all server handlers stateless	Found by walking the deployment before running it: the Mac in the three-machine topology can only dial out, and the propagation asymmetry would have shown up as "gossip works but my laptop never gets chunks". SPEC updated
2026-08-02	**The vector index extracted to zigstack/vector-index** and consumed as a package: the flat exact scan (implementation of record, static everywhere) and the FAISS dlopen accelerator now live in their own repo with their own CI and an agreement test between the two paths; loom's store keeps text, hashes, compression and gossip, and delegates vectors and search to the dependency. Vector storage improved in the move: one contiguous row-major buffer instead of a slice per chunk, the scan's cache locality	Shape 3 of the make-FAISS-a-dependency discussion, chosen over vendoring FAISS's C++/BLAS/OpenMP stack: the package is where HNSW lands when ANN scale demands it, benefiting loom and any other Zig consumer, and the accelerator/reference agreement contract travels with the code that must honor it
2026-08-02	**Brotli vendored as a build-time dependency**: the pinned v1.1.0 C sources compile into every release target through build.zig (pure C, zero dependencies, MIT), so at-rest chunk compression is always available and the dlopen probe is gone; the bindings became two linked externs and the test now asserts compression actually happens, not merely that the path degrades gracefully. FAISS stays dlopen by explicit contrast, recorded in build.zig itself: C++ + BLAS + OpenMP has no place in a four-target musl-static cross build, and at loom's 64K-chunk cap the exact scan is the same algorithm FAISS-Flat runs; FAISS engages when installed as a BLAS-accelerated flat search, and its ANN families (the reason to ever vendor more) would be better answered by an HNSW in Zig if scale demands it	The rule this pair sets for future native code: vendor what Zig's own toolchain can cross-compile statically everywhere; dlopen what cannot, with a pure-Zig fallback that keeps the feature testable on CI. Verified: musl x86-64 and arm64 cross builds and the native suite all green with brotli linked
2026-08-02	**RAG storage and convergence, answered precisely**: (1) FAISS stores nothing compressed here: the flat inner-product index holds raw f32 vectors by design (its PQ/SQ compressed families are unwired), and chunk *text* never enters FAISS at all. (2) Compression now exists at both layers where it pays: the wire already ran every RAG frame through the frame encoder's adaptive snappy (kept only when smaller); at-rest chunk text is now brotli-compressed when `libbrotlienc/dec` are present (dlopen, the FAISS/Vulkan loader trade), with raw storage as the fallback. At-	Brotli over snappy for at-rest text is the right split: snappy stays the wire codec (speed, already adaptive), brotli's ratio wins on stored prose, and neither becomes a dependency; both decline to raw cleanly. The rotation bug is worth its whitepaper line: "gossip to all peers" was true per-round from day one, but eventual convergence for late joiners needed the window to move

Date	Decision	Rationale
	rest format is purely local: hashes are of raw text and Push carries raw text, so mixed-library networks interoperate unchanged. (3) Chunk additions ARE a global topic: the Inv/Want/Push exchange runs against every known peer each gossip round plus transitive re-gossip; the inventory window now rotates when a store exceeds the 512-hash round cap, closing the hole where a rejoining peer could never learn chunks older than the newest window. Every hash is advertised within ceil(count/cap) rounds; a test pins the full-coverage property	
2026-08-02	**Gossiped RAG chunks with FAISS-accelerated retrieval** (`--rag`): nodes keep a chunk store searched by cosine before every prompt reaches the engine (top-k prepended as context); new chunks, ingested via `POST /v1/rag/chunks` or learned from peers, reach the whole network through an Inv/Want/Push exchange piggybacked on the gossip round. FAISS rides behind dlopen (`libfaiss_c`, the Vulkan-loader trade: no build dep, no runtime requirement) with an exact-scan fallback that is the reference implementation and the CI-tested path. The load-bearing design choice: only text travels. One network serves one model (network_id), so every node recomputes the identical embedding (mean-pooled token_embd rows, L2-normalized) from the same text; vectors are never accepted from the wire	The two features lock together deliberately: network_id is what makes text-only RAG gossip sound, because embedding equality across nodes is exactly the same-model guarantee the membership gate enforces. Poisoning reduces to what it should be: a peer can contribute bad *text*, visible and dedupable, never a bad vector behind good text. Caps bound every round (512-hash inventory, 32-chunk push, 8 KB chunks, 64 K store); eventual convergence across rounds. Embedder quality is the known v1 trade: mean-pooled input embeddings are crude, and swapping in a real embedding model later changes only `setEmbedder`
2026-08-02	**network_id: chainId semantics for LLM networks** (wire proto v2): one loom network serves one model, and every Heartbeat/Announce now carries a u64 network identity. Peers on a different network are refused (`ERR wrong_network`) at the p2p surface and dropped at gossip merge, so a record from another LLM network can never enter the mesh table, however it arrived. `--network-id N` configures it; the default derives from the weight manifest's leading eight bytes, so nodes sharding the same model agree with zero configuration	The split of duties is deliberate and mirrors Ethereum: network_id gates *membership* (stable across hardforks when set explicitly), manifest_version keeps gating *content* (expert fetch already refused cross-version requests). The planned ENR record gains network_id alongside the manifest fields. SPEC.md updated as the governing document
2026-08-02	**glm-dsa loads and decodes (dense fallback) -- the GLM 5.2 engine gate opens**: the deepseek2 engine now accepts arch glm-dsa with arch-prefixed config keys and the per-head sizes under key/value_length_mla, and supports the split attn_k_b/attn_v_b tensor pair	Correctness first, sparsity second: dense attention over MLA is exact at any context, just O(seq); the lightning indexer (colibri port three proper, with its full-selection==dense validation recipe) now has every tensor name

Date	Decision	Rationale
	the real conversion ships instead of the fused attn_kv_b (k_b arrives transposed, which turns the absorb step into one plain matvec per head). Grounded in the actual unsloth GLM-5.2 GGUF: the 9 MB metadata shard was pulled and every key and tensor shape read from it, including the indexer family (indexer.attn_q_b/attn_k/k_norm/proj; 32 heads, key 128, top-k 2048) which the dense path tolerates and ignores -- llama.cpp-mainline parity. Two new suite gates: split-vs-fused token-exact equality on the in-memory MLA fixture, and a generated glm-dsa GGUF loading and decoding end to end. GPU fast paths stay fused-only; split files run host attention	and shape it needs recorded here rather than guessed
2026-08-02	**MTP speculative decode, ported from colibri (port two of three)**: the NextN block GLM ships (and loom previously skipped) drafts the token after next from (current residual stream, embedding of the token just chosen); the main model verifies the draft in one 2-wide batched forward. Greedy-only by design: acceptance is an argmax comparison of the same arithmetic, so MTP-on output is token-identical to MTP-off by construction, and the suite enforces exactly that with a differential test against a fixture that now carries the NextN glue. The NextN layer runs host-side on its own KV lane (kept contiguous by a backfill forward on every accepted draft), its experts sit in the resident bundle so drafting never fetches, and LOOM_NO_MTP=1 disables. A tokens-per-forward line prints alongside the expert-tier stats	On the devnet the verify batch unions expert fetches across the drafted pair, so each accepted draft amortizes the WAN miss cost of two tokens over one fetch round -- aimed squarely at the measured 0.1 tok/s regime. Colibri reports 2.2-2.8 tokens/forward when acceptance pays and warns that int4 MTP heads collapse acceptance (their #8); our head is stored Q4_K, so the acceptance rate measured on the devnet decides whether the head gets dequantized at load. Overhead is real on compute-cheap models (the fixture runs slower with spec on), which is why the greedy gate and the kill switch exist
2026-08-02	**PILOT router-lookahead prefetch, ported from colibri** (JustVugg/colibri, Apache-2.0 -- the project whose expert-streaming technique seeded loom v0, since grown to a mature engine that validated this exact lever at 71.6% one-layer-ahead routing predictability): at each MoE layer the engine runs the next layer's router on the current hidden state and fire-and-forget prefetches the predicted experts (bounded pool of 8, lossy by design) while the current layer computes; the real router then scores the prediction, and the measured hit rate prints with the expert-tier stats. LOOM_NO_PILOT=1 disables for A/B. Mispredicted fetches still persist to the store, so even a miss warms the node	The fetch for layer L+1 otherwise starts only after L completes, serializing network stalls behind compute; overlapping them attacks both the WAN-miss case and SSD streaming. Verified live on a two-node distributed run (52/52 confirmed on the synthetic model; the GLM devnet number is the next measurement). MTP speculative decode and the DSA indexer are the next two colibri ports, in that order

Date	Decision	Rationale
2026-08-02	**Cold-start, part two: delegate-while-cold and batched shard streaming**	A node below its holdings threshold (default 0.5) forwards generations over a new GEN wire command to the warmest live peer (>=0.9 held) and relays the answer, badged in the API response and the chat UI; any failure falls back to local generation, so the path can only help. Fixes the alpha's worst first-impression: a fresh joiner now gets full-quality answers at the warm peer's speed from minute one while its own store fills. Alongside it, PACKR batches up to 256 shard fetches into one round trip (DATA/ABSENT stream, END terminator), removing RTT-times-shard-count from every sync; both commands degrade cleanly against older peers, and every shard stays digest-verified regardless of who sent it or in what batch. Verified on a two-node swarm: a 10%-held joiner's chat answered by the origin (loom_delegated in the response, console lines on both ends) over a PACKR-batched sync
2026-08-02	**Cold-start, part one: hot-first sync order and generation-priority repair**	Two changes from the first night of real alpha traffic. Sync order: nodes count which shards they serve (GETR bumps a per-shard counter) and answer a new HEAT query with the hottest ids; a syncing peer fetches resident chunks, then the heat list, then the tail, so the Zipfian head of the corpus arrives first and a joiner is useful after a fraction of the transfer. Repair priority: the eager-repair loop now yields while a generation is in flight, because both share one downlink and the interactive request is the one a human is waiting on. The hint is untrusted by construction: it can only reorder a transfer whose every shard is digest-verified
2026-08-02	**Alpha kit: opt-in numeric telemetry over the existing wire** (`METRICS <json>` to the bootstrap peer, once a minute; collectors opt in with `--alpha-ingest`, everyone else refuses with `ERR no_ingest`) plus two bring-up fixes: unknown flags under `loom node` now error instead of silently starting a node (#179), and the p2p listener probes its port before binding, refusing to share it with another process (#180, the SO_REUSEPORT split). The report is one JSON line of operational numbers; the field list in docs/ALPHA.md is the contract with testers, and prompts, text, and RAG content are structurally absent	The alpha needs fleet-wide evidence (tok/s vs hold fraction, hit rates, sync behavior across real links) and the grant application needs the same numbers. Reusing the p2p line protocol avoids a second transport and keeps ingest consent explicit on both ends: senders opt in per node, collectors opt in per file
2026-08-01	**Devnet bootnode is 168.119.167.242:8771** (registry updated; supersedes the	The operator provisioned a different box: 4 vCPU, 8 GB RAM, 150 GB SSD. RAM is

Date	Decision	Rationale
	REDACTED-IP entry from earlier today, which was never reachable)	sized for origin duty (shard serving at `--ram-gb 0.5`), not fast local inference; the disk holds one merged model copy with room to spare, and the split halves never land on it (merged over sshfs from the old box)
2026-08-01	**The shared retrieval plane written into the paper** (Section 4 subsection): text-only chunk gossip, one-network-one-model embedding equality, dedup by content hash, brotli at rest, the NAT pull direction, and the measured three-machine retrieval result. Expiry policy recorded: chunks never expire today; the planned mechanism is an optional ingester-stamped TTL carried in the hashed envelope (network-wide consistent expiry, no tombstones) plus use-ranked eviction under the store cap, deliberately not a blanket TTL	The feature existed in SPEC and the log but not in the paper's narrative; a reader of Sections 1-13 would not have known the swarm shares retrieval context at all. The TTL decision is recorded now because gossip makes deletion a design problem, not an afterthought: without expiry-in-envelope, any peer still holding a chunk resurrects it
2026-08-01	**LocalAI added to related work** (Section 2 table, a paragraph, reference [15])	It is the most complete self-hosted OpenAI-compatible server and it does run across machines, so the comparison must be made precisely: federated mode needs a full replica per node (throughput, not capacity), and worker mode is llama.cpp RPC layer-split (activations on the serial path, slices fully resident). Neither offers partial stores, in-flight weight verification, or churn repair, which is the axis Loom occupies; on every other axis (modalities, backends, maturity) LocalAI is broader
2026-08-01	**Devnet bootnode moves to a dedicated box (REDACTED-IP:8771)**; the registry entry is updated in the same change	The first box (REDACTED-IP, 75 GB disk) cannot hold the merged 73.7 GB GLM-4.5-Air Q4_K_M alongside its split download halves, and hardfork upgrades need room for two model versions besides. The registry is the single place a joiner learns the entry point, so the IP change is one line
2026-08-01	**Results sections brought up to measured reality** (Abstract, Section 10, Section 11, Conclusion): the paper no longer describes scalar-kernel localhost validation. Recorded as evidence: 57.8 tok/s CPU (53x scalar), Metal 39.7-44.4 tok/s, Vulkan ~32-34 tok/s (RTX 3060, gap to llama.cpp localized), the two-machine WAN run (25.8 ms RTT, 3.6% store, token-identical, 1,063 digest-verified fetches), and the three-machine two-continent swarm (100/33/20% stores via node RPC, wrong-network rejection, RAG gossip including the NAT pull path, retrieval correcting a hallucination,	A whitepaper that understates its own evidence is as stale as one that overstates it; the limitations section now lists the real ones (no continuous batching, glm-dsa unimplemented, split GGUF unread, RAG store memory-only, three-node scale) instead of ones already fixed

Date	Decision	Rationale
	unaided churn recovery after an origin OOM kill)	
2026-08-01	**License: Apache 2.0 with a NOTICE file**	Chosen for the attribution requirement: every redistribution must retain the copyright notices and carry the NOTICE attribution text forward, which MIT does not provide; the explicit patent grant matters for an inference engine; and it stays compatible with the MIT-licensed ecosystem (GLM weights, llama.cpp, all deps). Visible in-product credit is deliberately not required; badgeware-style clauses deter adoption
2026-08-01	**The network registry carries default bootnodes, and joining the devnet is one command**: each named network entry may list bootnodes; `--network <name>` with no explicit `--bootstrap` and no local GGUF dials the registry default (printed as such), so `scripts/join-devnet.sh` reduces to install-if-missing plus `loom node --network devnet` with NAT-safe defaults (hold-fraction 0.2, RAG on, advertise unset). Explicit `--bootstrap` and origin mode (`--gguf`) always win over the registry default	A network whose id, model, and entry point all live in the binary makes "join the devnet" a zero-configuration act, which is what a PoC network needs to grow; keeping the override order (explicit flag > registry) preserves every existing workflow. Devnet's registry bootnode is the Hetzner box (REDACTED-IP:8771)
2026-08-01	**Windows becomes a release target** (`x86_64-windows-gnu` cross-build, published as a zip and smoke-gated on a real Windows runner before publish; Docker images stay linux/amd64 + linux/arm64): five portability seams closed: argv arrives as WTF-16 and is decoded through the std `Args` iterator; RSS reads peak working set; the store's read-only weight map uses `CreateFileMappingW`/`MapViewOfFile`; monotonic timing goes through a shared `nowMonoNs` (`QueryPerformanceCounter` on Windows, `clock_gettime` elsewhere); and both dlopen probes grow `LoadLibrary` branches (`vulkan-1.dll` here, `faiss_c.dll` in vector-index v0.1.1)	The port touched only OS seams the code had already isolated; no engine, wire, or storage logic changed, which is the design working as intended. The pread fallback that covers an unmappable store now also covers nothing: Windows maps too. WSL2 remains the documented route for anyone wanting the Linux build
2026-08-01	**SwiGLU folded into the expert down projection** (pipeline layout widened to five bindings): the fused q5_0/q8_0 down kernels compute silu(gate)*up where it is consumed: one dispatch and one full drain fewer per MoE layer. Best 192-token average yet (39.5 ms cumulative); 89/89 on both platforms over three runs. This closes the Vulkan single-node optimization phase. Final state: ~32 tok/s end-to-end, marginal in the 13-15 ms band (~70 tok/s),	The phase ledger, complete: 0.6 -> ~32 tok/s across 17 merged PRs; every kernel f64-oracle-pinned; ten negative results recorded; the remaining ~1.5x localized to ~300 barrier drains whose no-barrier ceiling (8.5 ms = 117 tok/s) is proven on this card. The recorded path there, norm-into-consumer fusion then the layer-megakernel shape (llama.cpp's ~7-node granularity), is an architecture project, deliberately deferred: the local path now exceeds the

Date	Decision	Rationale
	past the M5 Metal path's 44.4, at ~60-65% of llama.cpp's 112 on the same card	roadmap's needs, and v1's distributed work (the thesis) resumes with this foundation
2026-08-01	**The value side fused: wsum + W_v in one kernel per head**: o_latent accumulates in shared memory (its only consumer is the same workgroup) and the W_v rows read the quantized kv_b plane directly, q4_k and q8_0 variants from one template so the existing differentials exercise the shape. One dispatch, one buffer and one full drain fewer per layer; with the absorb fusion, the attention block is down from four dispatches/four barriers to two/two. All 89 tests on both platforms, generation verified	Where the fusion series stands against the 8.5 ms / 117 tok/s no-barrier ceiling: attention is consolidated; the remaining barrier mass sits in the FFN chain (route/gate-up/swiglu/down/reduce, structurally serial) and the head (norms, rope, projections). The norm-into-consumer fusion and a 5-binding pipeline layout (to fold swiglu into the down kernel) are the next two items; past those, the ceiling likely needs the layer-megakernel shape rather than pairwise fusion. Marginal sits in the 13.8-15.7 ms band run-to-run on this box: thermal variance now exceeds single-fusion deltas, itself a sign the easy wins are spent
2026-08-01	**Absorb fused into the attention kernel** (dispatch-consolidation series, opener): q_abs computes into shared memory inside the per-head attention workgroup, its only consumer, deleting the separate absorb dispatch (27/token). Net time neutral: the deleted barrier was re-spent explicitly ordering the cache-row rope before attention (a 3060-only race the frame differential caught deterministically: llvmpipe green, NVIDIA red, the cache-row check firing exactly as designed). Five suite runs clean after the fix	Bookkeeping for the series: each fusion must remove a barrier, not just a dispatch, to collect the ~15 us; the next pairs (wsum+W_v, norms into consumers, rope into the head group) are chosen on that criterion. The 8.5 ms no-barrier ceiling (117 tok/s) stands as the target
2026-08-01	**Integer-quantized activations: complete, correct, neutral; and a decisive barrier measurement**: quant_act (f32 -> q8_1-style blocks) + an integer-dot q4_k id kernel where the masked quant word is the packed i8x4 (dotPacked4x8, zero decode ALU), capability-probed (VK_KHR_shader_integer_dot_product; llvmpipe untouched), env-gated LOOM_VK_INT8 for gate/up. Generation fluent with the expected int8-class divergence; performance neutral: those kernels were already bandwidth-saturated, confirming Ampere's spare ALU was never the constraint. The decisive by-product: re-running LOOM_NO_BARRIER at today's speed shows 8.5 ms/token without barriers = 117 tok/s, llama.cpp's exact territory. Barriers now cost ~5.4 ms/token (39%), up from 7% relatively when tokens took 35 ms	The remaining 1.6x is now one quantified thing: ~350 full-pipeline barrier drains per token at ~15 us each. The fix is dispatch-count consolidation: fusing absorb into attention, norms into consumers, fewer/wider ops per layer, llama.cpp's ~7 nodes per layer against loom's ~20 dispatches. That is the whole remaining roadmap to 112

Date	Decision	Rationale
2026-08-01	**Reduce fused into the residual stream**: `moe_reduce_dev` gains an accumulate flag; the frame's routed reduce writes straight into x, removing one dispatch and two barriers per MoE layer (the standalone routed block keeps fresh-write semantics, flag 0). Verified over three suite runs on the 3060 plus llvmpipe; marginal holds ~14 ms. First of the small-dispatch-tail series	The tail-shrinking continues (adds and copies remain), and the integer-activation kernel family (llama.cpp's q8_1 activations + packed integer dots) is the other named remainder on the way to 112
2026-08-01	**q5_0/q8_0 padded quants-first at upload** (22 -> 24 and 34 -> 36 byte strides, quants first so every load is a direct word read; a `paddedLen` helper corrects every plane-stride and row-bytes computation, which the q6_k round showed must be type-keyed at every site). Kernel ticks: q5_0_id −15%. The first measurement looked like a 10 tok/s regression: the repack loop (millions of 16-byte memcpys through fresh page allocations) had doubled warmup; rewritten with fixed-size copies into a reused scratch buffer. Marginal decode ~14 ms/token (~72 tok/s), best yet; end-to-end 31.7 with warmup recovered	The gap to llama.cpp's 112 now decomposes to the two named remainders: their integer-quantized activations (q8_1 activations against quantized weights with packed integer dots; loom dots f32 activations throughout) and the long tail of tiny serialized dispatches between full barriers. Both are on the queue in that order
2026-08-01	**q6_k blocks padded to word alignment at upload** (next item on the profile's ranked list): the 210-byte on-disk block defeats aligned loads, so the device copy pads each block to a 224-byte stride (+7% VRAM on the affected tensors) and the kernel's quant loads become direct word reads. Kernel ticks -16%; end-to-end holds the 32-34 tok/s band. The build also caught a classic: the first padded version repacked only the matvec/lm_head sites while Q4_K_M stores several *attention* tensors as q6_K; the oracle passed (its path padded correctly) and real generation printed garbage. Every WeightRef now routes through one type-aware fetch, and the generation check is part of the ship gate alongside the suite	Two lessons: a padding scheme must be keyed by tensor *type* at every site, not by call path: one helper, no exceptions; and an oracle can be green while the model is broken if the bug lives in which-buffer-was-uploaded rather than in kernel arithmetic. The ranked list continues: q5_0/q8_0 id down kernels (same padding treatment applies: 22 and 34-byte blocks), then the serialized chain
2026-08-01	**Post-VRAM profile acted on**: with the working set device-local the kernel table transformed (the f16 gate/up kernels now run ~250 GB/s) and exposed `moe_route` costing 0.5 ms/token: 26 single-thread sigmoid loops per token. Scoring now parallelizes across the workgroup (selection stays serial, as designed). 34.4 tok/s end-to-end, ~14.5 ms marginal (~69 tok/s); 89/89 both platforms	Ranked remainder by the same table: q6_k (16%, 210-byte blocks defeat uvec4 alignment; needs its own shape), the q5_0/q8_0 id down kernels (15%, still word-assembly loads), then the frame's serialized chain. Each now has a per-kernel tick count to be judged against

Date	Decision	Rationale
2026-08-01	**The working set moves to VRAM**: only the *weights* were device-local; every activation, expert slot, attention intermediate, and the 226 MB compressed cache lived in host-visible system RAM, so all ~600 dispatches per token paid PCIe first-touch latency on their inputs: a tax no kernel shape could hide, which is why ten shape experiments measured neutral. Phase 1 (intermediates device-local, `download` primitive for the non-frame readbacks): 20.6 -> 27.6 tok/s, id kernels collapse from 26% to 2% of GPU time. Phase 2 (cache device-local, a host-visible mirror strip fed by two in-frame copies so the engine's post-frame readback stays a memcpy): 32.1 tok/s end-to-end, 15.7 ms/token marginal = 63.7 tok/s, decisively past the M5 Metal path's 44.4. 89/89 on both platforms, three suite runs on the race-sensitive changes	The instrument that found it was the per-kernel profiler showing a *uniform* deficit across all kernels: the signature of an environmental cost, not a kernel cost. The M5 comparison also closes with a symmetry: Metal never had this bug class because unified memory has no wrong side of the bus. Remaining distance to llama.cpp's 112 is now 1.8x, with the frame's serialized-chain structure the leading suspect and the same instruments in place
2026-08-01	**Metal's overlap schedule ported to the frame**: the cache-row rope now runs concurrently with the absorb, and the routed reduce with the shared expert's gate/up (the disjoint-buffer overlaps Metal's encoder proved), two full drains fewer per layer, verified over four suite runs (overlap bugs are race-shaped). Performance: neutral, closing the incremental ledger at ten measured experiments	The ledger's conclusion: within a single serialized command stream on this GPU, no kernel-granularity or barrier-granularity change moves the marginal token time. loom Vulkan stands at 21 tok/s end-to-end / ~31 marginal (6.8x CPU); the 3.6x to llama.cpp's 112 requires the deep restructure: an execution model that keeps many operations resident (their graph scheduler's property), which is an architecture project, not an optimization pass. Until then, the M5-beating path on commodity NVIDIA runs through bigger cards: the bandwidth-proportional share of a 3090 puts current loom kernels near the M3 bar without further kernel work
2026-08-01	**The llama.cpp thread map itself, ported and measured**: q4_k kernels rebuilt in mul_mat_vec_q4_k's exact shape: 128-thread workgroups, 16 threads per super-block (v_im/v_in), contiguous warp quant loads, four rows sharing every activation read. A word-addressing bug on the first attempt produced garbage text and 3 oracle failures; fixed, 89/89. Performance: neutral (~49 ms avg, ~20.5 end-to-end)	The dispatch-shape hypothesis joins the eliminated list. Every level llama.cpp's kernels differ at (arithmetic, loads, reduction, precision, thread map) is now ported and measured neutral inside loom's execution model, which localizes their 3.6x to the execution model itself: graph-level scheduling that keeps many operations resident where loom's strictly-barriered single chain runs each small dispatch alone. Closing it means restructuring how the frame issues work (events/split-barriers, independent-op overlap), not another kernel variant. The negative-result ledger, the profiler, and the 112 tok/s reference build are the complete inheritance
2026-08-01	**Range-gated f16, and the final elimination**: a f16-product q4_k id variant used only	This closes the shader-micro-technique ledger: NR4 +4%, 128-bit loads +6%, unpack8 0,

Date	Decision	Rationale
	where the activation is provably post-rmsnorm (expert gate/up; products ~3e2, block sums ~1e4, under f16's 65504; the down projection's SwiGLU-scaled inputs stay f32). All 89 differentials pass. Performance: neutral; Ampere executes fp16 shader ALU at 1:1 with fp32; the f16 win on NVIDIA lives in tensor cores, not compute shaders	subgroupAdd 0, f16 0 (safe form) / NaN (unsafe form). llama.cpp's 3.6x on this card therefore comes from its dispatch/tiling architecture (spec-tuned wide workgroups covering rows with high per-thread ILP, and overlapped execution without full-pipeline barriers), not from any per-instruction trick. That architectural rework (or cooperative-matrix use) is the remaining path to the measured 112, and it is a redesign, not a patch
2026-08-01	**f16 kernel arithmetic: attempted, rejected by the oracle**: the q4_k pair's products moved to float16_t (llama.cpp's fp16 mul_mat_vec variant, shaderFloat16 enabled, f32 per-block accumulation to bound error). The f64 differential answered with NaN: f16's 65504 ceiling overflows on legal inputs the oracle generates, and would overflow on any real activation spike. Reverted; perf was ~neutral in the run that completed anyway	llama.cpp ships this variant accepting the overflow risk on devices it profiles as safe; loom's verification bar does not accept a kernel that can silently produce inf on legal input. Closing the remaining 3.6x to llama.cpp's measured 112 tok/s therefore requires either per-tensor range analysis to gate f16 safely, or the non-numeric routes (spec-constant shapes, finer synchronization); a design decision recorded here rather than made silently
2026-08-01	**The ceiling, measured**: llama.cpp's own Vulkan backend, built on the same RTX 3060 against the same DeepSeek-Coder-V2-Lite Q4_K_M, decodes at 112 +/- 14 tok/s (llama-bench tg64). loom sits at ~31 marginal	This converts the open question (can Vulkan on this card beat the M5's 44?) into a proven yes with 2.5x headroom, and turns further optimization from hypothesis into diffing against a working reference on identical hardware. The major unported technique standing: f16 kernel arithmetic (FLOAT_TYPE = float16_t via shaderFloat16; all loom kernels compute f32), then spec-constant workgroup tuning and finer-grained synchronization. Setup note for reproduction: Ubuntu 22.04 lacks glslc; the LunarG jammy repo provides shaderc, vulkan-headers and spirv-headers as separate packages
2026-07-31	**subgroupAdd reductions across the dmmv family** (the second llama.cpp structural item): the shared-memory tree, six barriers per row-group, became one subgroupAdd, one subgroupElect write, one barrier, one cross-subgroup pass, sized for llvmpipe's 8-wide subgroups. Measured on the 3060: neutral (20.3 tok/s, ~31 ms marginal), 89/89 green on both platforms	Every llama.cpp mat-vec technique is now ported and individually measured on this hardware: unpack8 neutral, subgroupAdd neutral, load width +6%, NR4 +4%. The uniform residual across all quant types survives arithmetic, reduction, and load-shape changes; what has not been tried is llama.cpp's per-device spec-constant tuning of workgroup size and rows-per-workgroup, which needs vkSpecializationInfo plumbing in the pipeline layer, and their higher-occupancy shapes for small-column tensors. The scoreboard and the elimination log are the inheritance for that session
2026-07-31	**llama.cpp Vulkan analysis applied**: read ggml-vulkan's mul_mat_vec_q4_k.comp + mul_mat_vec_base.glsl at source. Three tech-	What remains from the llama.cpp playbook, now the top of the queue: subgroupAdd reductions and spec-constant workgroup shapes,

Date	Decision	Rationale
	niques identified: `unpack8` one-instruction byte unpacking, `subgroupAdd` reductions (no shared-memory tree, no barriers), and spec-constant-tuned workgroup size/NUM_ROWS. Ported the first (q4_k pair, instance bumped to Vulkan 1.2 for SPIR-V 1.5): measured neutral: the driver was already fusing the shift chains, eliminating dequant ALU from the suspect list. 89/89 green	the structural differences rather than the arithmetic ones. The per-kernel scoreboard (LOOM_VK_KERNEL_PROF) is the judge
2026-07-31	**A Vulkan-native kernel profiler, and what it ruled out**: `LOOM_VK_KERNEL_PROF` writes a timestamp query after every recorded dispatch (the stream is fully barriered, so consecutive timestamps are per-dispatch GPU durations) and prints per-pipeline totals; the profiler Nsight Compute cannot be, since it profiles CUDA only. First table: q4_k + q4_k_id = 47% of GPU time at 54-75 GB/s effective, uniform across every quant type; GPU-busy ~28 ms of the ~35 ms marginal. Acting on it, the q4_k pair moved to 128-bit uvec4 loads (the 16-byte block header is exactly one uvec4), and gained only ~6%: 20.3 -> 21.1 tok/s end-to-end, 31.8 ms marginal. Also corrected: prefill was *already* on the GPU (the deepseek engine has no stepBatch, so prompt tokens run the whole-token frame); the startup cost is the one-time 10 GB VRAM upload, not CPU prefill	Ruled out by direct measurement, in one line each: submissions (cb/tok 1.0), residency (10.1 GB VRAM), barriers (~7% by LOOM_NO_BARRIER), coalescing (tile re-grids), x-traffic (NR4), and now load width (uvec4). The uniform ~5x-off-bandwidth across all dmmv kernels with all of those eliminated points at the dequant ALU chain and workgroup shape (64 threads, deep serial per-thread loops); the next experiments are wider workgroups and subgroup reductions, and they now have a per-kernel scoreboard to be judged against
2026-07-31	**NR4 dmmv + honest accounting**: the whole dmmv family (7 quantized kernels + f32) moved to four-rows-per-workgroup (one activation load dotted against four rows' weights) and every dispatch grid to ceil(rows/4). Gain measured, small: 19.8 -> 20.3 tok/s end-to-end, 36.6 -> 35.3 ms marginal. Two negative results recorded with it: the x-traffic hypothesis behind NR4 was mostly wrong (L2 already covered the shared activations), and a LOOM_NO_BARRIER diagnostic (garbage output, valid timing) bounded all ~540 per-token barrier drains at ~7%: barriers are not the bottleneck either. README gains a benchmarks section documenting both GPU chains step by step	The FFN kernels still read at ~60 GB/s effective against 360 with occupancy, coalescing, vectorized loads, submission count and barriers all ruled out by measurement; the remaining gap needs real GPU profiling (Nsight) rather than another remote hypothesis. Negative results are logged because the next person will otherwise re-run these exact experiments. 89/89 on the 3060 and llvmpipe throughout
2026-07-31	**First real-GPU validation** (rented RTX 3060 12 GB, driver 580.173, Ubuntu 22.04): all 85 tests pass on NVIDIA silicon unmodified, and DeepSeek-Coder-V2-Lite Q4_K_M generation is token-identical to the CPU path. Then the two predicted fixes, measured: bring-up	The discrete-GPU lesson in one number: a ~1 ms submit-and-wait makes a 3.6 MB matvec a loss on a device with 15x the CPU's bandwidth. The Vulkan backend still runs attention, norms and rope on the host, so every layer round-trips PCIe; Metal's 44 tok/s came precisely from

Date	Decision	Rationale
	shape 0.6 tok/s (weights in host memory, ~500 submit-and-waits per token, 131 MiB VRAM in use); `allocDevice` + staged `upload` puts weights in VRAM and the routed-MoE chain becomes one recorded submission (`beginCmd/record/barrier/submitWait`) -> 1.5; a measured matvec size cutover (small projections to the CPU, fused MoE + 172 MB lm_head to the GPU) -> 4.3 tok/s vs 3.1 CPU-only. Cutover default 2 MB (`LOOM_VK_MIN_BYTES` overrides); sweep was flat across 2-32 MB because no matvec sits between 3.6 MB and 172 MB	eliminating those round trips (1.8 command buffers per token). The path to NVIDIA parity and beyond (the 3060's 360 GB/s is ~2.4x the M5's unified memory, a 3090's 936 GB/s ~6x) is the same port Metal already proved: attention kernels, then layer-tail, then whole-token recording. Fixed setup note: Ubuntu's unattended-upgrade replaced the NVIDIA userspace under the loaded module on first boot (Vulkan then enumerates only llvmpipe); one reboot fixes it, and the auto-update timers are disabled on the box so it cannot recur mid-benchmark
2026-07-30	`moeFfnBlockRouted`: a whole routed MoE layer (route, id-indexed gate/up, per-slot SwiGLU, id-indexed down, device-gated reduce, shared expert) as one command buffer, taking router *logits*. Local path only; the distributed path's experts are store offsets, not tensor planes, and keep host routing. Correct by a differential test against the host-routed block and by identical generated text	First measured at 0.2 tok/s against the host's 3.7; the cause was the session's recurring bug, fourth instance: `gguf run` has its own generation loops and never called `materializeArenas`, so the mappings `deepseek.load` registers were never wired and every id dispatch paid GPU page faults on file-backed memory (~285 MB/s; 33-107 ms for dispatches that cost ~0.7 ms resident). Found by splitting the fused buffer into per-phase submissions and timing each (`LOOM_FUSED_DEBUG`, kept as the instrument). With one `materializeArenas` after each `parallelBegin` the order flips: fused 1.7 tok/s against host 0.4, back to back, identical text: the first GPU path to beat the host on this model. Absolute numbers on the development machine remain unstable; the per-phase timings and the same-minute relative result are the evidence
2026-07-30	The default `--ram-gb 4.0` is machine-independent and OOM-killed the origin on an 8 GB box (7.4 GB anonymous RSS); `--ram-gb 0.5` runs stably at 0.52-1.39 GB	The RAM tier is sized by a constant rather than by what the machine has, and for an origin serving from a complete local copy it is pure waste: every shard is local, so the cache can never hit. A default that kills the process on the class of hardware the project targets is the wrong default
2026-07-29	Steady-state decode on DeepSeek-V2-Lite Q4_K_M is 128 ms/token (7.8 tok/s) on a 17 GB M5 with the 9.7 GB store device-resident: expert FFN 58.9 ms (46%), expert get 26.3 (21%), attention 17.0 (13%), submissions 6.9 (5%)	Against ~2.5 tok/s at the start of the same session, from two changes that were both about reaching the weights rather than about arithmetic: making the mapping device-resident, and adding the Q5_0/Q8_0 kernels without which every MoE layer declined to the host. The expert FFN moves ~1.1 GB per token in 58.9 ms, about 19 GB/s against a ~110 GB/s streaming ceiling: the gap is random 6 MB reads with dequantization, not the kernel, and it

Date	Decision	Rationale
		is where any further work belongs. No comparison against llama.cpp is claimed: the harness used earlier timed from first stream delta to last, which collapses when a server buffers its output, and it once reported 1236 tok/s. Those figures are withdrawn
2026-07-28	Batched prefill (`stepBatch`, `ggml.matmul`) and vectorized attention inner loops	Decoding unpacks every weight once per token, but a prompt is known up front, so one unpacked weight can serve several tokens. Measured 1.4x on a 145-token prefill; the kernel microbenchmark shows 2.4x in isolation, and the gap is attention, which is quadratic in prompt length and bound by the KV cache rather than by weight reads, so there is nothing to amortize. Vectorizing the attention dot and value-accumulation loops was worth a further 11% on decode. Batch capped at 8: past that, register pressure costs more than the sharing returns

| 2026-08-03 | Distributed speculative decoding adopted as a roadmap direction, crediting DSD (Yu et al., arXiv:2511.21669) [16]: evolve delegate-while-cold from ship-the-whole-generation into draft-local / verify-remote -- the cold node drafts a window with its MTP head plus the hot head of its store, the warm peer verifies the window in one batched forward. Adopt DSD's stabilization findings (clamped window, EMA smoothing, sticky local/delegated mode switch) and its RTT crossover as an input to the existing measured-tier-order probe. Deliberately NOT adopted: their learned window-control DNN -- the same feature vector (recent acceptance, RTT, queue depth) drives a threshold heuristic that their own ablation puts within a few percent of the learned policy | The pieces are already on the shelf: MTP drafting (2026-08-02), GEN delegation (2026-08-02), startup bandwidth probes, and per-request acceptance stats. DSD contributes the missing frame -- speculation window as a *network* decision, not an engine constant -- and evidence that the win is real at exactly the RTTs the devnet measures (10-30 ms). A deterministic controller keeps the policy debuggable and honors the code-over-model rule | | 2026-08-02 | gpt-oss engine support (OpenAI gpt-oss-20b/120b), the extreme-sparsity devnet candidate: the GQA engine gains the gpt-oss family -- alternating sliding-window(128)/full attention layers, per-head attention sinks folded into the softmax denominator, per-expert gate/up/down FFN biases, the clamped `swiglu_oai` activation (alpha 1.702, limit 7), a router that selects top-4 on raw biased logits then softmaxes only the selected four, YaRN-scaled NEOX rope with the mscale-squared logit multiplier, and the o200k Harmony chat template. Ground truth read from the real MXFP4 GGUF header and llama.cpp's openai-moe graph, not guessed; a gpt-oss fixture (sinks, biases, window 4, YaRN keys) joins the arch suite and the batched-prefill differential test. Alongside, per user request: an RPC `{"method":"model"}` reporting the served model/arch/ctx, and the boot lines now name the model being served | gpt-oss-120b is the devnet model the bandwidth math actually wants: 117B total (too big for one commodity box, so the network is load-bearing) but only 5.1B active at top-4-of-128 -- 23x sparsity vs GLM-4.5-Air's ~9x -- putting warm-store decode in the tens of tok/s where the current devnet model measures fractions. The 20b sibling shares the architecture exactly, so oracle validation runs on an 11 GB file before any 63 GB commitment | | 2026-08-03 | DSD slice one implemented: draft-local / verify-remote delegation (roadmap 6 goes from direction to code). New `DRAFT` wire command: the cold node sends its full token-id context plus a drafted window; the warm peer greedy-verifies against the exact model and answers accepted-count + its own next token, its verification KV lane persisting across calls keyed by longest-common-prefix -- the context IS the session, no session id exists. The cold node drafts with its partial store (`State.draft_local`: routed experts read from local tiers only via `Source.getLocal`, missing experts skipped, surviving gates rescaled to full mass), so drafter quality rises as the node syncs. Window controller is DSD's own dynamic baseline, deterministic: gamma clamped [1,8], EMA(0.4) acceptance, raise above 0.75 / lower below 0.25, and a two-round sticky bail to wholesale GEN. Greedy requests only; LOOM_NO_DRAFT=1 for A/B; acceptance/gamma/bail counters in the alpha report (docs/ALPHA.md). Suite gate: the drafted loop run against a local verifier of the same model is required to produce byte-identical text to plain generation | Correctness comes from the verifier, not the drafter: every emitted token either matched the peer's greedy choice or is the peer's greedy choice,

so a 5%-held node serves *exact* answers while paying the peer only tokens (a DRAFT round is ~KBs) instead of experts (~MBs per missed layer). The learned window DNN stays unported per the 2026-08-03 adoption row; the same feature vector drives the threshold heuristic their ablation ranks within a few percent | | 2026-08-03 | HTTP-range bootstrap tier (`--bootstrap-http URL`): initial sync can pull wanted shards as parallel Range requests (6 streams) from any static mirror of the exact model file -- Hugging Face, R2/S3, one nginx -- before the p2p pass, which then fetches only what the mirror could not serve. Zero code on the mirror: the manifest already maps every shard to byte extents of the original GGUF, and `Store.writeRange` digest-verifies each shard identically to a peer fetch, so the mirror is untrusted by construction (plain http is accepted for LAN mirrors on the same grounds). A stale mirror after a hardfork fails digests loudly and the p2p pass mops up. Suite gate: an in-test HTTP range server serving a fixture GGUF must yield a fully-held, verified store. Exposed to testers as LOOM_HTTP_MIRROR in the join script | Measured on the devnet the same hour: a joiner syncing 2,736 shards from the single warm origin ran at 11 MB/s -- the origin uplink, not the joiner downlink, is the fleet bottleneck, and every concurrent join divides it. Content addressing makes the transport trust-irrelevant, so the cheapest fattest pipe wins; a CDN-backed mirror takes the origin out of the bulk path entirely and stops sync traffic from evicting the origin page cache (7 GB RAM serving a 73 GB model). The p2p plane stays authoritative for manifest, repair, heat, and the long tail | | 2026-08-03 | Split-mirror support for the HTTP bootstrap (`--bootstrap-http-split-gb G`): the mirror may host the model as fixed-size byte parts (`<url>.part-00000`, ...) that concatenate to the byte-identical file; the client maps file offset to (part, local offset) by arithmetic and a range can span a part boundary. The boundary-heavy case is a suite gate (4 KB parts against fixture extents). Devnet mirror lands on Hugging Face under this convention | Hugging Face -- the natural free CDN-backed host, where the model audience already is -- caps single files at 50 GB and the devnet's merged GGUF is 73 GB. Dumb byte-splitting preserves the byte-identity the manifest digests require (unlike gguf-split, whose parts carry their own headers), so the 50 GB cap costs one naming convention and zero verification changes | | 2026-08-03 | Gossip responses carry the responder's network_id on every AnnounceBatch entry (bug fix to the 2026-08-01 proto-v2 rollout, observed live on the devnet): `sendAnnounceBatch` had left the field at its zero default on the self entry and every relayed table entry, so the dialer's chainId filter discarded the entire response on any network with a nonzero id. A dial-out-only (NAT'd) node -- whose sole liveness-refresh path is the merge of that response, since heartbeats never touch the peer table and nobody can dial in -- decayed past PEER_TTL_NS and sat at "peers 0" indefinitely while its announces kept landing on the bootnode ("peers 1" remotely). Regression test pins that every batch entry survives the dialer's filter | The network-id gate is only sound if both directions carry the id; a filter that compares against a field the responder never sets quarantines every honest peer. The initially suspected mechanism -- TTL evicting the bootstrap address and the announce loop never re-dialing it -- was ruled out: the eager-churn table keeps stale addresses forever and the loop re-dials them every round, so no bootstrap re-dial floor was added | | 2026-08-03 | Persistent bounded fetch pool (issue #170): the per-layer expert prefetch no longer spawns and joins up to 64 OS threads per MoE layer; six persistent workers park

on a condition variable, jobs are shard ids in a bounded queue with duplicate collapse, PILOT's fire-and-forget predictions share the same pool (lossy on a full queue, as the old thread cap was), and queue overflow degrades to an inline fetch -- backpressure, not lost work. No Thread.spawn remains on the token path; workers spawn lazily on first prefetch so tests and local-only runs never own a thread | docs/BENCHMARKING.md flags prefetch thread contention as a leading suspect in the 4.4x in-engine expert-FFN bandwidth loss (issue #171): per-layer thread creation churns stacks and competes with the kernel worker pool for cores, and on the devnet the unbounded per-layer burst was also hammering the origin's connection cap. Duplicate collapse additionally stops one hot shard from being fetched N times concurrently. The compact hot-expert arena (#172) stays gated on #171's attribution profile rather than being built on the hypothesis | | 2026-08-03 | Engine-shaped FFN bench (`bench --ffn-engine`, issue #171 instrumentation) -- and its first verdict: the TLB hypothesis is dead on Apple silicon. Same kernels and shapes as `--ffn`, spread over a 4 GB slab (LOOM_BENCH_GB overrides) with three passes: sequential slots, router-like random slots, and random plus two concurrent writer threads standing in for the fetch pool. Measured on the 8-thread M-series dev machine: 21.2 / 21.2 / 21.9 / 20.5 GB/s for ring / sequential / random / random+writers -- slab size and access pattern cost nothing, concurrent writes ~6% | The 4.4x in-engine expert-FFN loss (docs/BENCHMARKING.md) is therefore NOT page-walk or slab-residency pressure, and issue #172's compact hot-expert arena is a no-go on current evidence -- it would rebuild the memory layout to fix a cost that measures at zero. The suspect that survives is what this bench still does not reproduce: file-backed mmap reads under RAM-budget pressure (page-ins, first-touch, per-access LRU and digest bookkeeping) versus the bench's warm anonymous slab. The next pass is file-backed; measure-first just paid for itself | | 2026-08-03 | DSD batch verification: the warm peer now verifies a draft window in ONE batched forward with per-lane logits capture (the MTP verify-lane machinery widened from 2 to 9 lanes), instead of one full forward per token. The first draft token is judged by the prefill logits it already had -- a rejection there costs zero forward work -- and rejected lanes leave junk KV that the next call's longest-common-prefix re-feed overwrites. Sequential fallback remains for the deepseek engine and one-token windows. The existing suite gates (draft acceptance vs stepped ground truth; drafted loop byte-identical to plain generation) now exercise the batched path | The live measurement that motivated it: 62.5% acceptance at 57% hold but 0.04 tok/s -- identical to wholesale -- because the verifier stepped sequentially at the 7 GB bootnode's disk-streaming speed. A batched window reads each weight once per window instead of once per token, so on a memory-starved verifier the dominant cost drops nearly gamma-fold; acceptance was already proven, this converts it into speed | | 2026-08-03 | `bench --ffn-file` attributes the 4.4x: file-backed mmap first-touch page-ins, case closed on the mechanism. The pass writes the expert slab to a file, maps it, and runs the same kernels warm, after an eviction attempt, and re-warmed, reporting mean AND best (best-of-N only ever times a re-touched slot) plus getrusage fault deltas. Measured on the M-series dev machine, 3 GB file: first-touch mean 1.6 GB/s with 47,520 major faults against 21.3 best and ~19-21 GB/s in every resident pass -- a 13x collapse, bracketing the recorded 4.8 GB/s in-engine number. TLB, access pattern, slab size and fetch contention all measured

~zero earlier the same day | The in-engine expert-FFN loss is demand paging: every expert whose pages left residency (LRU eviction hole-punches them) pays fault-per-page on its next read. The remediation is now data-backed and it is NOT issue #172's layout arena: asynchronous readahead when the router selects (madvise WILLNEED from the fetch pool) or read() into the cache for cold experts -- syscall readahead beats fault-per-page. Measure-first killed one plausible fix and produced a better one in a single day | | 2026-08-04 | Security P1 sweep (issues #140-#142, #157-#159; #156 was already fixed by the network_id stamp): (140) RAG ingest is gated -- with an admin token configured the bearer must match it, with none it is loopback-bind only, so a public OpenAI bind can no longer be poisoned anonymously; (141) free allowances now draw from a node-wide pool (`--free-pool`, default 50 quotas), so client-id rotation mints at most the pool instead of unlimited compute -- purchased credits unaffected; (142) client ids are capped at 256 bytes at the meter and both API boundaries, killing the O(accounts x 64KiB) memory bomb; (157) sidecars write temp+fsync+rename, persist before the first fetch and checkpoint after every tier/peer, and recovery refuses to recreate (i.e. truncate model.gguf) on anything but file-not-found -- torn metadata now fails loudly instead of destroying every downloaded range; (158) verification and eviction metadata (`verified`+generation, `last_use`+clock) are guarded by one store mutex with the digest hash kept outside the lock and a mid-hash generation check discarding stale publishes; (159) light-node failover is at-most-once -- a failure after request bytes are committed returns AmbiguousBackendFailure instead of replaying against the next backend and double-charging | The free-pool design keeps the alpha's zero-friction onboarding (a real user still gets a full quota) while making rotation a bounded, observable event rather than an infinite faucet; the sidecar rules follow the same fail-loud principle as the sync-failure banner: silent destructive recovery is the worst kind of helpful. Suite gates: pool-bounded rotation, oversized-id rejection, and corrupt-metadata-refuses-truncation | | 2026-08-04 | Security P2 sweep (#143, #144, #160, #162, #163, #164, #165): the OpenAI parser caps header count and bytes and rejects Transfer-Encoding framing it does not implement; CORS is off by default and opt-in per origin (`--cors-origin`) instead of a blanket * on a bearer-metered API; request validation stops failing open -- bad JSON, non-object bodies, wrong-typed or out-of-range parameters and empty prompts are 400s, and the response's model identity is always the node's loaded model (a mismatched request `model` is 404, never echoed). Unauthenticated JOIN gets a process-wide token bucket, and hearsay peer holdings no longer steer token-loop fetches -- a relayed claim must survive a first-hand dial before it can attract traffic. The admin token can come from a permission-checked file (`--admin-token-file`) instead of argv. The native checkpoint parser requires the index count to equal the config's expert count, an exact manifest length, an exact dense-blob size, and every expert entry to name an in-bounds extent of the declared length, with checked arithmetic and dimension ceilings behind it. Activation quantizers saturate instead of trapping on NaN/Inf. CI/release actions are pinned to commit SHAs | These are the failure modes that turn "someone can reach the port" or "someone can hand us a file" into a node that crashes, misreports what served a request, or wastes its fetch budget on addresses that never answer. The validation change is deliberately breaking: an HTTP 200 that silently generated from an empty prompt is worse than a 400, because every downstream evalu-

ation, cache and billing record inherits the lie | | 2026-08-04 | Split-GGUF consumed by merging, not by multi-file mapping (`loom gguf merge`, issue #32): every 100 GB+ published quant ships as `-0000N-of-0000C` shards (DeepSeek-V4-Flash, GLM-5.2), so loom reads the shard set and writes one canonical GGUF -- metadata copied verbatim from shard 0 minus the `split.*` keys, tensor infos re-emitted against a single aligned data section. Suite gate: a fixture is split three ways and merged back, and every tensor's bytes must be identical to the original with `split.*` gone | Loom's entire distribution model -- expert-aligned manifest, byte-range shards, digests, the HTTP mirror, peer fetch -- is defined over ONE logical file. Threading a (file, offset) pair through the manifest and the p2p wire would touch every layer to buy disk headroom on exactly one machine (the origin). Verbatim KV copying is the non-obvious requirement: the parser deliberately skips array element types the engine never reads, so re-serializing the parsed form would silently drop tokenizer metadata. Side benefit: llama.cpp's gguf-split leaves the operator's path, which is how the devnet's GLM-Air was merged by hand | | 2026-08-05 | Int8-activation MXFP4 matvec with a NEON `tbl` weight gather closes the CPU kernel gap for the MXFP4 family (gpt-oss; the MXFP4 quants of DeepSeek-V4-Flash and Ling). MXFP4 was routed through the generic dequant-to-f32 `matvecCodebook` -- the slowest kernel loom has -- and since gpt-oss is entirely MXFP4 that was the whole of a measured 2.4x CPU decode gap vs llama.cpp (the same 2.4x showed on TinyLlama Q4_K, resident, and gpt-oss-20b MXFP4). `kvalues_fp4` is already the doubled E2M1 table as integers {0,1,2,3,4,6,8,12,…} with the compensating 1/2 in the E8M0 block scale, so a weight nibble is a 16-entry i8 codebook lookup: gather with one aarch64 `tbl` instruction, quantize the activation once, one int8 dot per block. Resident microbench: 1.005 -> 0.221 ms, 4.5x, MXFP4 now level with q4_k (0.157) | The measurement discipline is the story: an SDOT rewrite of the q4_k dot was tried first and reverted as a wash (LLVM already vectorizes it), and thread-count and Metal were ruled out -- the gap was one specific kernel on one specific format. `tbl` was the unlock; a scalar codebook gather reached only 1.2x. End-to-end on a RAM-starved node stays paging-bound (kernel speed is invisible when decode waits on page-ins, the #218 lesson), so the win lands where the model is resident -- which is exactly the distributed-serving regime | | 2026-08-05 | Q2_K and Q3_K int8-activation matvecs added to the affine kernel set (`matvecQ2K`/`matvecQ3K`), plus reference dequant and golden self-consistency in the existing per-type test. These are the low-bit K-quants every large MoE ships its most aggressive size in: Qwen3-30B/235B-A3B publish a Q2_K build (169 Q2_K + 24 Q3_K expert tensors) and even the IQ2_XXS build carries one Q2_K tensor, so the missing pair blocked loading the entire quant, not just those tensors. Both are 16-wide-sub-block formats like Q6_K -- Q2_K is `d*scale*q2 - dmin*min` per sub-block (a dot plus a `sum(xq)` min-correction), Q3_K folds its hmask high bit into the weight (`w = q2 - (hmask&m ? 0 : 4)`, range −4..3) so it needs no min term and its 6-bit scales come from the same aux shuffle as llama.cpp. Resident microbench 0.287/0.286 ms, on par with q6_k (0.301); Qwen3-30B-A3B Q2_K decodes coherently end to end (`Paris... Berlin... Rome`) | The devnet target Qwen3-235B-A22B Q2_K (86 GB) reuses the qwen3moe GQA engine, so this is the only new engine-side work needed to serve it -- no new architecture. The full-model rate on the 16 GB dev box (3.1 tok/s vs llama.cpp CPU 9.2) is the #218 paging wall again, not the kernel: the resident microbench proves the kernel is level with

loom's own Q6_K, and the 10.9 GB model barely fits, so 169+24 low-bit expert tensors get demand-paged per token. The self-consistency test reuses the existing harness -- the int8 kernel decoding random-but-valid bytes must match the f32 reference dequant (itself a bit-exact mirror of llama.cpp) within the int8 activation tolerance | | 2026-08-05 | Unified Q8_K-style K-quant kernels (`matvecK256/matmulK256`): all five K-quants decode to one shape and share one integer accumulation core, with `sdot` on aarch64+dotprod. Activations quantize per-256 super-block (one scale, per-16 sub-sums), every sub-block's `scale*dot` accumulates as integers in a 4-wide partial vector, and floats are touched ONCE per super-block instead of eight times -- the per-sub-block float tail was what kept the old kernels dot-bound. The uniform decode (`KDec`: 8x32 i8 weight groups + 16 per-16 scales/mins) is what makes the batched matmul real: decode a super-block once, run every lane against it. Measured (M-series, 4 threads): matvec q2_k 0.264->0.126, q3_k 0.274->0.104, q4_k 0.141->0.079, q5_k 0.301->0.104, q6_k 0.298->0.099 (1.8-3.0x); batched x8 0.374 ms vs 0.632 unbatched. Q2_K/Q3_K/Q5_K join the batched set, so a Qwen-class MoE's expert matmuls finally amortize; x86 (AVX-only Xeon, portable path) gains 1.1-1.6x per type and batch-scales end to end (Qwen3-30B Q2_K resident: 2.8 -> 3.6 tok/s aggregate at batch 8, previously flat). Matmul stays bit-identical to the matvec by construction (shared core; integer sums are order-exact), and the bit-identity test now covers every batched K-quant | Two measurement lessons paid for this. First, both prior failed prototypes attacked the wrong term: profiling the restructure showed the reduce count and the dot were survivable -- the killer was the float tail and, in the batched form, an accumulator array indexed by a runtime lane variable that LLVM spilled to the stack (load-modify-store per sub-block per lane ate the whole amortization; a register-resident per-lane accumulator with the decode hoisted fixed it). Second, on macOS the default build routes `backend.matmul` through the Metal wrapper, which adds a near-constant ~1.4 ms at these shapes regardless of thread count -- every earlier "batching loses" measurement on the Mac was measuring the wrapper, not the CPU kernel (follow-up filed). The aarch64 `sdot` path is comptime-gated on the target's dotprod feature, so baseline release builds keep the portable path instead of an illegal instruction | | 2026-08-05 | Continuous-batching decode (`decodeBatch + moeBatchUnion`): N independent sequences advance one token per forward, every weight matmul batched across them, attention per-sequence against its own KV (`Seq`). The MoE layer routes all lanes, then walks the union of selected experts: each expert's super-blocks decode once and run as a mini-matmul over exactly the lanes that chose it -- the batch-union the whitepaper's latency-hiding section promised, now implemented. `loom gguf batchbench` measures aggregate tok/s against batch size. Verified token-identical to N single-stream decodes across llama/qwen2moe/glm4moe/gpt-oss | This is the vLLM lesson (iteration-level batching) applied at the engine layer, and it is deliberately sequenced AFTER the K-quant kernel rewrite: measured before that rewrite, batch scaling was FLAT (~2.0 tok/s aggregate at batch 1..8 on resident Qwen3-30B Q2_K) because the batched matmul did not amortize the weight unpack and the low-bit types had no batched kernel at all -- the scheduler was writing checks the kernels could not cash. With the unified kernels it scales (2.8 -> 3.6 tok/s aggregate at batch 8 on the same rig). The remaining ceiling is expert overlap: 128 experts with 8 active means unrelated lanes rarely share an expert, so the union is nearly disjoint and

the dense projections carry most of the amortization; overlap -- and the win -- grows with batch depth and with models whose expert count is smaller relative to activation | | 2026-08-06 | Devnet hardfork: the canonical model moves from GLM-4.5-Air Q4_K_M (73 GB, glm4moe) to Qwen3-30B-A3B Q2_K (unsloth/Qwen3-30B-A3B-GGUF, ~11 GB, qwen3moe); network id 1337 is unchanged. Per the roadmap's upgrade rule this is a majority hardfork on a new GGUF file: the bootnode already serves the new model -- the registry's warn-only devnet arch policy surfaced the transition as `expects arch 'glm4moe', serving 'qwen3moe'` at startup, exactly the mismatch-discovery a PoC network is for -- and this change realigns the registry, join script, and docs with what the network serves. The join script's default mirror now points straight at the upstream unsloth file: a single ~11 GB GGUF under Hugging Face's 50 GB cap, so the split-part convention (2026-08-03) is no longer needed on devnet (split default drops to 0), and a stale or mismatched mirror still fails digests loudly with the p2p pass mopping up. Supersedes the devnet entry of the 2026-08-02 registry row; testnet and mainnet are untouched | The 73 GB checkpoint made devnet heavy for exactly the audience it exists for: a joiner needed ~25 GB of disk and an origin-uplink-bound sync of hours (measured 11 MB/s, 2026-08-03) before a first token. Qwen3-30B-A3B Q2_K drops that to ~4 GB and minutes, and puts the just-landed low-bit K-quant kernels (2026-08-05) and the qwen3moe GQA engine on a live network, with 128-expert top-8 routing keeping the distributed-expert thesis honest at ~9x sparsity. The 235B sibling remains the recorded scale-up target (2026-08-05); the 30B is the same engine and quant family, joinable by anyone today | | 2026-08-06 | Devnet joiner default hold-fraction rises 0.2 -> 0.7 (join script `LOOM_HOLD` default; a bare `loom node` already defaults to 1.0). Joiner disk budget goes ~4 GB -> ~9 GB against the ~11 GB model; docs updated to say plainly that below ~0.5 a cold joiner's tokens are origin-bound while the network is small | Two live measurements forced this. The bootnode's own status line shows `UNDER-REPLICATED 4915/6262 (want R=2)`: at hold 0.2 a joiner covers so little of the corpus that nearly every shard still has a single holder -- the origin -- so the network's replication target is unmet and the origin stays a single point of failure, which contradicts the ">=2 sources per expert" principle this document states as non-negotiable. And a real WAN joiner at hold 0.2 measured 0.04 tok/s with a flat ~66% hit rate: ~130 expert misses x ~1.8 MB per token is ~230 MB of WAN transfer per token from one origin, the whitepaper's own bandwidth-table regime. At 0.7 a joiner runs most tokens locally AND materially raises replication with its first sync. The hardfork's smaller model is what makes this affordable: 0.7 of 11 GB costs less disk than 0.2 of the old 73 GB did | | 2026-08-09 | Cross-model KV cache transfer recorded as research lever 7 (roadmap section 12), gated on the 235B tier. NVIDIA's closed-form KV mapper [17] converts a small family member's KV cache into a larger member's, skipping the large model's prefill (73-98% retention on its Qwen3 pairs, 4-25x faster than re-prefill, stable multi-turn). Verified from both GGUF headers that the devnet's current and target models are a matched-KV pair (Qwen3-30B-A3B and 235B-A22B: 4 KV heads, head-dim 128; layer/query-head counts differ, which the method tolerates). Recorded uses, by leverage: cross-model DSD (full 30B as drafter, mapped KV so the 235B verifier never prefills, co-located so the transfer is a local matmul), cost-quality cascading with mid-conversation upgrade, and session continuity across the 30B-to-235B hardfork | On loom the avoided

cost is not FLOPs but the WAN expert fetches that dominate big-model prefill (measured: ~230 MB/token for a cold joiner), so the win should exceed the paper's single-box 4-25x. Deliberately a lever, not a commitment: the paper's scope is dense full-attention models, and its own matched-KV Ministral pairs collapse to 42-44% retention -- matched KV correlates with transfer but does not guarantee it -- so the MoE pair must be measured, with the paper's attention-output-cosine diagnostic (r=+0.57 with retention) as the cheap go/no-go screen and calibration run against the quantized models' actual KV. The mapper artifact (4-12 GB, immutable, content-addressable) distributes over loom's own weight layer; applying it is the batched per-head matmul the unified K-quant kernels already serve | | 2026-08-09 | Verifier-side expert selection recorded as research lever 8 (extends the DSD lever), from AcceptMoE [18] -- the first catch of the weekly research radar. DSD's batched verify materializes the expert union of the draft window; AcceptMoE weights each position's router demand by its commitment probability, self-sizes the eligible set by the demand's effective rank (root token's natural top-k always preserved), and prunes low-demand nonresident experts against the live cache under a rerouting budget. Measured on Qwen3-30B-A3B -- the devnet model -- under expert offloading: −38.6 to −48.6% expert-weight traffic from residency pruning, 2.06x end-to-end, −0.27 pp mean accuracy | The verifier union is loom's actual DSD bill: every nonresident union member is a disk page-in on the bootnode (verify measured at disk-streaming speed) or a WAN fetch on a partial holder, so cutting union traffic multiplies DSD throughput where it is weakest. Loom's chain-shaped window makes the paper's offline commitment traces unnecessary -- acceptance^depth from the gamma controller's live EMA supplies the weights -- and shard residency is already tracked per fetch. Gated honestly: masked routing is not distribution-preserving (their −0.27 pp), so it must ship opt-in with the default path keeping the drafted-loop byte-identity test; and loom's <= 9-lane window is far smaller than their 64-token trees, so the absolute saving must be re-measured in loom's regime rather than assumed | | 2026-08-09 | Lever 8 first slice implemented: opt-in commitment-weighted verifier expert masking (`--verify-expert-mask`). During a DSD batched verify, each MoE layer routes all window lanes naturally, discounts each lane's router demand by acceptance^depth (BETA 0.6, the devnet's measured draft acceptance), builds the eligible set as lane 0's natural top-k (the anchor) plus the top effective-rank experts by discounted demand, prunes low-demand NONRESIDENT members on the distributed path under the rerouting budget the mask already incurred, and re-routes every lane top-k over the masked logits. Counters record natural-union vs eligible sizes per verify, and the verifier logs the reduction | Shipped exactly as the lever's gates demanded: opt-in (a `verify_mask` State flag set only when the node passes `--verify-expert-mask`), with the default path still covered by the byte-identity test suite -- and a new suite gate proves the two safety invariants on every GQA arch: lane 0's logits are BYTE-IDENTICAL to the unmasked path (the committed token's verdict cannot change, because the anchor keeps its full natural top-k), and the eligible set never exceeds the natural union (the mask can only reduce expert traffic). The chain-shaped window made the paper's offline commitment traces unnecessary, as predicted. What remains of the exit criteria is the devnet measurement: verifier expert-fetch reduction, tok/s, and drafted-output divergence with the flag on | | 2026-08-10 | Q2_K and Q3_K Metal dmmv kernels (+ device-id variants): the devnet model's expert tensors gain a Mac GPU path. Written K-Search-style [the CUDA-to-MLX kernel-transfer methodology]: the CPU kernels' layout knowledge (per-16 scales, 4-crumbs-per-byte, hmask third bit, the 12-byte 6-bit scale shuffle) translated into the house Metal pattern (NR0=4 rows per SIMD group, lane-coalesced byte pairs, float2 dots, no dynamically-indexed locals) -- both kernels passed their dequantize-then-dot f32 oracles on the first run. Q2_K/Q3_K join dmmvFor/dmmvIdFor and the moe block, with a new suite case pinning the exact Qwen3 expert mix (Q2_K gate/up, Q3_K down, 768-wide FFN) as accepted | Landing them surfaced two pre-existing latent bugs, both fixed here. MTL_SHADER_VALIDATION caught mla_absorb reading past a stale W_k upload: mlaSetWk was first-upload-wins across the suite's shared context, so a later test dispatched an earlier test's smaller buffer with its own larger dims -- now replace-on-reupload plus a size guard at every absorb dispatch (declines to the host path). And the wrapFor cache's documented contract (wrapped memory outlives the context) is violated by tests looping over short-lived fixtures: a freed fixture's page base reused by a smaller allocation revives a stale MTLBuffer whose GPU mapping can reference the old physical pages -- well-formed wrong results, undetectable from the CPU side, appearing as a different innocent test failing per heap layout. Every metal test now resets the wrap cache first. Neither bug can bite production (weights live in long-lived registered mappings), but both could burn debugging sessions -- one already had | | 2026-08-10 | The Vulkan barrier-drain search ran on a rented RTX 3090 and closed with two measured negative results and a corrected decomposition. Instrumented first: `LOOM_FRAME_TIME` (kept, env-gated) splits each token into host record time vs submit-to-completion, and its median is the low-noise fitness the shared VM's end-to-end numbers cannot provide (llama-bench itself read +/-15% there). Truth on the 3090: record 0.75 ms, GPU frame 8.0 ms median (~125 tok/s marginal), no-barrier floor 4.9 ms -- so ~430 compute-to-compute drains cost ~3.1 ms (39%), and llama.cpp's 172 tok/s (5.8 ms, same card and model) sits just above loom's no-barrier floor. Candidate A fused the residual add into the following rmsnorm and hoisted every head norm into the previous layer's tail (2 fewer drain intervals per layer, token-identical output): 8.0 -> 8.4-9.6 ms, slower. Candidate B folded the f32 router matvec into the route workgroup (1 fewer interval): +3 ms, much slower. Both reverted; only the probe ships | The two failures teach the same lesson from opposite directions: every remaining barrier guards a true RAW edge, so interval-count reduction must serialize previously-parallel work into the surviving dispatches (a one-workgroup add_rmsnorm replaces a 32-workgroup add; a one-workgroup fused route walks 2048-element rows serially), and at this granularity the serialization loss beats the ~7 us drain saved -- the third GPU in a row confirming the 2026-08-01 ledger verdict, now with cleaner instrumentation than that verdict had. The measured path to llama.cpp parity is now quantified, not guessed: f16 kernel arithmetic attacks the 4.9 ms floor (their standing technique loom has not ported), and the layer-megakernel restructure (~7 sync points per layer, their granularity) attacks the 3.1 ms drain bill with ~8x fewer intervals while the work stays wide inside each kernel. Session traps recorded for the next renter: apt on a fresh box replaced the NVIDIA userspace under the loaded module mid-setup (the exact 2026-07-31 trap; reboot and disable unattended-upgrades before benchmarking), the GPU path is opt-in (`--gpu-ops`) and silently CPU-falls-back without

it, and `zig build test` harness mode SIGABRTs on this VM while the same binary run directly passes 122/122 (pre-existing on main, not chased) | | 2026-08-10 | f16 kernel arithmetic: measured neutral on the RTX 3090, and the finding closes the direction on this hardware class. The `dmmv_q4k_id_f16` idiom (f16 products, f32 accumulate, norm-bounded inputs only) was extended to q4_k and q6_k variants and wired at every legality-safe frame site (q/kv_a projections, gate/up, lm_head): 8.6 vs 8.6 ms median frame, exactly what GA102's 1:1 fp16:fp32 ALU rate predicts -- consumer Ampere gains nothing from half-precision products, and the variants were reverted unshipped. What ships instead: the route kernel's top-k selection parallelized (each round a 64-lane argmax with value-then-lowest-index tie order, matching the serial first-max-wins scan exactly; softmax scoring parallelized the same way), because the kernel profile showed the old kernel spending ~24 us per call with one thread walking n_expert x n_used comparisons while 63 lanes idled. Per-call GPU ticks 23.7K -> 14.0K (-41%) over 1664 calls; route+host differential and the full suite green | The f16 null result sharpens the roadmap more than a win would have: loom's no-barrier kernel floor (4.9 ms) already matches llama.cpp's implied kernel time (5.8 ms total minus ~1 ms sync), so the ENTIRE remaining 3090 gap is synchronization granularity -- the layer-megakernel restructure -- and no kernel-arithmetic project will move it. (f16 products may still pay on AMD/Intel, where fp16 is genuinely double-rate; the reverted variants live in this branch's history.) The route win is real but below this VM's +/-1.5 ms end-to-end noise floor; the per-kernel timestamp scoreboard is the evidence, the same standard the uvec4-load keep used. The rented-VM measurement lesson now stands twice: end-to-end tok/s on shared tenancy cannot resolve sub-milli effects -- trust per-dispatch GPU timestamps and interleaved medians, in that order | | 2026-08-10 | Model co-design recorded as research lever 9, and its stage 0 executed: the devnet model's activation shape, measured. The lever inverts levers 1-8: instead of optimizing loom around a given model's routing, train a model whose routing is cheap to distribute -- loom's steady state reduces to fetch-bytes/token under a cache budget, and that is a trainable property today's load-balance losses actively pessimize. Instrumentation shipped: `LOOM_EXPERT_TRACE` (one line per routed MoE layer in the GQA engine) plus `scripts/expert-trace-stats.py`. First measurement, Qwen3-30B-A3B decode, 600 tokens over 3 prompt domains: hottest 25% of experts serve 63.4% of activations (top-decile skew 3.51x over uniform -- Zipfian but bounded, the balance loss visibly at work); adjacent tokens reuse 42.7% of experts (per-layer range 8-63%); the draft/batch window union grows sublinearly (W=8 touches 45% of worst case, W=32 20%); a co-occurrence table trained on 300 tokens predicts 31.2% of the next layer's experts vs a 9.9% hot-set baseline; cache replay puts LRU at 118 MB/token fetched at a 25%-of-experts budget (20.5% miss) against 344 MB/token for a static pinned set | Three conclusions the numbers force. First, temporal locality beats static skew ~3x at every budget (LRU vs pinned), so cache policy and any trained stickiness matter more than pinning heuristics. Second, cross-layer routing is already 3x more predictable than the hot-set baseline using a table almost too small to train -- the strongest early signal that pre-gated/predictable routing (the lever's stage-2 regularizer and the speculative-prefetch lever's substrate) has real headroom. Third, the absolute bill is the argument for the lever: 118 MB/token at a generous cache budget is why partial holders feel fetch-bound, and no systems-side

lever changes the shape that produces it. Parser lesson recorded: the batched prefill path routes layer-major, which a layer-index-wrap tokenizer shreds into fragments -- the reader keeps only complete decode tokens, and prefill's cost is the batch union, measured separately. Stage 1 (policy replay ceiling) and stage 2 (router-only retraining exchange rate) are the recorded next steps; stage 0's traces are the input to both | | 2026-08-10 | Lever 9 stage 1 executed: the systems-side ceiling, measured by replay (`scripts/expert-replay-sim.py`; 3,002 decode tokens over 11 prompt domains, every number measured on the held-out second half; 1.5 MB/expert, 1 ms/layer compute, LRU at a 25%-of-experts budget = 112 MB/token demand). Cache policies: plain LRU beats decayed-LFU at every budget and global-vs-per-layer slot pools are within noise -- reactive caching is solved, and recency-based prefetch is subsumed outright (its predictions are already resident; measured zero effect). Conventional pipelined prefetch: a depth-1 lookahead is nearly worthless even for an ORACLE (~10% of stall hidden -- one layer-compute of lead cannot cover a multi-ms fetch), and the pipelined optimum sits at depth 8-16, hiding ~55% of stall at 375 MB/s. Pre-gating -- every layer's experts predicted at layer 0, so layer L's fetches get L layer-computes of lead -- is the regime change: oracle pre-gating takes a 375 MB/s partial holder from 2.9 to 8.5 tok/s and a 1250 MB/s one from 7.3 to 16.0 tok/s against a 20.8 compute floor, i.e. fetch-bound becomes nearly compute-bound. A trace-learned co-occurrence pre-gate captures only ~30% of that oracle gap. Domain diversity also flattened global skew (hottest 25% of experts: 63.4% of activations on 3 domains -> 54.6% on 11), while stickiness held at 45% | The stage's purpose was to bound what systems work alone extracts, and the bound is now sharp: with the best reactive cache and the best conventional prefetcher, a partial holder stays 2-4x below its compute floor, and the single property that closes the gap -- whole-token routing predictability -- is precisely the one that must be TRAINED into the model (pre-gated routing, stage 2/3), not engineered around it. The oracle-vs-learned-table gap is stage 2's quantified target and its go/no-go: a trained predictor (or a model trained to be predictable) must land meaningfully above the ~30% the dumb table already gets for the training case to hold. The skew-flattening under domain diversity cuts the other way from stage 0's 3-domain reading -- pinning weakens as workloads diversify, shifting yet more of the lever's value onto predictability and stickiness, the two trainable properties. Sim simplifications recorded for honesty: prefetched experts bypass cache insertion (no pollution, no reuse credit -- roughly cancelling for a ceiling estimate), and bandwidth is a flat rate with no per-fetch latency term | | 2026-08-11 | Lever 9 stage 2b: a bolt-on pre-gate predictor works -- a 12M-parameter frozen-backbone head captures 75-90% of oracle pre-gating. Experiment (`scripts/pregate-probe.py` + `pregate-dump.py`, OLMoE-1B-7B on a rented 3090, FineWeb-Edu stream, all metrics held-out): train an MLP (layer-0 hidden -> every layer's expert multi-hot) on a FROZEN model -- zero quality risk, the head is a side artifact. Accuracy scaled with data and width without saturating: 29.5% -> 48.3% -> 60.0% in-training, 69.9% on a fresh-data final pass (hot-set baseline 20.5%), and the per-layer curve RISES with depth (layer 10: 69.2% from layer-0 state alone) -- the exact opposite of the intuition that deep routing is unknowable early, and the right shape for pre-gating since deep layers get the most fetch lead. Fed back into the stage-1 replay as an aligned prediction stream (`--pregate-pred`): at 375 MB/s the head lifts a 25%-budget partial holder from

3.8 to 5.6 tok/s (oracle 6.1), at 1250 MB/s from 11.2 to 19.9 (oracle 27.0) -- 75-90% of the oracle's stall reduction, with 2.4x less wasted prefetch traffic than the co-occurrence table (35.6 vs 86.2 MB/token). Cross-model shape note: OLMoE routes much flatter than Qwen3 (hottest-25% coverage 37.5% vs 54.6%; LRU@25% miss 48.5% vs 19.6%) -- its stronger load-balance training visibly destroys cacheability, the lever's central claim shown on a second family -- yet its cross-layer structure is MORE predictable (next-layer cooc 48.1% vs 32.3%) | Two verdicts. First, the saturation hypothesis died: the first run's 29.5% looked like "post-hoc prediction plateaus low, predictability must be trained in," and doubling data twice refuted it -- the honest reading now is that unmodified MoEs already carry most of their routing information in layer-0 state, and a trivial head extracts it. Second, this changes the lever's shape: an immediately deployable systems win exists (predictor-driven prefetch needs no model change, no retraining, no distribution shift -- the head ships as a small artifact beside the model), while stage 2a (router retraining) is re-scoped to the remaining oracle margin plus the cacheability properties a predictor cannot fix (skew, stickiness). Measurement discipline note for the log: the first prediction dump was written DURING training and scored the head's infancy (the sim showed the "60%" head losing to the dumb table until the dump was redone post-training on fresh data) -- eval artifacts must come from the final model, a lesson as old as ML and still cheap to relearn. Next: the Qwen3-30B probe (4-bit backbone fits the 3090) to show transfer to the deployed model, then wiring the head into loom's fetch path for end-to-end devnet numbers | | 2026-08-11 | The pre-gate probe transfers to the deployed model: 50.9% all-layer prediction on Qwen3-30B-A3B, and depth does not decay it. Same recipe as stage 2b (frozen backbone, NF4-quantized to fit the 3090; 2048-wide head; ~1.8M FineWeb positions, single pass; all metrics held out): the head predicts 50.9% of every layer's top-8 against a 22.7% hot-set baseline, across 47 predicted layers spanning 39-65% per layer -- and the DEEPEST layer is the best-predicted (layer 47: 65.4%), with even the weakest mid-stack band (layers 18-20, 30-32, ~40%) holding 1.8x baseline. Three times the layer count of OLMoE, three times the total parameters, a quantized backbone, and the layer-0 signal still carries every layer's routing. By the OLMoE scaling curve (which sat at ~48% at this data budget and reached 69.9% with more), 50.9% is a floor, not a ceiling | This was the fork in the road, and it forked toward deployment: the head that produced these numbers is FOR the model the devnet serves, so the next step is engineering, not research -- load the head in the GQA engine (two matvecs the existing kernels already run), call it after layer 0, and hand its predictions to the fetch source's existing prefetch API with per-layer lead time; measure tok/s on a partial holder before/after. Session tax recorded honestly: the run took five attempts -- an unsloth prequantized checkpoint whose fused-expert quant state transformers 5.x silently drops (native abort; pin transformers 4.x and quantize on the fly), an HF rate-limit stall on the last shard cured by pulling it from the ModelScope mirror at 40 MB/s, a CUDA allocator fragmentation OOM (expandable_segments fixed), a host OOM because the 4-bit loader holds the whole bf16 model in RAM on a 49 GB box (a 32 GB swapfile absorbed it), and a stale automation monitor double-launching the probe and OOM-killing its twin -- kill your stale watchers when the plan changes | | 2026-08-11 | The pre-gate head is wired into the engine: --`pregate-head` loads a trained predictor and prefetches every layer's experts from layer

0. New `src/gguf/pregate.zig`: the LPG1 format (header + f32 w1/b1/w2/b2 in PyTorch Linear layout, written by `scripts/pregate-export.py`), a two-matvec forward through the existing compute backend with tanh-GELU (differs from the training-time erf GELU by <1e-3, far below top-k reordering), and first-max-wins top-k matching the router's tie order; a hand-computed differential pins load, predict, and topk. The engine hook is one line at the top of the layer loop: at layer 1, `st.x` is exactly the post-layer-0 residual stream the head was trained on, and `pregatePrefetch` hands each remaining MoE layer's predicted experts to `Source.prefetchAsync` nearest-layer-first -- the tightest deadline gets the queue's head, the deepest layers the most lead. Prefetch semantics are inherited, not invented: residency-checked, lossy on a full queue (a dropped prediction beats a stalled token path), and mispredicted fetches still persist to the store. Decode path only; prefill pays batch-union costs instead. Wired in both serving paths (`loom node`, armed before the blocking run per the v0.36.1 lesson, GQA engine only with a loud decline elsewhere) and `gguf run` on a store; the exported Qwen3 head (66 MB, 47 layers x 128 experts) is the first real artifact | This closes the loop the lever opened: PILOT (colibri's router-lookahead) had the right idea at depth 1, the stage-1 replay proved depth 1 cannot cover a multi-ms fetch, the probes proved depth-47 prediction exists in the model, and now the engine consumes it. The head is a side artifact -- no model change, no distribution shift, byte-identical output with the flag off and on (prefetch only warms caches; routing never reads predictions) -- so the flag can default on once measured. What remains is the number this was built for: before/after tok/s on a partial devnet holder with the exported head | | 2026-08-11 | Tier-3 conversion refuted by the spectrum probe: Qwen3-30B's experts are full-rank and share almost nothing. The convert-and-heal path to low-rank routed deltas (factorize experts into a shared core plus rank-r residuals, then distill-heal) hinged on one measurable property, and `scripts/expert-spectrum.py` measured it in GPU-minutes with zero training: across sampled layers 0/12/24/36/47 and all three projections, the cross-expert mean carries ~1% of expert energy (there is no shared core), and the residual SVD spectra are nearly flat -- rank 32 of 768 captures 16-19% of energy, rank 64 only 27-30%. A rank-32 conversion would discard >80% of the expert signal, far beyond what distillation can heal | The probe did exactly what the lever's staging demanded: die cheaply at the first unfavorable number, before any training spend. The finding also sharpens WHY: fully-trained experts under a balance loss diversify into genuinely independent full-rank matrices -- the same force that flattens usage (stage 0's cacheability finding) also erases low-rank structure. Consequence for the roadmap: tier-3 fetch granularity cannot be retrofitted onto an existing checkpoint; it must be imposed at pretraining time (stage 3 proper, where the deltas START low-rank), which raises stage 3's cost and keeps it gated behind stage 2a's exchange rate. The near-term path is unchanged and unblocked: the bolt-on head (tier 1) is wired and awaiting its devnet number, and stage 2a (cacheability retraining) needs no architecture change | | 2026-08-12 | The pre-gate head's devnet number: 16.1% aggregate wall-time reduction on a live 50% partial holder, at prefetch's worst-case bandwidth. Protocol built for attribution: a fresh 50%-hold store synced from the bootnode, greedy decoding (every run emits the identical 96 tokens, so the expert-access sequence is fixed), and the store restored from a snapshot before every run because fetches persist and would otherwise warm the later runs. Three A/B pairs against the

bootnode's ~2.5 MB/s uplink: 2682->1949 s (-27.3%), 2361->2334 s (-1.2%), 2419->1981 s (-18.1%); aggregate 7462->6264 s. Output byte-identical throughout; the head loaded and engaged on every pre-gated run | The regime explains both the size and the spread. On a saturated link the fetched bytes are fixed by determinism, so prefetch cannot reduce traffic; its only edge is keeping the pipe full through the round-trip gaps that sequential fetch-on-miss leaves idle, and how many gaps a baseline run has depends on peer load at that moment -- hence 27% in one pair and 1% in another. This is the mechanism's floor, measured where it has no bandwidth to work with; the stage-1 replay puts the win near 78% at 1250 MB/s, so the fast-link A/B (two nodes on a datacenter network) is the recorded next measurement and the paper's bandwidth-curve figure. Setup notes: the first sync attempt died of a full disk (the 32 GB swapfile from the probe sessions plus the 61 GB HF snapshot; WriteFailed at 1.1 GB), and the box's pkill/pgrep self-match trap claimed two more shell sessions -- bracket the pattern or match by exact process name | | 2026-08-12 | Lever 9 stage 2a: the training-side exchange rate, measured -- consistency-trained routing nearly halves the cache miss rate, at a steep small-budget perplexity price. Setup (`train_2a.py`, OLMoE-1B-7B on the 3090): retrain ONLY the 16 router matrices (2M params, backbone frozen so Delta-ppl is the honest price of the routing change), balance loss zeroed, a symmetric-KL consistency term between adjacent tokens' router distributions swept at lambda 0 / 0.5 / 2.0, 8M FineWeb tokens each; traces scored by the same scripts and token counts as every prior stage. Results against the frozen baseline (ppl 9.73, stickiness 38.0%, LRU@25% miss 48.5%, 93.2 MB/token): lambda 0 drifts to ppl 10.64 with a slightly WORSE shape on every cache metric (stickiness 35.2%, miss 51.6%); lambda 0.5 reaches ppl 11.03 with stickiness 46.0%, miss 40.2%, 77.1 MB/token; lambda 2.0 reaches ppl 12.66 with stickiness 57.4%, miss 26.9%, 51.7 MB/token -- 45% less fetch traffic | Three conclusions. The control kills the cheap story: just removing the balance loss buys nothing -- the consistency term does all the useful work, so cacheability is trainable but not free. The exchange rate at this budget is steep (-45% traffic for +30% ppl), with the obvious caveat that 8M tokens is barely a nudge for a 7B model and the price plausibly shrinks with longer healing -- the scale-up question is now precise rather than speculative. And the suggestive detail: lambda 2.0's shape lands almost exactly on Qwen3's natural statistics (coverage 53.6% vs 54.6%), as if consistency pressure recovers what OLMoE's stronger balance loss erased; coincidence or mechanism is a camera-ready question. Engineering tax recorded: the first sweep died on a dtype mismatch (fp32 router weights inside a bf16 model -- wrap the gate so it casts at its boundaries) and would have died again silently on reentrant gradient checkpointing, which skips backward for segments whose inputs carry no grad, exactly the frozen-backbone case; non-reentrant mode plus a first-step gradient-flow sanity print are now part of the recipe. Two remaining sweep-2a artifacts: retrained router checkpoints for all three lambdas on the box, aligned traces rescued nowhere yet | | 2026-08-12 | Fast-link pre-gate battery, paper ablations, and a corrected record. Fast link (Hetzner Helsinki node, measured 119 MB/s to the bootnode -- within 5% of the replay's 125 MB/s row): five snapshot-controlled pairs, baselines 277-286 s vs pre-gated 252-261 s, aggregate −9.0% with tight spread; a replication battery landed at −6.4%. Lower than the slow link's 16.1% and the decomposition says why: at high holdings on four shared vCPUs the fast link makes fetches

nearly free relative to compute, so little stall remains to hide -- the win tracks the STALL SHARE, not the bandwidth. The record correction: every battery (both boxes) ran at 70.6% holdings, not the requested 50/20 -- the bootstrap sync ignores `--hold-fraction` and pulls the network-default wanted set, with the flag surviving only as a lazy runtime eviction cap (issue #252, found by measurement). Ablations (OLMoE, fixed config, one variable each): input = embeddings 53.6% / post-layer-0 60.0% / post-layer-1 62.6% -- token identity alone carries most of the routing signal (a fairness point toward SiDA), layer 0 is the knee where context is nearly free and lead time is not yet spent; width = 512:53.6 / 1024:56.9 / 2048:60.0 / 4096:62.5, monotone with no plateau, so all reported accuracies are capacity floors; prefetch budget k = 8/12/16 reaches 19.9/22.1/23.5 tok/s at 1250 MB/s against the oracle's 27.0, at 1x/2.4x/4.1x the wasted bytes | The fast-link result is the honest kind: it contradicts the naive expectation (more bandwidth, more win) and confirms the model underneath (the win is the hidden stall share; at 70.6% holdings and a low compute ceiling there is little stall). The regime the replay predicts the big numbers for -- fast link, LOW holdings -- is precisely the configuration issue #252 makes unbuildable today, so the missing curve point is blocked on a sync-path fix rather than on more measurement. The ablation trio settles the reviewer battery: the SiDA-style static input is decent (53.6) but leaves the contextual gain on the table, capacity scaling says 69.9%/50.9% are floors, and over-fetch is a real bandwidth-for-throughput knob reaching 87% of oracle at k=16. Session lesson, filed under automation hygiene: an scp placed inside a remote heredoc runs ON the remote and fails silently -- the A3 replays ran against a script that was never there, and only the empty grep output gave it away | | 2026-08-12 | Paper reproducibility closed out: every script behind a reported number is now in `scripts/`, and the binary artifacts ship as release assets. `router-retrain.py` (the stage-2a experiment: frozen backbone, fp32-boundary gate wrapper, non-reentrant checkpointing, consistency loss, built-in ppl eval and trace dump) lands for the first time; `pregate-probe.py` and `pregate-dump.py` are updated to the versions the ablations actually ran (`--model`/`--load-4bit` for the Qwen3 transfer, `--input-layer` for the A1 ablation, `--top-pred` for the A3 budget replay). The trained heads (OLMoE 2048-wide, Qwen3 47x128 in both .pt and LPG1 form), the three retrained router checkpoints, and the gzipped activation traces attach to the v0.40.3 release, since a git repo is the wrong home for 200 MB of weights | The audit that triggered this found the paper's artifacts statement was ahead of reality: the analysis scripts were pushed PR-by-PR as they were written, but the training script only ever lived on the rented box, and the probe scripts in-repo were the pre-ablation versions -- reproducible-in-spirit is not reproducible. Rule worth keeping: before any draft claims "everything is released," diff the claim against `git ls-files` and the release asset list | | 2026-08-13 | Striped repair lands: systematic Reed-Solomon parity in the propagation plane (`stripe.zig`, `STRIPE` op), following commonware's stripes design [19]. Consecutive manifest ranges form groups of k=8 with m=2 parity pieces over GF(2^8); the Cauchy generator depends only on geometry, so every honest holder computes byte-identical parity on the fly from digest-verified reads, and only full-group holders serve (parity is a linear mix of the whole group; partial holders answer `not_held`). Bootstrap and the eager repair loop both end with a parity pass that rebuilds groups missing at most two ranges from local data plus parity rows round-robined across peers;

rebuilt ranges pass the standard manifest-digest check before touching disk. Decode is a one-shot Gauss-Jordan inversion, exhaustively tested over every 4-of-6 erasure pattern plus random k=8 erasures | The trust analysis is the decision: with fixed (non-recoded) pieces and a digest commitment the swarm already agrees on, decode-then-verify makes pollution cost one failed group and a retry, so plain RS needs none of the homomorphic-hash machinery RLNC would (principle from the 2026-07-11 row stands: nothing coded in the token loop). Scope stated honestly rather than oversold: under today's pull model a parity server necessarily holds the group's data, so the pass buys load spreading, not availability; the wire format is the foundation for the real wins -- parity-only cold holders at 1.25x overhead instead of 2x replication, and coded push rollout for version upgrades -- which reuse it unchanged | | 2026-08-13 | Issue #252 fixed: `--hold-fraction` is honored again, unblocking the low-holdings measurement regime. Root cause was not the sync at all but the registry's rejoin path: `Registry.join` returned the STORED assignment for any known address, unconditionally -- so a box that ever joined at the default fraction (0.7, hence the observed 4419/6262 = 70.6% every time) could never come back smaller, and the CLI flag survived only as the lazy runtime eviction cap. Two-sided fix: the registry now re-assigns on a rejoin whose capacity differs (old picks release their coverage points first, so a grow extends the previous set and a shrink keeps its least-covered core, with committee bookkeeping consistent), and `joinSwarm` caps the received want-set at the requested fraction client-side, so a pre-fix bootnode cannot force a larger hold either | Found by measurement, fixed at the root: the paper's fast-link battery ran at 70.6% holdings instead of the requested 50/20 because of this path, and the regime the replay predicts the largest pre-gate wins for (fast link, low holdings) was unbuildable until now. The client-side cap is deliberate defense in depth -- assignments come from whatever bootnode a swarm runs, and the joiner should not need the bootnode upgraded to control its own disk. The low-holdings fast-link battery is now unblocked and queued | | 2026-08-14 | Loom system paper planned for MLSys 2027 (deadline expected late October 2026); the evaluation batteries are scripted in advance as `scripts/sys-eval/`. Four batteries, each a committed, self-contained script encoding the snapshot-controlled A/B protocol from the pre-gating measurements: low-holdings join+sync with store snapshot (`10`/`20`, the bandwidth-holdings surface), churn with a mid-generation holder kill and repair/striped-repair accounting (`30`), and llama.cpp single-node + RPC pipeline baselines pinned to a tag (`40`). The capacity headline (E1: Qwen3-235B Q2_K, ~86 GB, pooled across 4-6 boxes none of which could hold it) is recorded as a recipe pending rehearsal of the join flow. Paper skeleton drafted with every eval slot mapped to its script | The TMLR reviewer's criticism of the pre-gating paper (live evidence thinner than the simulated regime) applies with more force to a system paper, so the eval plan is decided before the deadline pressure arrives: capacity demonstration first, baselines that include the move-activations alternative (llama.cpp RPC) the whitepaper argues against, churn measured as curves rather than anecdotes. Scripts are committed rather than improvised on rented boxes because the pregate batteries taught that lesson twice (the scp-in-heredoc silent failure, the eval-dump-mid-training scoring error) | | 2026-08-23 | Two FreeToken mechanisms adopted into the fetch path (FreeToken [20], arXiv:2608.16157 -- the same expert-cache problem one memory tier down: VRAM over host RAM where loom is

RAM over the swarm). First, bandwidth-weighted holder selection: the uniform round-robin start for peer fetches is replaced by smooth weighted round-robin over a per-peer throughput EMA (measured on every completed fetch; 1 MB/s prior for cold peers so they still get probed), applied within the committee group and within the mesh group separately so SPEC's committee-first order is untouched. This is FreeToken's q* principle -- measured bandwidths, not topology, divide miss work -- mapped to loom's actual asymmetry: heterogeneous peer links rather than PCIe-vs-host-memory contention. A datacenter peer now absorbs proportionally more of a layer's parallel misses than a residential one, and both finish together instead of the fast link idling. Second, routing-blind prefill streaming: stepBatch queues every missing expert shard of a MoE layer for async fetch before that layer's attention runs, because a prompt batch activates nearly the whole layer (stage-0 measurement: W=32 union already touches most of it), so no routing information is needed to know the fetches are wanted -- attention compute is the lead time. Lossy by design (pool overflow drops the tail, the FFN loop fetches the remainder as before); decode paths untouched, where the union is small and PILOT/pre-gating already predict it. SWRR pick and EMA fold are unit-tested for exact proportionality (3:1 bandwidth -> 3:1 first choices, deterministically) | An honest correction is part of this row: the initial read of FreeToken proposed splitting misses between local disk and peers, which does not type-check against loom's store -- held and missing shards are disjoint tiers, a miss has exactly one source class. What survives the port is the principle (measure, then divide proportionally) applied where loom genuinely has two unequal paths: peers of different speeds, and prefill's compute-vs-transfer overlap. The third FreeToken idea, remote expert execution as a miss-path alternative to fetching (ship the KB activation to the holder instead of pulling the 19 MB expert), is deliberately NOT adopted: a remote FFN output cannot be digest-verified, so it reopens per-expert compute verification (v2) and stays a recorded lever gated on the trust design, with a measured RTT-vs-transfer crossover as its go/no-go | | 2026-08-23 | Fast origin bootstrap (the second FreeToken port [20]): an origin restart adopts its previous boot's sidecar manifest instead of re-hashing the whole model file. Guard: the file's size must match the manifest and the first and last ranges must spot-verify against their recorded digests; any failure falls back to the full build, and LOOM_NO_FAST_BOOT=1 forces it. The whole-store proactive audit (audit #5 P0-5) is rescheduled, not removed: node.zig now runs auditHeld in a background thread right after serving starts, and the read path digest-verifies every shard at use regardless (a corrupt shard reads as absent, its bit clears, eager repair re-fetches). The regression test proves the semantics end to end: a mid-file byte flip survives the spot-check (so the fast path really did skip the full hash), is caught at first read, and clears its holdings bit. Also added: LOOM_NO_PREFILL_STREAM and LOOM_NO_BWRR kill switches so the batteries can attribute each fetch lever separately | The devnet bootnode spent 1-2 minutes hashing its 11 GB file on every restart before the RPC port would bind, and the 235B capacity target would push that toward 15 -- FreeToken's observation that engine startup is frequent and its cost is mostly redundant verification applies verbatim. The trust reasoning is the decision: boot-time full hashing was defense-in-depth ON TOP of per-read verification, so moving it off the critical path trades nothing except the window between boot and the background sweep, during which a tampered

shard is still caught at first use. Deliberately NOT extended to the joiner reopen path (openDir) yet: its synchronous audit also feeds sync-resume accounting, so deferring it there needs its own look -- recorded as follow-up | | 2026-08-24 | KV session reuse on the generation path (the third FreeToken port [20]): the last request's KV cache persists per engine, and the next request re-prefills only past the longest common token prefix. The mechanism is the DSD verify lane's self-keying LCP rule (2026-08-03 row), now applied to `generate` itself: a multi-turn chat extends its own history, so each turn pays prefill for the new suffix instead of the whole conversation -- on the CPU boxes the devnet runs, prefill of a grown context was the dominant per-turn cost. The last prompt token is always re-fed so logits are fresh on a full-prefix hit; a diverging prompt truncates at the divergence and re-feeds from there, so no stale conversation can leak. Off on the GPU-recorded path (device KV staleness is its own question), invalidated after MTP speculative decode (stepSpec's fed-token bookkeeping is not modeled -- the session clears rather than guesses), and LOOM_NO_KV_REUSE=1 disables it for A/B. Differential test: repeated, extended, and diverging prompts all byte-identical to fresh-state runs | Peak memory is unchanged -- the State existed for the duration of every generation already; only idle memory rises by one KV cache, which the OOM-scarred bootnode budget tolerates because the OOM killer prices peaks, not idles. The honest limitation is single-lane: one cached conversation per engine, so interleaved chats from different clients get correctness (LCP falls back to a short prefix) but no speedup -- a radix tree over multiple lanes is the known upgrade path (SGLang's design, FreeToken's prefix-tree variant) and is deliberately deferred until a workload demonstrates the need | | 2026-09-10 | Recover-LoRA adapters wired into the engine (`recover_lora.zig`, `--recover-lora`), recipe generalized from Edge0 (github.com/ Edge0-AI/Edge0). Per-expert low-rank deltas trained to recover the quality an aggressive quantization destroyed: W'x = Wx + B(Ax), rank ~8, alpha baked into B at export. The loom-shaped insight is residency arithmetic: adapters for ALL 6,144 devnet experts total ~830 MB f16 -- small enough to stay fully resident on every node while the Q2_K expert bodies keep streaming, so quality moves toward a higher-bit quant at the 2-bit fetch bill, and the adapters never touch the fetch path or the store. LRA1 file format (header caps, exact-size check), shape validation against the loaded model before attach (a stale or foreign file is refused, not trusted), applied in both the single-stream FFN and the continuous-batching union path. Differential tests pin three guarantees: zero adapters are a bit-exact no-op, nonzero adapters move the logits, and batched decode stays within tolerance of the serial path with adapters attached. Training script staged (`scripts/ recover-lora-train.py`): dequantizes the DEPLOYED GGUF's expert tensors via gguf-py into the HF model before freezing -- bnb-4bit is not a stand-in, a different rounding gives the adapters a different error to learn -- with the house recipe traps (non-reentrant check-pointing, first-step grad sanity, export from final weights only) baked in | Philosophically this is the pre-gate head's sibling and the third entry in the bolt-on-artifact family: frozen base, small trained side-artifact, one file beside the model, zero distribution-shift risk (the flag off = byte-identical old behavior). Edge0's own numbers (~3-4 points recovered on 4-bit MLX models) do NOT transfer automatically to Q2_K -- the quality-recovered-per-resident-MB curve at rank {4,8,16} on the devnet model is the experiment, staged for the next GPU rental alongside the BTX stages. Their 'prerouter' (+59% from one-step

routing prediction on SSD) is deliberately NOT adopted: it is PILOT, which loom shipped in August and measured as regime-dependent -- one step of lead covers SSD latency, not WAN fetches, which is precisely why the pregate head predicts the whole horizon; the cross-regime agreement is recorded here as supporting evidence for the pregate paper's related work |

Appendix B: Source layout

```
spec/          protocol specification (SPEC.md)
docs/          roadmap and planning
whitepaper/    this document
src/main.zig   CLI entry
src/core/      hashing/Merkle, tensor math, int4 quant, stats, iobench
src/engine/    loom-format MoE engine (MLA, router, expert cache, checkpoints)
src/gguf/      GGUF parser, GGML + codebook kernels, MLA and GQA engines, shared MoE routing, BPE
src/p2p/       distribution: wire frames, gossip, committees, sync, token-loop fetch
src/node/      daemon: node orchestration, RPC, model resolver, light node, metering
```

Appendix C: Provenance

Loom's design draws on the colibri expert-streaming technique; llama.cpp and GGML for the GGUF format, quantization layouts, and reference kernel semantics [7]; the DeepSeek V2/V3 model architecture (MLA, sparse routing) [1][2]; Ethereum networking patterns (bootnodes, ENR, gossip, committees) [12]; and content-addressed storage (Merkle verification, version-pinned manifests) [10]. The implementation is original Zig with three source dependencies, all compiled into the static binaries: blockblaz/zig-snappy [13] (wire compression), google/brotli v1.1.0 (at-rest chunk text), and zigstack/vector-index (vector similarity search, extracted from this project). FAISS, when present on a node, is loaded at runtime by vector-index and never required.